Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática




# Optimización Escalable y Descentralizada del Tráfico Urbano con Almacenamiento Verificable para Ciudades Inteligentes

**Kevin Mathias Galeano Saldivar**
**María José Duarte Kowalewski**

Proyecto de Trabajo de Grado presentado en conformidad a los requisitos para obtener el grado de Ingeniería en Informática

San Lorenzo - Paraguay

Noviembre - 2026

Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática




# Optimización Escalable y Descentralizada del Tráfico Urbano con Almacenamiento Verificable para Ciudades Inteligentes

**Kevin Mathias Galeano Saldivar**
**María José Duarte Kowalewski**

San Lorenzo - Paraguay
Noviembre - 2026

Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática




# Optimización Escalable y Descentralizada del Tráfico Urbano con Almacenamiento Verificable para Ciudades Inteligentes

**Kevin Mathias Galeano Saldivar**
**María José Duarte Kowalewski**

Asesor:
Marcos Villagra

Proyecto de Trabajo de Grado presentado en conformidad a los requisitos para obtener el grado de Ingeniería en Informática
San Lorenzo - Paraguay
Noviembre - 2026

*A nuestras familias,*
*por su apoyo incondicional y su paciencia*
*a lo largo de todos estos años.*

# AGRADECIMIENTOS

Agradecemos al Ing. PhD. Derlis Gregor y al profesor Marcos Villagra, asesor de este trabajo, por las discusiones técnicas, sus aportes, su orientación, dedicación y el apoyo brindado durante el transcurso de esta investigación.

Finalmente, y de manera muy especial, agradecemos a nuestras familias por su paciencia, su apoyo incondicional y por estar presentes en cada etapa de este proceso.

# Optimización Escalable y Descentralizada del Tráfico Urbano con Almacenamiento Verificable para Ciudades Inteligentes

Autores: Kevin Mathias Galeano Saldivar, María José Duarte Kowalewski

Asesor: Marcos Villagra

## RESUMEN

Este trabajo presenta el diseño, la implementación y la evaluación experimental de un sistema modular y descentralizado para la optimización adaptativa del control semafórico urbano y la persistencia verificable de los datos asociados. La arquitectura propuesta integra cuatro microservicios: un módulo de simulación de tráfico basado en SUMO (traffic-sim), que incorpora un submódulo de control local por presión (Max-Pressure) que reparte finamente las fases dentro del ciclo semafórico, un módulo de análisis y optimización que combina inferencia difusa tipo Mamdani con optimización por enjambre de partículas para ajustar el largo de ciclo semafórico por clúster de sensores (traffic-sync), un módulo de almacenamiento distribuido basado en IPFS y contratos inteligentes sobre una red BlockDAG (traffic-storage), y un módulo de orquestación con interfaz REST (traffic-control). Los módulos se comunican mediante APIs HTTP/JSON, lo que garantiza bajo acoplamiento y facilita la integración con sensores reales.

La evaluación experimental demuestra que el sistema escala de forma aproximadamente lineal con el número de sensores: para 2000 sensores, el ciclo completo de análisis y optimización se completó en 67,6 s ($\approx$ 0,034 s por sensor), con consumo de memoria estable entre 198 y 219 MB. En cuanto al almacenamiento verificable, las transacciones alcanzaron confirmación en la red BlockDAG en $\approx$3,5 s (subida extremo a extremo de $\approx$7 s), con un costo inferior a $\$2\times10^{-5}$ USD por operación, superando en más de cuatro órdenes de magnitud el costo operativo de Ethereum y reduciendo entre 77 y 94 % la latencia de confirmación respecto a Ethereum. Los resultados respaldan la viabilidad del sistema para despliegues municipales con recursos computacionales limitados, incluyendo hardware embebido de bajo costo. Una campaña de evaluación experimental A/B de 180 comparaciones pareadas (tres planes de tiempo fijo de referencia, seis escenarios de demanda y diez semillas aleatorias, sobre la red vial de San Lorenzo) mostró que el control jerárquico propuesto resultó favorable en 140 de 180 comparaciones (77,8 %), bajo una metodología diseñada para neutralizar el sesgo de supervivencia de las métricas de tiempo de viaje. Los hallazgos principales fueron presentados en la 2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON 2025).

**Palabras Clave**: optimización de tráfico urbano, control semafórico adaptativo, lógica difu-

sa, optimización por enjambre de partículas, IPFS, BlockDAG, almacenamiento verificable, microservicios, ciudad inteligente.

# Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments

Authors: Kevin Mathias Galeano Saldivar, María José Duarte Kowalewski

Advisor: Marcos Villagra

## ABSTRACT

This work presents the design, implementation, and experimental evaluation of a modular, decentralized system for adaptive urban traffic signal optimization and verifiable data persistence. The proposed architecture integrates four microservices: a SUMO-based traffic simulation module (*traffic-sim*) that includes a local pressure-based control submodule (Max-Pressure) fine-tuning phase allocation within the signal cycle, an analysis and optimization module combining Mamdani fuzzy inference with Particle Swarm Optimization to tune the signal cycle length per sensor cluster (*traffic-sync*), a distributed storage module built on IPFS and smart contracts over a BlockDAG network (*traffic-storage*), and an orchestration module with a REST interface (*traffic-control*). All modules communicate via HTTP/JSON APIs, ensuring loose coupling and straightforward integration with real-world sensor infrastructure.

Experimental evaluation demonstrates approximately linear scalability with the number of sensors: for 2000 sensors, the full analysis-and-optimization cycle completed in 67.6 s ($\approx$ 0,034 s per sensor), with stable memory consumption between 198 and 219 MB. For verifiable storage, transactions reached confirmation on the BlockDAG network in $\approx$3.5 s (end-to-end upload of $\approx$7 s) at a cost below $\$2\times10^{-5}$ USD per operation, an operating cost more than four orders of magnitude lower than Ethereum's, while reducing confirmation latency by 77–94 % compared to Ethereum. These results support the system's viability for municipal deployments with limited computational resources, including low-cost embedded hardware. An experimental A/B evaluation campaign of 180 paired comparisons (three fixed-time reference plans, six demand scenarios, and ten random seeds, over the San Lorenzo road network) showed that the proposed hierarchical control was favorable in 140 of 180 comparisons (77.8 %), under a methodology designed to neutralize the survivorship bias inherent to travel-time metrics. The main findings were presented at the *2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies* (CHILECON 2025).

**Keywords**: urban traffic optimization, adaptive signal control, fuzzy logic, particle swarm optimization, IPFS, BlockDAG, verifiable storage, microservices, smart city.

# ÍNDICE

Página

**1. INTRODUCCIÓN** **1**
1.1. Motivación y contexto . . . 2
1.2. Objetivos . . . 2
1.2.1. Objetivo general . . . 2
1.2.2. Objetivos específicos . . . 2
1.3. Contribuciones . . . 3
1.4. Estructura del documento . . . 4

**2. INGENIERÍA DE TRÁFICO Y CONTROL SEMAFÓRICO** **5**
2.1. Semáforos para el Control del Tránsito de Vehículos . . . 5
2.2. Semáforos de Tiempos Fijos o Predeterminados . . . 6
2.3. Semáforos Accionados por el Tránsito . . . 7
2.4. Técnicas Algorítmicas y Computacionales para Control Adaptativo . . . 8
2.5. Control por Presión Máxima (Max-Pressure) . . . 10
2.6. Parámetros Clásicos de Diseño de Tiempos Semafóricos . . . 12
2.7. Modelado del Control Semafórico como Problema de Optimización . . . 13
2.8. Coordinación de Semáforos y Progresión de Pelotones . . . 14
2.9. Consideraciones Operativas y Métricas de Desempeño . . . 15
2.9.1. Sesgo de supervivencia en la evaluación de simulaciones de tráfico . . . 16
2.10. Integración de Técnicas de Optimización y Control Difuso . . . 16

**3. VISIÓN GENERAL DEL SISTEMA** **19**
3.1. Módulos Principales . . . 19
3.2. Flujo General de Información . . . 20
3.3. Justificación de la Arquitectura . . . 21

**4. SIMULACIÓN DE TRÁFICO** **24**
4.1. Arquitectura del Módulo . . . 24
4.2. Controlador Táctico Local: Max-Pressure . . . 25
4.3. Flujo de Integración . . . 29
4.4. Escenario de Simulación y Parámetros de Detección . . . 31

**5. ALMACENAMIENTO DISTRIBUIDO Y TECNOLOGÍAS DESCENTRALIZADAS** **33**
5.1. Blockchain . . . 33
5.2. BlockDAG . . . 34
5.3. IPFS . . . 36
5.4. Interfaz del Contrato de Almacenamiento . . . 37

**6. OPTIMIZACIÓN Y SINCRONIZACIÓN DE TRÁFICO** **39**
6.1. Notación . . . 39
6.2. Lógica Difusa Mamdani . . . 41
6.3. Agrupamiento Jerárquico por Enlace de Ward . . . 42
6.4. Optimización de Enjambre de Partículas (PSO) . . . 44
6.4.1. Memoización de Resultados . . . 47
6.5. Ejemplos de Ciclos de Optimización . . . 48

**7. CONTROL Y ORQUESTACIÓN DEL SISTEMA** **52**
7.1. Arquitectura del Módulo . . . 52
7.2. Flujo de Procesamiento . . . 54
7.3. Interacción con los Módulos . . . 55
7.4. Contrato de la API REST . . . 56

**8. EVALUACIÓN EXPERIMENTAL DEL SISTEMA** **58**
8.1. Evaluación del módulo traffic-sync . . . 59
8.1.1. Entorno y Protocolo de Prueba . . . 59
8.1.2. Tiempo de Ejecución . . . 60
8.1.3. Uso de CPU y Memoria . . . 60
8.2. Evaluación del módulo traffic-storage . . . 61
8.2.1. Métricas de Transacción Base . . . 61
8.2.2. Descomposición del Pipeline de Almacenamiento . . . 63
8.2.3. Consumo de Gas y Costos Operativos . . . 63
8.3. Comparación con Tecnologías Alternativas . . . 64
8.3.1. BlockDAG frente a Ethereum . . . 64
8.4. Evaluación del control semafórico: campaña A/B . . . 64
8.4.1. Diseño experimental . . . 65
8.4.2. Escenarios de demanda . . . 65
8.4.3. Planes fijos de referencia . . . 66
8.4.4. Metodología de evaluación insesgada . . . 67
8.4.5. Resultados globales . . . 68
8.4.6. Rigor estadístico . . . 70
8.4.7. Distribución completa por semilla . . . 71
8.4.8. Grillas de veredictos . . . 71
8.4.9. Análisis de patrones . . . 74
8.5. Discusión . . . 75
8.5.1. Viabilidad del Sistema . . . 76
8.5.2. Desafíos Identificados y Limitaciones . . . 76
8.5.3. Hallazgos del control semafórico . . . 76

**9. CONCLUSIÓN** **77**
9.1. Resultados principales . . . . . 77
9.2. Divulgación científica . . . . . 79
9.3. Limitaciones . . . . . 79
9.4. Trabajo futuro . . . . . 80
9.5. Síntesis final . . . . . 80

**REFERENCIAS** **82**

# LISTA DE FIGURAS

Página


3.1. Diagrama de la arquitectura del sistema mostrando la interacción de datos entre los módulos de control, simulación (que incluye el control local Max-Pressure), sincronización y almacenamiento. . . . . 23

4.1. Arquitectura del módulo traffic-sim y sus componentes principales [44]. . . . . 26
4.2. Formación del *gridlock* por *spillback* y contramedidas del controlador. La columna izquierda es la cadena causal del bloqueo; cada recuadro sombreado señala el eslabón sobre el que interviene una contramedida. La flecha discontinua de la izquierda indica que el bloqueo se realimenta: una vez cerrado el ciclo, y con los teletransportes de SUMO deshabilitados, la red no se recupera por sí sola. . . . 28

5.1. Estructura básica de una cadena de bloques, mostrando la dependencia criptográfica entre bloques consecutivos [45]. . . . . 34
5.2. Arquitectura básica de IPFS: fragmentación de archivos, generación de CID y recuperación distribuida [8]. . . . . 36
5.3. Diagrama de secuencia del flujo de almacenamiento distribuido en traffic-storage. 37

6.1. Variable lingüística *velocidad promedio* con tres conjuntos difusos modelados mediante funciones de membresía trapezoidal, triangular y aproximadamente gaussiana [28]. . . . . 41
6.2. Cadena de inferencia difusa de Mamdani, ilustrada con el escenario de congestión severa de la Tabla 6.5. Los recuadros amarillos son datos y los azules, operaciones. 43
6.3. Esquema conceptual del enjambre PSO en un espacio de búsqueda bidimensional [7]. . . . . 45
6.4. Bucle del PSO unidimensional con memoización. Un acierto de caché entrega el ciclo memorizado en menos de un milisegundo; en caso contrario se ejecuta el enjambre hasta converger o agotar su presupuesto de iteraciones. Ambas ramas confluyen en la entrega de $C^*$ al orquestador. El tamaño del enjambre, los umbrales y la regla de reinicio figuran en la Tabla 6.4. . . . . 50
6.5. Arquitectura del módulo traffic-sync, que integra la lógica difusa Mamdani, el agrupamiento jerárquico de Ward y la optimización PSO del ciclo. La flecha discontinua marca la realimentación: la función de aptitud vuelve a invocar al sistema difuso en cada iteración del enjambre. La capa Max-Pressure local, externa a traffic-sync, recibe el presupuesto de ciclo como techo blando. . . . . 51

7.1. Arquitectura del módulo traffic-control mostrando el flujo de orquestación. El almacenamiento principal se realiza mediante IPFS y BlockDAG a través de traffic-storage, mientras que la base de datos SQL se utiliza únicamente como índice auxiliar de metadatos. . . . . 53

7.2. Diagrama de secuencia del flujo completo de procesamiento en el sistema distribuido. . . . . 56

8.1. Tiempo de ejecución del módulo traffic-sync en función de la cantidad de sensores simulados. El crecimiento aproximadamente lineal (≈0,034 s/sensor) confirma la escalabilidad del módulo. . . . . 61
8.2. Uso promedio de CPU y consumo de memoria RAM del módulo traffic-sync bajo diferentes cargas de sensores simulados. El CPU disminuye conforme aumenta la concurrencia; la memoria permanece prácticamente estable en el rango 200–220 MB. . . . . 62
8.3. Desglose del tiempo de ejecución en las operaciones de subida y descarga del módulo traffic-storage. La confirmación en la red BlockDAG domina la latencia de subida (≈50 %), mientras que la descarga se completa en ≈1,6 s sin necesidad de esperar confirmación. . . . . 63
8.4. Corridas favorables al sistema propuesto, de 10 semillas, por escenario y campaña. 69
8.5. Mediana de la variación porcentual de throughput del sistema propuesto respecto de cada plan fijo, por escenario. . . . . 69
8.6. Mediana de la variación porcentual del tiempo total de sistema del sistema propuesto respecto de cada plan fijo, por escenario. . . . . 70
8.7. Dispersión de las 180 comparaciones A/B: variación porcentual de throughput (eje horizontal) contra variación porcentual de tiempo de sistema (eje vertical), coloreada según el veredicto de la corrida. Las líneas discontinuas marcan el cero de cada eje. . . . . 72
8.8. Distribución completa, por escenario, de la variación porcentual de throughput sobre las diez semillas de la campaña MOPC 60 s. . . . . 73
8.9. Distribución completa, por escenario, de la variación porcentual del tiempo total de sistema sobre las diez semillas de la campaña MOPC 60 s, con la misma convención de caja y bigotes que la Figura 8.8. . . . . 73
8.10. Matrices de veredicto de las 180 comparaciones A/B, una por campaña de plan fijo de referencia. . . . . 74

# LISTA DE TABLAS

Página


4.1. Parámetros del controlador táctico Max-Pressure. . . . . 29
4.2. Escalas temporales del control jerárquico. Las cinco cadencias son concurrentes, no sucesivas: cada mecanismo decide sobre su propio período sin esperar a los demás. . . . . 31
4.3. Esquema del payload enviado de traffic-sim a traffic-sync ante un cuello de botella. Los campos aparecen agrupados por su nivel de anidamiento en el documento JSON. . . . . 32

5.1. Diferencias estructurales entre una blockchain lineal y un BlockDAG con consenso GHOSTDAG [9, 50]. La comparación es cualitativa: las cifras de rendimiento medidas en este trabajo se presentan en la Tabla 8.4. . . . . 35
5.2. Interfaz pública del contrato de almacenamiento. . . . . 38

6.1. Notación empleada en la formulación matemática de traffic-sync . . . . . 40
6.2. Parámetros de las funciones de membresía del sistema difuso Mamdani en traffic-sync . . . . . 42
6.3. Matriz de reglas del sistema de inferencia difusa Mamdani para nivel de congestión 44
6.4. Parámetros de configuración del algoritmo PSO en traffic-sync . . . . . 46
6.5. Resultados reales del módulo traffic-sync ante cinco escenarios de tráfico. . . . . 49

7.1. Endpoints de la API REST de traffic-control. . . . . 57

8.1. Especificaciones del entorno experimental . . . . . 60
8.2. Métricas de rendimiento del módulo traffic-storage (IPFS + BlockDAG) . . . . . 62
8.3. Consumo de gas y costo estimado por tipo de operación de almacenamiento . . . 64
8.4. Comparación técnica entre Ethereum y BlockDAG en el contexto de monitoreo urbano de tráfico. Valores de BlockDAG medidos en Primordial TestNet. . . . . 65
8.5. Especificaciones de la máquina que ejecutó la campaña A/B de control semafórico 66
8.6. Escenarios de demanda evaluados en la campaña A/B de control semafórico . . 66
8.7. Planes fijos de referencia utilizados en la campaña A/B . . . . . 66
8.8. Corridas favorables (de 10) y medianas de variación porcentual de throughput y tiempo de sistema, por escenario y campaña de comparación . . . . . 68
8.9. Mediana, por escenario y campaña, del porcentaje de vehículos pareados con mejora individual de tiempo de viaje . . . . . 68
8.10. Rigor estadístico por campaña: corridas significativas (de 60), tamaño de efecto medio y mediana del porcentaje de vehículos pareados mejorados . . . . . 71

# LISTA DE ACRÓNIMOS Y SÍMBOLOS

### Infraestructura y arquitectura de software

| | |
|---|---|
| API | *Application Programming Interface*; interfaz de programación que expone funcionalidades de un servicio para ser consumidas por otros módulos o clientes externos. |
| REST | *Representational State Transfer*; estilo arquitectónico para el diseño de APIs HTTP basado en recursos y verbos estándar. |
| HTTP | *HyperText Transfer Protocol*; protocolo de transporte sobre el que se exponen las APIs del sistema. |
| JSON | *JavaScript Object Notation*; formato de intercambio de datos estructurados utilizado en los mensajes entre módulos. |
| URL | *Uniform Resource Locator*; identificador de recursos web utilizado para apuntar a endpoints y nodos del sistema. |
| RPC | *Remote Procedure Call*; modelo de invocación remota empleado en la comunicación con nodos de la red BlockDAG. |
| SQL | *Structured Query Language*; lenguaje de consulta utilizado en la base de datos relacional de metadatos operativos. |
| CPU | *Central Processing Unit*; procesador principal del host, recurso monitoreado en los experimentos de rendimiento. |
| RAM | *Random Access Memory*; memoria principal del host, recurso monitoreado en los experimentos de escalabilidad. |
| IoT | *Internet of Things*; paradigma de dispositivos interconectados; fuente de datos real a integrar en despliegues futuros. |
| FastAPI | *FastAPI*; framework web asíncrono de Python utilizado para implementar las APIs REST de los módulos del sistema. |

### Simulación y tráfico urbano

| | |
|---|---|
| SUMO | *Simulation of Urban MObility*; plataforma de simulación de tráfico urbano de código abierto utilizada en el módulo *traffic-sim*. |

| | |
|---|---|
| TraCI | *Traffic Control Interface*; protocolo de comunicación de SUMO que permite controlar y consultar la simulación en tiempo real. |
| ITS | *Intelligent Transportation Systems*; sistemas inteligentes de transporte; marco general en el que se inscribe este trabajo. |
| MILP | *Mixed-Integer Linear Programming*; programación entera mixta; formulación clásica del problema de optimización de planes semafóricos. |
| TPS | Transacciones por segundo; métrica de rendimiento empleada para comparar el throughput de BlockDAG frente a blockchain lineal. |

**Inteligencia computacional**

| | |
|---|---|
| PSO | *Particle Swarm Optimization*; algoritmo de optimización por enjambre de partículas utilizado para seleccionar el largo de ciclo semafórico por clúster de intersecciones. |
| FIS | *Fuzzy Inference System*; sistema de inferencia difusa; implementado con modelo Mamdani para clasificar el nivel de congestión. |

**Almacenamiento descentralizado**

| | |
|---|---|
| IPFS | *InterPlanetary File System*; sistema de almacenamiento distribuido por contenido utilizado para persistir los datos de tráfico. |
| CID | *Content Identifier*; identificador criptográfico de contenido en IPFS; permite verificar la integridad de los datos almacenados. |
| DAG | *Directed Acyclic Graph*; grafo dirigido acíclico; estructura de datos base de IPFS y de las redes BlockDAG. |
| BlockDAG | Red de bloques organizada como grafo dirigido acíclico en lugar de cadena lineal; permite mayor throughput y confirmación rápida de transacciones. |
| PHANTOM / GHOSTDAG | Protocolos de consenso y ordenamiento para redes BlockDAG que definen la cadena principal y resuelven conflictos entre bloques paralelos. |
| PoW | *Proof of Work*; mecanismo de consenso basado en cómputo intensivo; referencia comparativa frente a BlockDAG. |

| | |
|---|---|
| PoS | *Proof of Stake*; mecanismo de consenso basado en participación económica; alternativa energéticamente eficiente a PoW. |
| P2P | *Peer-to-Peer*; modelo de red entre pares utilizado por IPFS y las redes BlockDAG para distribuir contenido y bloques. |
| DHT | *Distributed Hash Table*; tabla de hash distribuida que usa IPFS para localizar nodos que almacenan un CID dado. |
| Solidity | Lenguaje de programación de contratos inteligentes utilizado para implementar el registro inmutable en la red BlockDAG. |
| SHA | *Secure Hash Algorithm*; familia de funciones de resumen criptográfico empleadas en la construcción de CIDs e identificadores de bloque. |

**Símbolos matemáticos**

| | |
|---|---|
| $x_i$ | Posición de la partícula $i$ en el espacio de búsqueda del PSO; en traffic-sync representa un largo de ciclo semafórico candidato (variable escalar $C$, no un vector de tiempos por fase). |
| $v_i$ | Velocidad de la partícula $i$ en el PSO; determina la magnitud y dirección del desplazamiento en cada iteración. |
| $p_i$ | Mejor posición histórica (*personal best*) de la partícula $i$; denotada $p_{\text{best}_i}$ en la formulación de traffic-sync. |
| $g$ | Mejor posición global (*global best*) encontrada por el enjambre; denotada $g_{\text{best}}$ en la formulación de traffic-sync. |
| $w$ | Factor de inercia del PSO; controla el peso de la velocidad previa en la actualización de la trayectoria. |
| $c_1, c_2$ | Coeficientes de aceleración cognitivo y social del PSO. |
| $r_1, r_2$ | Números aleatorios uniformes en $[0, 1]$ usados en la ecuación de actualización de velocidad del PSO. |
| $C$ | Longitud de ciclo semafórico; duración total de una repetición completa de las fases. |
| $g_i$ | Tiempo de verde asignado a la fase o movimiento $i$ dentro del ciclo $C$. |
| $\mu$ | Grado de pertenencia en lógica difusa; valor en $[0, 1]$ que indica cuánto pertenece un valor a un conjunto difuso. |

| | |
|---|---|
| $\phi_j$ | Función de pertenencia $j$; define la forma del conjunto difuso (triangular, trapezoidal, gaussiana). |

**Control local por presión (Max-Pressure)**

| | |
|---|---|
| $P_i$ | Presión de la fase $i$; diferencia ponderada entre la congestión de entrada y de salida de sus carriles, más el bono de inanición; criterio de decisión del controlador Max-Pressure. |
| $q/\kappa$ | Razón de cola de un carril; cociente entre vehículos detenidos $q$ y capacidad nominal $\kappa$ (aproximada como largo del carril entre 7,5 m por vehículo), sin techo superior en 1. |
| $o_\ell$ | Ocupación física del carril de salida $\ell$; fracción $[0, 1]$ del largo del carril cubierta por vehículos. |

**Metodología de comparación A/B**

| | |
|---|---|
| $C$ | Largo de ciclo semafórico, $C \in [60, 120]$ s; variable de decisión del optimizador PSO de traffic-sync. |
| $\Delta$thr | Variación porcentual del número de vehículos que completan su viaje (*throughput*) entre las corridas A y B de una comparación. |
| $\Delta$sys | Variación porcentual del tiempo total de sistema, calculado a partir del resumen de salida de SUMO, entre las corridas A y B. |
| Banda de empate | Margen del $\pm 2\,\%$ dentro del cual una comparación A/B se considera neutra en lugar de favorable o desfavorable. |
| Guardia de throughput | Umbral del $\pm 10\,\%$ de diferencia en vehículos completados entre las corridas A y B; por encima de él, el veredicto se determina por throughput independientemente de los tiempos de viaje promedio. |
| $p$-valor | Probabilidad asociada al test de Wilcoxon de rangos con signo aplicado sobre evidencia pareada por vehículo entre las corridas A y B. |

# CAPÍTULO 1
# INTRODUCCIÓN

La congestión del tráfico urbano representa uno de los problemas más persistentes y costosos de las ciudades contemporáneas. El crecimiento sostenido del parque automotor, combinado con una infraestructura vial que raramente crece al mismo ritmo, genera demoras acumuladas que impactan directamente en la productividad económica, la calidad del aire y la calidad de vida de los habitantes [1, 2]. En ciudades de tamaño medio de América Latina, donde los recursos para grandes obras de infraestructura son escasos, la optimización del control semafórico existente se presenta como una de las intervenciones de mayor relación costo-beneficio para mejorar la fluidez del tráfico sin requerir expansión física de la red vial [3].

Los sistemas de control semafórico tradicionales basados en tiempos fijos o predeterminados presentan limitaciones fundamentales frente a la variabilidad inherente de la demanda urbana. Un plan de señales diseñado para el pico de la mañana es ineficiente en el valle del mediodía; un plan calibrado para días laborables falla en eventos o festividades. Esta rigidez operativa motiva el desarrollo de estrategias adaptativas capaces de ajustar los parámetros de señalización (duración de ciclo, distribución de tiempos verdes, desplazamientos entre intersecciones) en respuesta a las condiciones observadas en tiempo real [1, 3, 4].

La convergencia de tres tendencias tecnológicas abre nuevas posibilidades para abordar este problema. En primer lugar, la disponibilidad masiva de sensores de bajo costo y plataformas de simulación de tráfico de código abierto, como SUMO (Simulation of Urban MObility), permite evaluar y validar estrategias de control bajo condiciones controladas y reproducibles antes de su despliegue en campo [5]. En segundo lugar, las técnicas de inteligencia computacional, en particular la lógica difusa y los algoritmos de optimización metaheurística, ofrecen mecanismos para modelar el conocimiento experto sobre congestión y para buscar configuraciones semafóricas óptimas en espacios de decisión complejos, sin requerir modelos analíticos exactos de la demanda [6, 7]. En tercer lugar, las tecnologías de almacenamiento descentralizado (IPFS y

redes BlockDAG) permiten registrar los datos generados por el sistema de forma inmutable y auditable, sin depender de un operador central de confianza, lo que resulta especialmente relevante para infraestructuras de gestión pública donde la trazabilidad y la rendición de cuentas son requisitos institucionales [8, 9].

## 1.1. Motivación y contexto

En el contexto paraguayo, la gestión del tráfico urbano en ciudades intermedias y en el área metropolitana de Asunción enfrenta el desafío adicional de operar con presupuestos municipales limitados y con infraestructura tecnológica heterogénea. Las soluciones comerciales de control adaptativo, ampliamente desplegadas en ciudades de países desarrollados, suelen implicar costos de licenciamiento, dependencia de proveedores y dificultades de integración con sistemas legados, barreras que frenan su adopción a escala local. Este escenario plantea la necesidad de explorar alternativas basadas en software libre, componentes de hardware de consumo y arquitecturas abiertas que puedan desplegarse y mantenerse con los recursos técnicos disponibles en municipios de la región.

Paralelamente, la creciente adopción de paradigmas de smart city en la planificación urbana de América Latina genera demanda de soluciones que no solo optimicen parámetros operativos, sino que también garanticen la integridad y disponibilidad de los datos generados. En sistemas de gestión pública, la capacidad de auditar retrospectivamente el comportamiento del sistema (verificar qué configuraciones se aplicaron, cuándo y con qué efecto) constituye un requisito de transparencia que las plataformas centralizadas tradicionales satisfacen de forma insuficiente.

## 1.2. Objetivos

### 1.2.1. Objetivo general

Diseñar, implementar y evaluar experimentalmente un sistema modular y descentralizado para la optimización adaptativa del control semafórico urbano y la persistencia verificable de los datos asociados, utilizando técnicas de inteligencia computacional y tecnologías de almacenamiento distribuido sobre hardware de consumo estándar.

### 1.2.2. Objetivos específicos

- Desarrollar un módulo de simulación de tráfico que genere escenarios controlados y reproducibles mediante SUMO, incluyendo detección automática de cuellos de botella y exposición de métricas agregadas por intersección semafórica.
- Diseñar e implementar un módulo de análisis y optimización que clasifique el estado de congestión mediante inferencia difusa tipo Mamdani, agrupe sensores por proximidad y

ajuste, mediante optimización por enjambre de partículas (PSO), el largo de ciclo semafórico por clúster, delegando en un control local por presión (Max-Pressure) el reparto fino de las fases dentro de ese ciclo.

- Evaluar experimentalmente el desempeño del control semafórico jerárquico propuesto frente a planes de tiempo fijo de referencia representativos de la práctica vigente, mediante una campaña de comparaciones pareadas A/B sobre múltiples escenarios de demanda y semillas aleatorias, aplicando una metodología insesgada frente al sesgo de supervivencia inherente a las métricas de tiempo de viaje.
- Construir un módulo de persistencia verificable que almacene datos de tráfico y resultados de optimización en IPFS, registrando los identificadores de contenido en una red BlockDAG mediante contratos inteligentes para garantizar inmutabilidad y trazabilidad.
- Integrar los módulos anteriores mediante un servicio de orquestación con interfaz REST que coordine el flujo completo desde la recepción de observaciones hasta la confirmación del almacenamiento distribuido.
- Evaluar experimentalmente el desempeño del sistema en términos de escalabilidad, consumo de recursos, latencia de almacenamiento y costo operativo, comparando con alternativas basadas en Ethereum.

## 1.3. Contribuciones

El presente trabajo realiza las siguientes contribuciones:

1. Una arquitectura de microservicios de código abierto que integra simulación de tráfico, optimización difusa-PSO y almacenamiento descentralizado en un flujo operativo unificado, demostrada en hardware de consumo estándar.
2. Un protocolo de persistencia verificable para datos de tráfico urbano basado en IPFS y contratos inteligentes sobre BlockDAG, con confirmación en ≈3,5 s y costo inferior a $\$2\times10^{-5}$ USD por transacción, superando en más de cuatro órdenes de magnitud el costo operativo de Ethereum.
3. Caracterización empírica del perfil de escalabilidad del pipeline difuso-PSO, con crecimiento aproximadamente lineal en tiempo de procesamiento ($\approx 0{,}034$ s/sensor), estabilidad de memoria entre 198 y 219 MB y decremento de uso de CPU conforme aumenta la concurrencia, estableciendo una línea base reproducible para comparaciones futuras con alternativas de control adaptativo en hardware de consumo estándar.
4. Una campaña de evaluación experimental A/B del control semafórico jerárquico (PSO de ciclo por clúster combinado con Max-Pressure local) frente a tres planes fijos de referencia,

en seis escenarios de demanda y diez semillas aleatorias por celda (180 comparaciones en total), con una metodología insesgada frente al sesgo de supervivencia que arrojó 140 comparaciones favorables al sistema propuesto (77,8 %).

5. Un conjunto de experimentos reproducibles, con configuraciones, datos y métricas documentados, que sirven de línea base para investigaciones futuras en optimización de tráfico con requisitos de trazabilidad verificable.

Los hallazgos principales fueron publicados en el artículo *Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments*, presentado en la 2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON 2025), Valparaíso, Chile [10]. Ese trabajo previo estableció la arquitectura modular de cuatro servicios, la clasificación difusa Mamdani, el ajuste de temporización semafórica mediante PSO y el almacenamiento verificable sobre IPFS y BlockDAG, con resultados de escalabilidad hasta 2000 sensores y costo de registro despreciable, pero dejó explícitamente planteada como trabajo en curso la validación empírica sobre un mapa real de la ciudad de San Lorenzo, Paraguay, mediante simulación con SUMO. La presente tesis parte de esa base publicada y la extiende en tres direcciones: un rediseño jerárquico del control, en el que el PSO deja de ajustar directamente los tiempos de señal y pasa a decidir el largo de ciclo por clúster, complementado con una capa local de control Max-Pressure; una metodología de evaluación A/B insesgada frente al sesgo de supervivencia; y la validación empírica sobre San Lorenzo anticipada en [10], materializada aquí en una campaña de 180 comparaciones A/B.

## 1.4. Estructura del documento

El resto del documento se organiza de la siguiente manera. El Capítulo 2 presenta el marco teórico que fundamenta el trabajo: ingeniería de tráfico y control semafórico, lógica difusa y sistemas de inferencia Mamdani, optimización por enjambre de partículas, IPFS como sistema de almacenamiento distribuido por contenido, y las estructuras BlockDAG y blockchain empleadas para el registro inmutable. El Capítulo 3 describe la visión general del sistema, su arquitectura modular y el flujo de información entre componentes. Los Capítulos 4 a 7 detallan el diseño e implementación de cada módulo: simulación traffic-sim, optimización traffic-sync, almacenamiento verificable traffic-storage y orquestación traffic-control. El Capítulo 8 presenta la evaluación experimental con los resultados de escalabilidad, latencia y costo, así como la campaña A/B de evaluación del control semafórico frente a planes fijos de referencia. Finalmente, el Capítulo 9 recoge las conclusiones, limitaciones y líneas de trabajo futuro.

CAPÍTULO 2

# INGENIERÍA DE TRÁFICO Y CONTROL SEMAFÓRICO

La ingeniería de tráfico es la rama de la ingeniería del transporte que se ocupa del planeamiento, diseño, operación y gestión de instalaciones viales, con el objetivo de proporcionar un movimiento seguro y eficiente de personas y mercancías [2, 11]. Su enfoque combina el análisis de la interacción entre usuarios, vehículos e infraestructura con el estudio de parámetros macroscópicos de flujo, como volumen, velocidad y densidad, para evaluar y mejorar el desempeño de redes viales urbanas y periurbanas.

Dentro de este campo, el control semafórico constituye una de las herramientas centrales para regular el derecho de paso en intersecciones donde confluyen flujos vehiculares y peatonales conflictivos [2, 12]. El diseño de planes de señalización semafórica abarca la definición de fases, la asignación de tiempos de verde, amarillo y rojo, y la coordinación entre intersecciones vecinas, con el fin de equilibrar seguridad, capacidad y niveles de servicio. La evolución desde esquemas de tiempos fijos hacia controles actuados y adaptativos refleja la necesidad de responder a patrones de demanda cada vez más variables, especialmente en entornos urbanos complejos.

## 2.1. Semáforos para el Control del Tránsito de Vehículos

Los semáforos constituyen el principal dispositivo de control del derecho de paso en intersecciones urbanas y periurbanas con conflictos significativos de circulación vehicular y peatonal [13]. En el Manual de Carreteras del Paraguay se establece que los semáforos para el control del tránsito de vehículos deben emplearse para asignar tiempos de paso y de detención a los distintos movimientos que confluyen en la intersección, con el objetivo de reducir conflictos, ordenar las maniobras y aumentar la seguridad vial [13]. Esta función se complementa con el control de flujos peatonales y, cuando corresponde, con fases específicas para giros, carriles

exclusivos y prioridades especiales (por ejemplo, transporte público o vehículos de emergencia) [13].

Desde el punto de vista de la ingeniería de tráfico, un semáforo vehicular puede interpretarse como un sistema de control discreto en el tiempo que alterna estados de indicación verde, amarilla y roja para cada grupo de movimientos, de forma tal que en ningún instante se autoricen simultáneamente movimientos incompatibles [13, 2]. El diseño del plan de fases considera la geometría de la intersección, los volúmenes de tránsito por aproximación, la presencia de giros a la izquierda y a la derecha, los flujos peatonales y, en su caso, la coordinación con semáforos próximos en el corredor [13, 11]. Las indicaciones luminosas verde, amarilla y roja poseen significado normado asociado a las obligaciones legales de detenerse o avanzar, y se complementan con flechas direccionales cuando es necesario discriminar maniobras permitidas dentro de una misma aproximación [13].

En la normativa paraguaya se especifican además criterios de ubicación, número de caras y área de influencia de cada cabezal semafórico, de modo que las indicaciones resulten claramente visibles para todos los usuarios a los que se dirigen [13]. Estos lineamientos aseguran que el dispositivo de control implementado por el sistema desarrollado en el presente trabajo sea compatible con la infraestructura semafórica existente y respetuoso de las disposiciones vigentes en el país.

## 2.2. Semáforos de Tiempos Fijos o Predeterminados

Los semáforos de tiempos fijos o predeterminados operan con un plan de señales cuya secuencia de fases y tiempos se define previamente y se repite cíclicamente, sin interacción directa con el tránsito en tiempo real [13]. El Manual de Carreteras del Paraguay indica que estos equipos asignan a cada fase un tiempo de verde y un tiempo intermedio (amarillo y, si corresponde, tiempo de despeje en rojo) calculados a partir de volúmenes de tránsito de diseño, análisis de capacidad y niveles de servicio objetivo [13]. Los tiempos de fase se mantienen constantes durante el período de operación del plan, aunque se contempla el uso de distintos planes fijos según la franja horaria (pico de la mañana, valle, pico de la tarde, periodo nocturno) [13, 2].

La principal ventaja de los semáforos de tiempos fijos reside en su simplicidad de diseño, facilidad de implementación y previsibilidad del patrón de señales, lo que se traduce en menores requerimientos de equipamiento (no se requieren detectores de vehículos) y en un mantenimiento relativamente sencillo [2, 11]. Sin embargo, este tipo de control presenta limitaciones en entornos con variaciones significativas de la demanda, como redes urbanas con fuertes diferencias entre horas pico y valle o con eventos recurrentes que alteran los patrones de flujo, ya que el plan fijo no se adapta a la ocupación real de las aproximaciones y puede generar tiempos de verde desaprovechados o demoras excesivas en ciertas corrientes [1, 3].

Desde la perspectiva de la optimización, el diseño de planes de tiempos fijos puede formularse como un problema de programación entera mixta (Mixed-Integer Linear Programming,

MILP) en el que las variables de decisión, a saber, la duración del ciclo, los tiempos de verde por fase (splits) y, en el caso de redes coordinadas, los desplazamientos temporales entre intersecciones (offsets), son esencialmente discretas o deben satisfacer relaciones de discretización [14, 15]. En este marco, la función objetivo típicamente busca minimizar el retraso total de la red o maximizar la banda de progresión de pelotones a lo largo de un corredor, sujeta a restricciones de consistencia de ciclo, tiempos mínimos de verde, capacidad y sincronización [14, 16]. Los métodos analíticos clásicos, como las fórmulas de Webster [17], constituyen soluciones cerradas para versiones simplificadas de este problema: asumen una intersección aislada, demanda constante y condiciones de estado estacionario, permitiendo obtener expresiones analíticas para el ciclo óptimo y los splits que minimizan el retraso medio. Sin embargo, estas soluciones ignoran las interdependencias entre intersecciones, la naturaleza estocástica de los arribos y la presencia de restricciones operativas complejas [15, 1].

Cuando se considera la optimización de redes semafóricas con múltiples intersecciones coordinadas, el problema adquiere dimensiones combinatorias elevadas: el número de configuraciones posibles crece exponencialmente con el número de nodos, fases y rangos de valores admisibles para ciclos y offsets, lo que convierte al problema en NP–difícil [14, 15]. En estos casos, los métodos exactos de programación entera mixta pueden resultar computacionalmente intratables para redes de tamaño medio o grande, particularmente cuando se requiere resolver el problema en tiempo razonable para permitir ajustes frecuentes de los planes en respuesta a cambios de demanda. Por ello, la literatura ha recurrido a técnicas heurísticas y metaheurísticas, como algoritmos genéticos, simulated annealing y búsqueda local, para encontrar soluciones de buena calidad sin garantizar la optimalidad global [18, 19]. En particular, los algoritmos genéticos han mostrado eficacia para optimizar planes de tiempos fijos en condiciones de demanda sobresaturada, donde los modelos analíticos tradicionales pierden validez y se requiere exploración robusta del espacio de soluciones [18].

Diversos estudios en ingeniería de tráfico han mostrado que, en condiciones de demanda altamente variable, los esquemas de control fijo tienden a producir mayores tiempos de retraso medio, un mayor número de paradas y colas más extensas que los sistemas actuados o adaptativos [1, 3]. Este comportamiento motiva la búsqueda de estrategias de control más flexibles, como las basadas en lógica difusa o algoritmos de optimización metaheurística, que constituyen el foco del trabajo.

## 2.3. Semáforos Accionados por el Tránsito

Los semáforos accionados por el tránsito utilizan información en tiempo real captada por detectores (por ejemplo, lazos inductivos, sensores de radar o cámaras de video) ubicados en las aproximaciones para modificar la duración de las fases dentro de límites mínimos y máximos definidos [13]. El Manual de Carreteras del Paraguay distingue entre control parcialmente actuado (al menos una aproximación bajo control por detectores) y totalmente actuado (todas las aproximaciones controladas por demanda), estableciendo criterios de diseño para la ubicación

de detectores, la distancia respecto de la línea de detención y los períodos iniciales mínimos y extensiones de verde en función de la velocidad que comprende el 85 % del tránsito en el acceso [13].

En estos sistemas, el controlador decide en cada ciclo si extiende o termina el verde de una fase atendiendo a la detección de vehículos, lo que permite reducir tiempos de verde desaprovechados en movimientos con baja demanda y asignar capacidad adicional a las aproximaciones más cargadas [11, 1]. La literatura sobre control semafórico actuado y adaptativo destaca que este tipo de sistemas mejora el desempeño frente a planes fijos, especialmente en condiciones de demanda fluctuante, al disminuir el retraso medio por vehículo y el número de paradas, y al reducir la probabilidad de colas críticas en aproximaciones dominantes [1, 3].

El uso de detectores también permite implementar estrategias avanzadas como extensiones de verde para "limpiar" colas residuales, prioridad al transporte público o prioridad a vehículos de emergencia, resultando particularmente relevante en corredores urbanos con alta variabilidad de flujos [1, 20]. Estos conceptos constituyen la base sobre la cual se articula el sistema de control inteligente propuesto en este trabajo, donde un controlador difuso tipo Mamdani y algoritmos de optimización como Particle Swarm Optimization (PSO) se integran para ajustar dinámicamente parámetros de operación semafórica en respuesta a condiciones de tráfico cambiantes; el enfoque híbrido resultante se describe en detalle en la Sección 2.10.

## 2.4. Técnicas Algorítmicas y Computacionales para Control Adaptativo

La creciente disponibilidad de sensores de tráfico en tiempo real, el incremento de la capacidad de procesamiento computacional y los avances en inteligencia artificial han posibilitado el desarrollo de estrategias de control semafórico que van más allá de los enfoques clásicos basados en planes de tiempo fijo o en lógicas actuadas simples [1, 3]. Estas técnicas algorítmicas y computacionales permiten abordar el problema de optimización de tiempos semafóricos en condiciones de demanda estocástica y no estacionaria, superando las limitaciones de los métodos analíticos tradicionales que asumen flujos uniformes y equilibrio de estado [1, 4].

Entre las técnicas más consolidadas se encuentran los sistemas de control adaptativo en tiempo real, como SCOOT (Split, Cycle and Offset Optimization Technique) y SCATS (Sydney Coordinated Adaptive Traffic System), que ajustan automáticamente los parámetros de señalización a partir de información de ocupación y flujo captada por detectores instalados en las aproximaciones a las intersecciones [21, 22]. SCOOT, desarrollado en el Reino Unido, emplea modelos de tráfico microscópicos para predecir la llegada de pelotones y ajusta ciclo, splits y offsets de forma continua, minimizando una función de costo basada en demoras y paradas [21, 1]. SCATS, implementado originalmente en Australia, utiliza un enfoque de biblioteca de planes (plan library) en el que un controlador central selecciona dinámicamente el plan más apropiado de un conjunto predefinido en función de las condiciones de tráfico observadas, y

ajusta los tiempos de verde dentro de límites establecidos [22, 20]. Ambos sistemas han demostrado reducciones significativas en demoras y emisiones en redes urbanas de mediana y gran escala, aunque requieren infraestructura de detección extensa y calibración cuidadosa de sus parámetros operativos.

Paralelamente, el campo de la optimización metaheurística ha aportado algoritmos capaces de explorar eficientemente el espacio de soluciones de problemas combinatorios complejos, evitando quedar atrapados en óptimos locales. Los algoritmos genéticos (Genetic Algorithms, GA) emulan procesos de evolución biológica (selección, cruce y mutación) para hacer evolucionar poblaciones de soluciones candidatas (configuraciones de ciclo, splits y offsets) hacia regiones de alta calidad del espacio de búsqueda, y han sido aplicados con éxito a la optimización de redes semafóricas coordinadas y a la optimización multiobjetivo que balancea demoras, emisiones y equidad [19, 1]. Particle Swarm Optimization (PSO), inspirado en el comportamiento social de bandadas de aves o cardúmenes de peces, representa cada solución candidata como una "partícula" que se desplaza en el espacio de parámetros guiada por su propia experiencia (mejor posición histórica individual) y por la experiencia del conjunto (mejor posición global del enjambre), logrando convergencia rápida y robusta en problemas de optimización de tiempos semafóricos [7, 23, 24]. Otros algoritmos metaheurísticos aplicados al dominio incluyen *simulated annealing*, búsqueda tabú y optimización por colonias de hormigas, cada uno con fortalezas específicas en términos de diversidad de exploración, velocidad de convergencia y manejo de múltiples objetivos [1].

En el ámbito de la inteligencia artificial, los sistemas basados en lógica difusa (fuzzy logic) permiten incorporar conocimiento experto y razonamiento aproximado en el control de semáforos, modelando conceptos lingüísticos como "congestión alta", "demanda moderada" o "cola larga" mediante funciones de pertenencia y reglas de inferencia del tipo "si–entonces" [25, 26]. La aplicación de la lógica difusa al control de intersecciones semaforizadas fue propuesta tempranamente por Pappis y Mamdani, cuyo controlador difuso para una intersección aislada, basado en la estimación de colas a partir de detectores de lazo, constituye el antecedente directo de las arquitecturas fuzzy modernas de control de tránsito [27]. Los controladores difusos tipo Mamdani mapean variables de entrada continuas (por ejemplo, volumen, velocidad, densidad) a decisiones de control (extensión de verde, cambio de fase) de forma intuitiva y transparente, facilitando la interpretación y el ajuste de los parámetros por parte de operadores humanos [28, 29]. Las redes neuronales artificiales (Artificial Neural Networks, ANN) han sido exploradas como aproximadores universales de funciones complejas en el contexto de predicción de tráfico y control adaptativo, aunque su naturaleza de "caja negra" dificulta la interpretabilidad de las decisiones [1]. Más recientemente, las técnicas de aprendizaje por refuerzo (Reinforcement Learning, RL), en particular Q-learning y Deep Q-Networks (DQN), han ganado prominencia al permitir que un agente de control aprenda políticas óptimas de asignación de verde a partir de la interacción directa con el entorno de tráfico, sin requerir un modelo explícito del sistema y adaptándose automáticamente a patrones de demanda cambiantes [30, 4]. Estos enfoques de aprendizaje profundo han mostrado resultados prometedores en simulaciones, superando en

algunos casos a SCOOT y SCATS en escenarios de alta variabilidad, aunque aún enfrentan desafíos de estabilidad, seguridad y aceptación regulatoria para su despliegue en sistemas reales [4].

Finalmente, enfoques más radicales como los sistemas autoorganizados proponen que cada intersección opere de forma autónoma, coordinándose con sus vecinas mediante intercambio de información local y reglas de decisión descentralizadas, sin depender de un controlador central [31]. Esta perspectiva se inspira en fenómenos naturales de autoorganización y busca mayor robustez frente a fallos y escalabilidad en redes extensas, aunque plantea interrogantes sobre garantías de desempeño y estabilidad global del sistema.

En conjunto, estas técnicas algorítmicas y computacionales representan una evolución desde los paradigmas tradicionales de control fijo y actuado hacia estrategias verdaderamente adaptativas, capaces de responder de forma dinámica y autónoma a la variabilidad inherente del tráfico urbano moderno. El presente trabajo se inscribe en esta línea, empleando lógica difusa tipo Mamdani combinada con PSO; la descripción detallada del enfoque híbrido adoptado se presenta en la Sección 2.10.

La descomposición de un problema de control de tráfico en un nivel estratégico, que opera sobre una escala temporal más lenta y un alcance espacial más amplio, típicamente repartiendo capacidad o presupuesto de verde entre subredes o corredores, y un nivel táctico, que ejecuta esa asignación en tiempo real a nivel de fase individual dentro de cada intersección, constituye un patrón de control jerárquico recurrente en la literatura de señalización adaptativa; de hecho, es precisamente la arquitectura industrial que SCOOT y SCATS ya explotaban mediante planes de ciclo fijo parametrizados centralmente y ajustados localmente [21, 22]. El presente trabajo materializa el mismo patrón de dos niveles con componentes distintos: un nivel estratégico que combina clustering jerárquico de Ward y optimización PSO para repartir presupuesto de tiempo de ciclo entre clústeres de la red, y un nivel táctico que ejecuta ese presupuesto mediante el controlador por presión máxima (Max-Pressure) descrito en la Sección 2.5, evitando así tanto la rigidez del ciclo fijo de la formulación de Webster como los puntos ciegos de Max-Pressure operando sin ninguna coordinación de más alto nivel.

## 2.5. Control por Presión Máxima (Max-Pressure)

Una familia de estrategias de control semafórico de naturaleza distinta a las descritas anteriormente tiene su origen fuera del dominio del tránsito vehicular, en la teoría de control de redes de conmutación de paquetes. El concepto de *back-pressure* fue introducido para el enrutamiento y la asignación de recursos en redes de colas con restricciones, donde se demostró que una política de decisión basada exclusivamente en las longitudes de las colas locales, sin necesidad de conocer las tasas de llegada ni de resolver un modelo probabilístico del tráfico, es capaz de estabilizar el sistema (es decir, mantener las colas acotadas) para cualquier vector de demanda que se encuentre dentro de la región de capacidad de la red [32]. Esta propiedad de estabilidad garantizada sin modelo de llegadas resulta particularmente atractiva frente a los

enfoques clásicos de diseño de planes semafóricos, que requieren supuestos de estacionariedad y conocimiento a priori de los volúmenes de demanda.

La adaptación de este principio al control de intersecciones semaforizadas, conocida como Max-Pressure, formaliza la noción de presión de un movimiento vehicular como la diferencia entre la ocupación (o longitud) de la cola de entrada de dicho movimiento y una medida agregada de la ocupación de las colas de salida hacia las cuales dicho movimiento descarga vehículos, ponderada por las tasas de saturación correspondientes [33]. En cada instante de decisión, el controlador evalúa la presión total de cada fase admisible, entendida como la suma de las presiones de los movimientos que dicha fase habilita, y selecciona para servicio la fase de máxima presión. Este mecanismo de decisión es completamente descentralizado y reactivo, pues decide solo con el estado instantáneo de las colas: no requiere predicción de arribos futuros ni coordinación explícita con intersecciones vecinas, ya que toda la información necesaria (colas de entrada y de salida) se mide localmente en cada instante.

La principal fortaleza teórica de Max-Pressure reside en que, bajo los mismos supuestos generales que sustentan la teoría de back-pressure, se demuestra que la política de servir siempre la fase de máxima presión estabiliza las colas de la red completa (es decir, mantiene su crecimiento acotado en el tiempo) para toda demanda contenida en la región de capacidad de la red vial, sin necesidad de estimar un ciclo óptimo, splits u offsets de antemano [33]. Esta garantía se extiende, bajo condiciones adicionales de la geometría de la red, a arterias completas con múltiples intersecciones coordinadas de forma implícita a través de la propagación de presión entre nodos adyacentes, y ha sido validada empíricamente en despliegues sobre corredores arteriales reales, donde el controlador Max-Pressure mostró reducciones sostenidas en la longitud de colas frente a esquemas de tiempos fijos y actuados convencionales [34].

No obstante, esta garantía de estabilidad descansa sobre un supuesto que resulta central para los fines de la presente tesis: la formulación original de Max-Pressure asume colas de capacidad de almacenamiento infinita, de modo que la presión de salida de un movimiento puede crecer sin límite sin alterar la validez de la comparación entre fases. En una red vial real, sin embargo, el almacenamiento disponible en cada tramo es finito y está determinado por la longitud física del carril y la densidad de apilamiento de los vehículos. Cuando la cola de un movimiento de salida alcanza dicha capacidad física, se produce un fenómeno de *spillback*: los vehículos excedentes bloquean físicamente el tramo anterior, impidiendo el avance de movimientos que, según el cálculo de presión, deberían recibir servicio.

En consecuencia, una implementación directa de Max-Pressure puro puede asignar verde a un movimiento cuya cola de salida ya está completamente bloqueada por spillback, sin que el vehículo servido tenga posibilidad real de avanzar, o puede inducir situaciones de bloqueo mutuo (*gridlock*) entre movimientos que se descargan recíprocamente sobre tramos saturados. Esta limitación, que está ausente en el dominio de redes de conmutación de paquetes (donde los búferes pueden modelarse con capacidad arbitrariamente grande sin pérdida de generalidad) pero es crítica en el dominio vial, motiva las extensiones incorporadas al sistema de control desarrollado en este trabajo, que complementan la lógica de presión máxima con mecanismos

de detección de ocupación y compuertas de saturación para evitar la asignación de verde a movimientos efectivamente bloqueados, según se detalla en el capítulo de simulación.

## 2.6. Parámetros Clásicos de Diseño de Tiempos Semafóricos

En el diseño clásico de control semafórico, la configuración de una intersección se describe mediante un conjunto reducido de parámetros numéricos que determinan su desempeño operativo: la longitud de ciclo, la repartición de verdes por fase (splits), los tiempos de amarillo y todo–rojo, y los tiempos perdidos en cada etapa del ciclo [2, 11]. La longitud de ciclo $C$ representa la duración total de una repetición completa de las fases del semáforo, mientras que los splits determinan qué fracción de ese ciclo se asigna al verde efectivo de cada movimiento o grupo de movimientos. Los tiempos de amarillo y todo–rojo se definen para advertir el fin del derecho de paso y garantizar el despeje seguro de la intersección antes de otorgar el verde a flujos conflictivos, y los tiempos perdidos agrupan intervalos en los que, por razones de seguridad o arranque, la capacidad efectiva de la intersección es menor a la teórica [2].

Estos parámetros influyen directamente en la capacidad y el retraso en cada aproximación. Para una misma demanda, ciclos más largos tienden a aumentar la capacidad por fase pero también pueden incrementar el tiempo medio de espera si los usuarios deben esperar largos intervalos de rojo, mientras que ciclos muy cortos reducen el retraso pero pueden volverse ineficientes si el tiempo perdido ocupa una proporción significativa del ciclo [11, 1]. La capacidad se relaciona con la proporción de verde asignada a cada movimiento y con la saturación de los carriles, en tanto que el retraso medio por vehículo, el número de paradas y la longitud de colas son métricas fundamentales para evaluar la calidad de servicio de un plan de tiempos [2, 12].

Métodos de diseño tradicional, como la formulación de Webster, proporcionan expresiones para estimar un ciclo "óptimo" fijo a partir de los volúmenes de diseño y de los tiempos perdidos, buscando un compromiso entre capacidad y retraso medio [17, 12]. En la práctica, estos valores se calculan para condiciones de flujo representativas y se implementan como planes de tiempo fijo, eventualmente diferenciados por franja horaria. En contraste, el enfoque desarrollado en este trabajo utiliza un sistema de inferencia difusa tipo Mamdani y un algoritmo de optimización PSO para ajustar dinámicamente la longitud de ciclo por clúster de intersecciones en función de mediciones de tráfico en tiempo real, mientras que el reparto del verde entre fases (los *splits*) se resuelve localmente en cada intersección mediante el controlador por presión máxima descrito en la Sección 2.5, con el objetivo de reducir retrasos y mejorar el desempeño frente a condiciones de demanda variables [1, 3].

La fracción de tiempo perdido por ciclo, $(C-\sum g_i)/C$, donde $C$ es la longitud de ciclo y $\sum g_i$ la suma de los verdes efectivos asignados a cada fase, sintetiza la disyuntiva entre ciclos cortos y largos que la formulación de Webster busca resolver: para un mismo conjunto de tiempos de amarillo y todo–rojo, un ciclo más corto diluye peor esa pérdida fija entre menos segundos

totales, mientras que un ciclo más largo la diluye entre más segundos y reduce su peso relativo, a costa de un mayor retraso medio de los vehículos que deben esperar intervalos de rojo más extensos. Los tres planes de referencia empleados en la campaña experimental de esta tesis (Tabla 8.7) ilustran esta disyuntiva con valores concretos: el plan MOPC de 60 s, con 25 s de verde por fase, tiene un tiempo perdido por ciclo del 16,7 %; el plan derivado de netconvert, de 90 s con 42 s de verde, lo reduce al 6,7 %; y el ciclo largo de 120 s con 57 s de verde lo lleva al 5 %. Esta progresión muestra en la práctica por qué los métodos analíticos clásicos tienden a favorecer ciclos más largos cuando el objetivo es minimizar exclusivamente la pérdida de capacidad por tiempo perdido, aun cuando ello pueda no ser óptimo en términos de retraso medio bajo demanda variable.

## 2.7. Modelado del Control Semafórico como Problema de Optimización

El diseño y operación de sistemas de control semafórico puede formularse como un problema de optimización combinatoria en el que se busca determinar los valores de un conjunto de variables de decisión que minimicen (o maximicen) una función objetivo, respetando un conjunto de restricciones operativas y de seguridad [15, 16]. Esta perspectiva permite trascender los enfoques tradicionales basados en fórmulas empíricas y abordar de manera sistemática la complejidad inherente a redes urbanas con demanda variable y múltiples intersecciones interdependientes.

En su formulación más general, el problema de optimización de control semafórico define como variables de decisión los parámetros temporales que gobiernan la operación del sistema: la duración del ciclo $C$, los tiempos de verde efectivo asignados a cada fase o movimiento (splits $g_i$), los desplazamientos temporales entre intersecciones consecutivas (offsets $\phi_j$), y en algunos casos la secuencia misma de fases [15, 2]. Estas variables deben elegirse dentro de rangos factibles definidos por la geometría, la capacidad de las aproximaciones y los requerimientos normativos de tiempos mínimos y máximos.

La función objetivo del problema representa la métrica que se desea optimizar y puede adoptar diversas formas según el criterio de desempeño que se privilegie. Entre las formulaciones más comunes se encuentran la minimización del retraso total experimentado por todos los vehículos en la red, la minimización del número total de paradas, la maximización del throughput (número de vehículos que atraviesan el sistema por unidad de tiempo), o la minimización de emisiones contaminantes y consumo de combustible derivados de aceleraciones y frenados [15, 3, 1]. En aplicaciones recientes se han propuesto funciones objetivo multicriterio que balancean simultáneamente eficiencia operativa, equidad entre corrientes de tráfico, seguridad y sostenibilidad ambiental, empleando técnicas de optimización multiobjetivo como frentes de Pareto o funciones de utilidad ponderadas [19, 3].

El problema se completa con un conjunto de restricciones que garantizan la factibilidad y seguridad de las soluciones. Estas restricciones incluyen tiempos mínimos de verde para permitir

que los peatones crucen la intersección de forma segura, tiempos máximos de verde para evitar demoras excesivas en movimientos conflictivos, tiempos de amarillo y todo–rojo calculados en función de la velocidad de aproximación y las distancias de despeje, y restricciones de capacidad que limitan los flujos admisibles en cada carril o aproximación [2, 12]. En el caso de redes coordinadas, se añaden restricciones de consistencia de ciclo (todas las intersecciones deben operar con el mismo ciclo común o con múltiplos enteros del ciclo base) y restricciones de sincronización que determinan los valores admisibles de los offsets para lograr progresión de pelotones [35, 20].

Los métodos analíticos tradicionales, como las fórmulas de Webster [17] o el método del Highway Capacity Manual (HCM) [11], constituyen soluciones cerradas para versiones simplificadas del problema de optimización, en las que se asumen condiciones de estado estacionario, demanda uniforme y geometrías regulares. Estas formulaciones proporcionan estimaciones rápidas del ciclo óptimo y de los splits correspondientes, y resultan útiles para el diseño preliminar de planes de tiempo fijo. Sin embargo, en la práctica, las condiciones reales de tráfico urbano se caracterizan por una alta variabilidad temporal (diferencias entre horas pico y valle, fluctuaciones aleatorias de la demanda), heterogeneidad espacial (diferencias de volumen entre aproximaciones), y la presencia de eventos no recurrentes (accidentes, obras, eventos especiales) que invalidan las hipótesis de estacionariedad de los modelos analíticos [1, 3].

Esta variabilidad convierte al problema de optimización semafórica en un desafío computacional complejo, especialmente cuando se considera la operación coordinada de múltiples intersecciones a lo largo de corredores o redes completas. En estos casos, el espacio de búsqueda de soluciones crece exponencialmente con el número de intersecciones y de fases, y el problema adquiere la estructura de un problema de optimización combinatoria NP–difícil, para el cual no se conocen algoritmos exactos con tiempo de ejecución polinomial [15, 19]. Ante esta complejidad, las técnicas algorítmicas avanzadas, como algoritmos genéticos, Particle Swarm Optimization (PSO), simulated annealing y métodos de búsqueda local, se han consolidado como herramientas eficaces para explorar el espacio de soluciones, escapar de óptimos locales y encontrar configuraciones de alta calidad en tiempos de cómputo razonables [19, 1].

En síntesis, la formulación del control semafórico como problema de optimización combinatoria proporciona un marco conceptual riguroso que permite integrar distintos criterios de desempeño, considerar múltiples restricciones operativas y de seguridad, y justificar el empleo de técnicas computacionales avanzadas, como los sistemas de inferencia difusa y los algoritmos metaheurísticos, que serán presentados en secciones posteriores y que constituyen la base del sistema de control inteligente desarrollado en este trabajo.

## 2.8. Coordinación de Semáforos y Progresión de Pelotones

En redes urbanas, los semáforos rara vez operan de manera completamente aislada; por el contrario, se busca coordinar varios equipos a lo largo de un corredor para facilitar la progresión de pelotones de vehículos y reducir el número de paradas sucesivas [2, 11]. La coordinación se

logra ajustando parámetros como la longitud de ciclo común entre intersecciones, los offsets (desplazamientos temporales entre inicios de verde) y, en algunos casos, la secuencia de fases, de modo que un grupo de vehículos que parte con verde en una intersección encuentre verdes sucesivos en las siguientes, configurando la llamada "onda verde" [12, 35]. Estos esquemas buscan compatibilizar la operación de múltiples nodos de control, sacrificando a veces el óptimo local de una intersección en beneficio del desempeño global del corredor.

La coordinación introduce dependencias fuertes entre los parámetros de distintas intersecciones: cambios en la longitud de ciclo o en los splits de una intersección pueden degradar la progresión en tramos adyacentes, generando nuevas demoras o incrementando el número de paradas en el corredor [35, 11]. Por ello, el diseño de planes coordinados suele apoyarse en herramientas de simulación y optimización que consideran simultáneamente varias intersecciones, y en prácticas de monitoreo operativo que permiten ajustar offsets y ciclos en respuesta a cambios de demanda. En este contexto, la idea de un sistema de control y monitoreo distribuido, apoyado en tecnologías como BlockDAG e IPFS para registrar configuraciones y métricas de desempeño, resulta especialmente pertinente: aunque el presente trabajo se centre en la optimización a nivel de una intersección, la misma arquitectura puede escalar a corredores coordinados donde las decisiones de tiempo de verde, ciclo y offset deben gestionarse de forma conjunta y trazable.

## 2.9. Consideraciones Operativas y Métricas de Desempeño

La evaluación de un plan de tiempos semafóricos se basa en un conjunto de métricas operativas que permiten cuantificar su impacto sobre los usuarios de la vía. Entre las más utilizadas se encuentran el retraso medio por vehículo, el número de paradas, la longitud de colas y el nivel de servicio (Level of Service, LOS); en estudios recientes se añaden además indicadores de consumo de combustible y emisiones contaminantes [2, 11, 1]. El retraso medio mide la diferencia entre el tiempo de viaje real y el tiempo teórico sin interferencias, mientras que el número de paradas y la longitud de colas caracterizan la discontinuidad del flujo y la probabilidad de bloqueo de accesos o de intersecciones adyacentes [2, 12].

El nivel de servicio resume de forma cualitativa la calidad de operación en categorías que van desde "A" (pocas demoras, operación confortable) hasta "F" (congestión severa), definidas a partir de rangos de retraso medio por vehículo y otras variables [11, 12]. Estas métricas permiten comparar diferentes esquemas de control (planes fijos, control actuado, control adaptativo) bajo condiciones de demanda similares y constituyen la base para justificar cambios en la programación de semáforos desde una perspectiva de ingeniería.

### 2.9.1. Sesgo de supervivencia en la evaluación de simulaciones de tráfico

Un riesgo metodológico poco discutido explícitamente en la literatura de control de tráfico es el sesgo de supervivencia al calcular métricas de desempeño, como la demora promedio o el tiempo de viaje promedio, únicamente sobre los viajes que lograron completarse dentro de la ventana de simulación. Bajo condiciones de sobresaturación o *gridlock* parcial, una fracción de los vehículos puede quedar atrapada en la red sin llegar a destino antes de que finalice la simulación; si el análisis descarta esos viajes incompletos, la métrica agregada refleja únicamente el subconjunto de tráfico que "sobrevivió" a la congestión, sesgando la comparación a favor de cualquier controlador que, lejos de resolver la congestión, simplemente deje más vehículos varados y fuera de la muestra. La literatura metodológica que trata este problema de forma explícita y sistemática en el contexto de herramientas de microsimulación como SUMO [36] es escasa: los trabajos consultados reconocen la existencia de viajes no completados como un artefacto a reportar por separado, pero no formalizan su tratamiento como fuente de sesgo en la comparación entre políticas de control. El presente trabajo aborda explícitamente esta brecha, incorporando el conteo y tratamiento de viajes incompletos como parte del protocolo de evaluación en lugar de excluirlos silenciosamente del cálculo de métricas agregadas, según se detalla en la metodología estadística del capítulo de experimentos (Sección 8.4).

En el contexto de este trabajo, el desempeño de los planes semafóricos se evalúa a partir de indicadores derivados de la salida del sistema difuso de congestión (valor numérico de congestión y su categoría lingüística), junto con variables macroscópicas como volumen por minuto, velocidad media y densidad vehicular. El controlador difuso Mamdani proporciona una medida agregada de congestión a partir de estas variables; dicha medida integra, junto con costos de espera y de capacidad dependientes de la carga, la función objetivo con la que el algoritmo PSO selecciona la longitud de ciclo de cada clúster, mientras el reparto del verde entre fases queda a cargo del controlador local por presión máxima. La automatización de la recolección de estas métricas y su análisis en tiempo real plantea desafíos de almacenamiento, trazabilidad e integridad de datos. En sistemas de control inteligente distribuido, donde múltiples nodos (intersecciones) operan de forma coordinada, resulta fundamental contar con mecanismos que aseguren la auditabilidad de las decisiones de control y la inmutabilidad de los registros históricos de configuración y desempeño [37, 38]. Este requisito motiva el uso de arquitecturas descentralizadas como BlockDAG e IPFS, que serán analizadas en secciones posteriores.

## 2.10. Integración de Técnicas de Optimización y Control Difuso

La convergencia entre técnicas de optimización metaheurística y sistemas de inferencia difusa ha dado lugar a enfoques híbridos que combinan la capacidad de razonamiento aproximado

y manejo de incertidumbre de la lógica difusa con la potencia de búsqueda y ajuste paramétrico de algoritmos como Particle Swarm Optimization (PSO), algoritmos genéticos y otras metaheurísticas [1, 39]. En el contexto del control semafórico, estos sistemas híbridos fuzzy–metaheurísticos permiten abordar de manera integrada dos desafíos complementarios: por un lado, la captura y formalización del conocimiento experto sobre las relaciones entre variables de tráfico y decisiones de control mediante reglas lingüísticas y funciones de pertenencia difusas; por otro, la optimización automática de los parámetros del sistema difuso (formas y umbrales de las funciones de pertenencia, pesos de las reglas, ganancias de salida) de forma que se maximice una función objetivo operativa, como la minimización de retrasos o la maximización del throughput de la red [40, 41].

En un sistema de control difuso tipo Mamdani aplicado a semáforos, las funciones de pertenencia de las variables de entrada (por ejemplo, volumen vehicular, velocidad media, densidad, longitud de cola) y de salida (extensión de verde, ajuste de ciclo, cambio de fase) se definen inicialmente con base en conocimiento experto o en datos históricos, pero su sintonización fina puede requerir ajustes iterativos costosos y poco sistemáticos. PSO ofrece una alternativa eficiente y automática: cada partícula del enjambre representa una configuración específica de parámetros del sistema difuso, y la posición de la partícula se actualiza iterativamente en función de su desempeño histórico y del desempeño del mejor individuo global, explorando el espacio de configuraciones hasta encontrar un conjunto de parámetros que optimice la métrica de desempeño deseada [7, 23, 24]. Este enfoque de optimización de parámetros difusos mediante PSO ha demostrado reducir significativamente el tiempo de diseño del controlador, mejorar la robustez frente a variaciones de demanda y permitir la adaptación online de parámetros en respuesta a condiciones cambiantes [39, 40].

Extensiones más complejas incluyen sistemas neuro–fuzzy, en los que se combinan redes neuronales artificiales con lógica difusa para permitir aprendizaje automático de las reglas de inferencia y funciones de pertenencia a partir de datos, y que pueden ser entrenados o ajustados mediante metaheurísticas como PSO, algoritmos genéticos o búsqueda cuco (Cuckoo Search) [42]. Estos enfoques híbridos neuro–fuzzy–metaheurísticos aprovechan la capacidad de generalización y aprendizaje de las redes neuronales, la interpretabilidad de los sistemas difusos y la eficiencia de búsqueda de las metaheurísticas, constituyendo una vía prometedora para el diseño de controladores adaptativos en dominios complejos como el control de tráfico urbano [1, 41].

El sistema propuesto en este trabajo se inscribe en esta línea de sistemas híbridos fuzzy–PSO, si bien con una arquitectura jerárquica que separa explícitamente la escala a la que opera cada componente. La red de intersecciones se agrupa previamente en clústeres mediante técnicas de agrupamiento jerárquico aglomerativo del tipo propuesto por Ward, que minimiza el incremento de varianza intra-cluster en cada paso de fusión y produce agrupaciones compactas de intersecciones con patrones de demanda similares [43]. Sobre esta partición, un controlador difuso tipo Mamdani evalúa el nivel de congestión de cada clúster a partir de variables macroscópicas de tráfico (volumen, velocidad, densidad), y un algoritmo PSO optimiza, a nivel de

clúster, la longitud del ciclo semafórico común $C \in [60, 120]$ segundos que minimiza el índice de congestión estimado por el sistema difuso. A diferencia de un esquema fuzzy–PSO plano en el que la optimización determina directamente los tiempos de verde de cada fase, en la arquitectura adoptada el reparto del verde entre movimientos dentro de cada intersección es resuelto por una capa de control local basada en el principio de Max-Pressure descrito en la Sección 2.5, que distribuye la capacidad disponible entre las aproximaciones en función de la presión de sus colas, utilizando el ciclo optimizado por PSO como techo blando (*soft cap*) más que como una partición rígida en splits fijos. Esta separación de responsabilidades permite que la optimización metaheurística, computacionalmente más costosa, opere a la escala agregada del clúster y con una frecuencia de actualización menor, mientras que la asignación de verde a nivel de intersección responde en tiempo real a la ocupación instantánea de colas, incorporando además las salvaguardas frente a spillback discutidas anteriormente. El resultado combina el conocimiento experto y la interpretabilidad de las reglas difusas, la capacidad de búsqueda global de PSO para el parámetro de ciclo, y la garantía de estabilidad de colas del control por presión máxima a nivel local.

Más allá de las ventajas operativas del enfoque híbrido, la implementación de sistemas de control inteligente en infraestructuras críticas como redes semafóricas plantea requisitos adicionales de trazabilidad, auditabilidad e integridad de datos. En un sistema distribuido donde múltiples intersecciones operan de forma coordinada o autónoma, resulta fundamental registrar de manera inmutable y verificable las decisiones de control tomadas (tiempos de verde asignados, parámetros del controlador difuso, resultados de la optimización PSO), las condiciones de tráfico observadas (volúmenes, velocidades, densidades capturadas por sensores) y las métricas de desempeño resultantes (retrasos, emisiones, niveles de servicio). Estos registros permiten auditar el comportamiento del sistema, identificar configuraciones problemáticas, validar la conformidad con normativas y políticas de operación, y facilitar la depuración y mejora continua del controlador. Las arquitecturas descentralizadas basadas en BlockDAG (Directed Acyclic Graph de bloques) e IPFS (InterPlanetary File System) ofrecen soluciones escalables y confiables para el registro y almacenamiento distribuido de estos datos, temas que serán analizados en profundidad en secciones posteriores del presente trabajo. En síntesis, la integración de técnicas de optimización metaheurística y control difuso no solo mejora el desempeño operativo del sistema semafórico, sino que abre la puerta a arquitecturas distribuidas y auditables que responden a las demandas de transparencia y confiabilidad de las ciudades inteligentes modernas.

# CAPÍTULO 3

# VISIÓN GENERAL DEL SISTEMA

El sistema propuesto se concibe como un ecosistema modular de microservicios para el monitoreo, análisis y optimización del tráfico urbano en tiempo (casi) real, diseñado para operar en entornos de ciudad inteligente. La arquitectura se compone de cuatro módulos principales, cada uno con responsabilidades bien definidas y comunicándose mediante interfaces estandarizadas (APIs REST y mensajes JSON). Esta organización modular facilita la escalabilidad, el mantenimiento independiente de componentes y la integración con fuentes de datos heterogéneas.

## 3.1. Módulos Principales

El sistema se estructura en torno a cuatro módulos fundamentales:

- **traffic-sim**: Módulo de simulación de tráfico urbano, que genera escenarios de prueba controlados y reproducibles para evaluar el comportamiento del sistema bajo diferentes condiciones de demanda vehicular. Permite emular redes viales con semáforos y flujos vehiculares variables. Incorpora además un controlador local de presión máxima (Max-Pressure), que decide en tiempo real qué fase servir según la presión de las colas de entrada y de salida, con un mecanismo anti-*spillback* que penaliza las fases cuyos carriles de salida están saturados para evitar bloqueos.

- **traffic-control**: Módulo de orquestación que actúa como punto de entrada unificado al sistema. Recibe observaciones de tráfico, valida los datos, coordina las invocaciones a los servicios especializados de optimización y almacenamiento, y mantiene metadatos operativos en una base de datos relacional.

- **traffic-sync**: Módulo de análisis y optimización que procesa las métricas de tráfico recibidas, evalúa el nivel de congestión mediante técnicas de inteligencia computacional y

determina, mediante PSO, el largo de ciclo semafórico óptimo por clúster de sensores.

- **traffic-storage**: Módulo de persistencia verificable que almacena datos del sistema en infraestructuras descentralizadas, garantizando trazabilidad, integridad e inmutabilidad de los registros históricos mediante el uso de tecnologías de almacenamiento distribuido y registro de auditoría.

## 3.2. Flujo General de Información

El flujo operativo del sistema sigue un ciclo cerrado de captura, análisis, optimización y registro:

1. **Detección**: en traffic-sim, el detector de cuellos de botella muestrea vía TraCI cada intersección semaforizada cada 15 s de tiempo simulado, registrando conteo de vehículos, velocidad media, densidad (en vehículos por kilómetro de carril) y longitud de cola. Cuando una intersección supera los umbrales de cuello de botella se dispara el ciclo de optimización, sujeto a un periodo de enfriamiento de 60 s por intersección, de modo que la re-optimización no sature el pipeline con información redundante mientras las recomendaciones estratégicas, que cambian en escala de minutos, todavía no se modificaron.

2. **Armado del clúster y payload crudo**: se agrupan los semáforos ubicados a una distancia de hasta 200 m del cuello de botella detectado y se construye un payload con identificadores de SUMO sin normalizar y densidad expresada directamente en veh/km. traffic-sim no aplica ninguna normalización sobre estos datos: el contrato de diseño reserva esa responsabilidad exclusivamente al eslabón siguiente.

3. **Normalización y transporte**: el punto de entrada de ingesta de traffic-control valida, normaliza y persiste el batch recibido, sin tomar decisiones de control. Las normalizaciones aplicadas incluyen la extracción de identificadores numéricos mediante una expresión regular, la división de la densidad entre 100 con acotamiento al intervalo $[0, 1]$, la compleción del desglose por clase vehicular y el forzado de la versión de esquema. El batch se sube a traffic-storage, se recupera con reintentos (y una ruta de contingencia que usa el batch local si el almacenamiento falla, de modo que un fallo de persistencia nunca bloquee el control) y se reenvía a traffic-sync.

4. **Decisión estratégica**: traffic-sync encadena un sistema de inferencia difusa tipo Mamdani, que evalúa el nivel de congestión de cada sensor a partir de densidad, velocidad y flujo; un clustering jerárquico de Ward, que agrupa sensores con patrones de congestión similares; y un algoritmo PSO unidimensional, que busca por clúster la longitud de ciclo $C$ que minimiza una función de aptitud sensible a demora, capacidad y riesgo de atasco.

5. **Ejecución táctica con techo blando**: el ciclo óptimo devuelto por la capa estratégica no se traduce en una partición rígida en *splits* fijos, sino que actúa como techo blando (*soft*

*cap*) sobre la duración de fase: el controlador local de traffic-sim, basado en el principio de Max-Pressure, decide en tiempo real qué fase servir según la presión de colas de entrada y salida, respetando ese límite superior orientativo.

Las fases de detección, armado del clúster y ejecución táctica ocurren en traffic-sim; la normalización y transporte, en traffic-control; y la decisión estratégica, en traffic-sync. La Figura 3.1 recoge este mismo flujo de datos a nivel de los cuatro módulos del sistema.

Este diseño separa deliberadamente dos escalas temporales de decisión. La escala estratégica opera en minutos: el ciclo difuso–Ward–PSO se dispara solo cuando se detecta un cuello de botella y está sujeto al enfriamiento de 60 s por intersección descrito arriba, de modo que recalcula el presupuesto de ciclo con una frecuencia compatible con el costo computacional de la optimización y con la velocidad real de cambio de los patrones agregados de demanda. La escala táctica opera en segundos: la decisión de a qué fase dar verde se recalcula en cada paso de simulación en función de la presión instantánea de colas, respondiendo a la variabilidad de corto plazo que un recálculo cada varios minutos no podría capturar. Mantener estas escalas separadas, en lugar de que un único optimizador decida simultáneamente el ciclo y el reparto de fases, evita que ambas capas compitan por la misma variable de decisión a frecuencias incompatibles: la capa estratégica fija un presupuesto agregado sin necesidad de resolver, a esa misma frecuencia, el problema combinatorio de reparto de verde por fase, mientras que la capa táctica reacciona a la ocupación instantánea de colas sin necesidad de ejecutar en cada paso una optimización costosa a nivel de clúster.

## 3.3. Justificación de la Arquitectura

La elección de una arquitectura de microservicios sobre un diseño monolítico responde a requisitos concretos del sistema. Los cuatro módulos tienen ciclos de vida y stacks tecnológicos radicalmente distintos: traffic-sim depende de SUMO y librerías de red vial; traffic-sync combina scikit-fuzzy con optimización metaheurística; traffic-storage requiere un cliente web3 y el daemon de IPFS; traffic-control actúa exclusivamente como orquestador sin procesamiento intensivo. Empaquetar estos cuatro dominios en un solo proceso eliminaría la posibilidad de reemplazar, por ejemplo, el simulador por sensores IoT reales sin recompilar el núcleo analítico.

La comunicación mediante REST/JSON se eligió frente a alternativas binarias como gRPC porque las latencias dominantes del sistema son la carga a IPFS (≈1,2 s) y la confirmación en BlockDAG (≈3,5 s), por lo que la sobrecarga de serialización JSON no es el cuello de botella. REST también reduce la barrera de integración con infraestructura municipal existente y con sensores IoT estándar que ya exponen APIs HTTP.

La división en cuatro módulos refleja cuatro responsabilidades diferenciadas: simular y controlar localmente (traffic-sim), coordinar y validar (traffic-control), analizar y optimizar a escala estratégica (traffic-sync), y persistir con verificabilidad (traffic-storage). El control local por presión máxima reside en traffic-sim, y no en traffic-sync, por una razón de latencia y volumen de

información: la decisión de qué fase servir se toma en la escala de segundos a partir del estado de cada carril en cada instante, y trasladarla a un servicio remoto exigiría enviar ese flujo de datos por la red en cada paso de simulación, con una latencia incompatible con esa escala. Por ello solo es viable como un componente integrado en la interfaz TraCI de la propia simulación, mientras que traffic-sync atiende la capa estratégica, de cadencia mucho menor. Fusionar, por ejemplo, orquestación y optimización acoplaría la lógica de validación de entradas con el ciclo difuso-PSO, dificultando las pruebas independientes de cada componente.

Las propiedades que resultan de estas decisiones son:

- **Bajo acoplamiento**: Los módulos se comunican exclusivamente mediante contratos REST/JSON; evolucionar o reemplazar un componente no requiere modificar los demás.
- **Escalabilidad**: traffic-sync escala linealmente con el número de sensores ($\approx 0{,}034$ s/sensor hasta 2000 sensores), como se demuestra en el Capítulo 8.
- **Trazabilidad verificable**: cada lote de datos recibe un CID de IPFS inmutable, registrado en el contrato inteligente, lo que permite auditar retroactivamente cualquier decisión de control.
- **Reproducibilidad**: SUMO genera escenarios con semillas deterministas, garantizando que los experimentos del Capítulo 8 puedan reproducirse de forma independiente.
- **Interoperabilidad**: la interfaz REST de traffic-control permite sustituir traffic-sim por sensores físicos sin cambios en los módulos de análisis ni de almacenamiento.

La Figura 3.1 ilustra de forma esquemática la organización modular del sistema, mostrando los cuatro módulos principales y sus relaciones.

En síntesis, esta arquitectura modular proporciona una plataforma flexible y escalable para la gestión inteligente de tráfico urbano, combinando capacidades de simulación, análisis adaptativo y persistencia verificable en un sistema distribuido que puede integrarse con infraestructuras reales de monitoreo vehicular. Los capítulos subsiguientes describen en detalle la implementación, las tecnologías subyacentes y el funcionamiento de cada módulo.

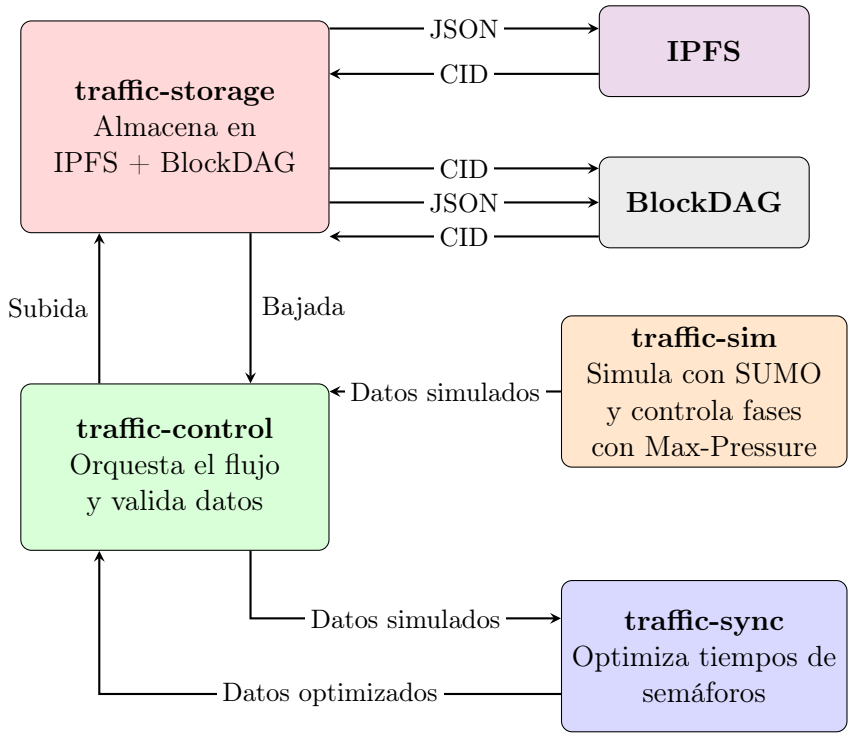



Figura 3.1: Diagrama de la arquitectura del sistema mostrando la interacción de datos entre los módulos de control, simulación (que incluye el control local Max-Pressure), sincronización y almacenamiento.

# CAPÍTULO 4
# SIMULACIÓN DE TRÁFICO

El módulo traffic-sim implementa un entorno de simulación de tráfico urbano basado en SUMO (Simulation of Urban MObility), utilizado para evaluar el comportamiento del sistema propuesto bajo escenarios controlados y reproducibles. SUMO es un simulador microscópico de tráfico ampliamente adoptado en la comunidad científica, que permite modelar redes viales, rutas de vehículos y programas semafóricos, y ejecutar simulaciones paso a paso mediante una interfaz de control remoto [44]. En este trabajo, traffic-sim se integra con el servicio de optimización traffic-sync para cerrar el ciclo "simulación–detección–optimización–aplicación" sobre redes generadas a partir de datos reales o escenarios de prueba.

## 4.1. Arquitectura del Módulo

La estructura de traffic-sim se organiza en torno a un orquestador de simulación y un conjunto de componentes especializados para detección, control y comunicación externa. El orquestador de simulación encapsula la lógica principal: carga la configuración de SUMO (archivos de configuración de red, aristas, nodos, rutas y semáforos), inicializa la simulación mediante la interfaz de control remoto y gestiona el avance de la simulación en pasos discretos [44]. El punto de entrada recibe como argumento un archivo comprimido de escenario (generado, por ejemplo, mediante la herramienta auxiliar de construcción de mapas) y ejecuta la simulación en modo sin interfaz gráfica o con interfaz gráfica.

Los detectores de cuellos de botella monitorean métricas como densidad de vehículos, velocidad promedio y longitud de cola por tramo o aproximación. Los umbrales y parámetros de detección (por ejemplo, densidad mínima, velocidad máxima para considerar congestión, intervalo entre detecciones, duración mínima del evento) se centralizan en la configuración del sistema, lo que permite ajustar la sensibilidad del sistema sin modificar la lógica de detección.

Cada evento de cuello de botella confirmado dispara la formación de un clúster de semáforos vecinos, ubicados a una distancia máxima de 200 m de la intersección primaria, y el envío del pipeline de optimización estratégica (fuzzy + PSO); para evitar disparos redundantes sobre un mismo clúster mientras la optimización previa aún no produjo efecto observable, el orquestador impone un período de enfriamiento (*cooldown*) de 60 s entre optimizaciones consecutivas del mismo grupo de semáforos.

El controlador de semáforos opera en dos modos. En el modo estático (*baseline*), se limita a aplicar sobre SUMO, a través de la interfaz de control remoto, los tiempos de verde y de ciclo recibidos desde el servicio de optimización, sin ninguna capacidad de decisión local: la fase activa y su duración quedan enteramente determinadas por el plan semafórico vigente. En el modo dinámico, en cambio, el controlador de semáforos delega la decisión de qué fase servir en cada instante a un controlador táctico local de tipo Max-Pressure, descrito en la Sección 4.2, que reacciona a la presión de tráfico observada en cada carril con una cadencia de segundos, muy por debajo de la escala de tiempo a la que opera el optimizador estratégico.

El cliente HTTP se comunica con la API del módulo de optimización: cuando se detecta un cuello de botella, este cliente construye una carga útil con métricas agregadas de tráfico para la intersección crítica y lo envía al punto de entrada correspondiente. Las utilidades de métricas agregan y validan las variables macroscópicas de SUMO (viajes completados, tiempos de viaje, velocidades y colas), garantizando que la información enviada al módulo de optimización siga el esquema esperado por el sistema difuso y el optimizador PSO.

Las herramientas de visualización proporcionan análisis posterior de resultados: parsers de archivos de salida de SUMO (información de viajes, resúmenes, datos de vehículos), funciones para generar gráficos de distribución de tiempos de viaje, evolución temporal de congestión y comparaciones entre diferentes configuraciones de semáforos, así como envoltorios sobre herramientas nativas de SUMO. Estas utilidades permiten cuantificar el impacto de las estrategias de control sobre métricas clave como tiempo de viaje, tiempos de espera y niveles de congestión.

La Figura 4.1 ilustra la arquitectura del módulo traffic-sim, mostrando los componentes principales y sus interacciones con SUMO y el servicio traffic-sync.

## 4.2. Controlador Táctico Local: Max-Pressure

Mientras el par difuso–PSO de traffic-sync opera como capa *estratégica*, recalculando planes semafóricos ante la detección de un cuello de botella con una cadencia del orden de decenas de segundos, la red requiere además una capa *táctica* capaz de reaccionar a la evolución de las colas en la escala de segundos, sin esperar al próximo ciclo de optimización. Con ese fin, el modo dinámico de traffic-sim incorpora un controlador local de presión máxima (*Max-Pressure*), fundamentado en los resultados de estabilidad de [33] y en la teoría de control de colas por *backpressure* sobre redes de [32]: bajo esta familia de políticas, servir en cada instante la fase con mayor diferencia entre la congestión que alimenta y la que genera en los carriles de destino garantiza estabilidad de la red ante cualquier patrón de demanda admisible, sin requerir un

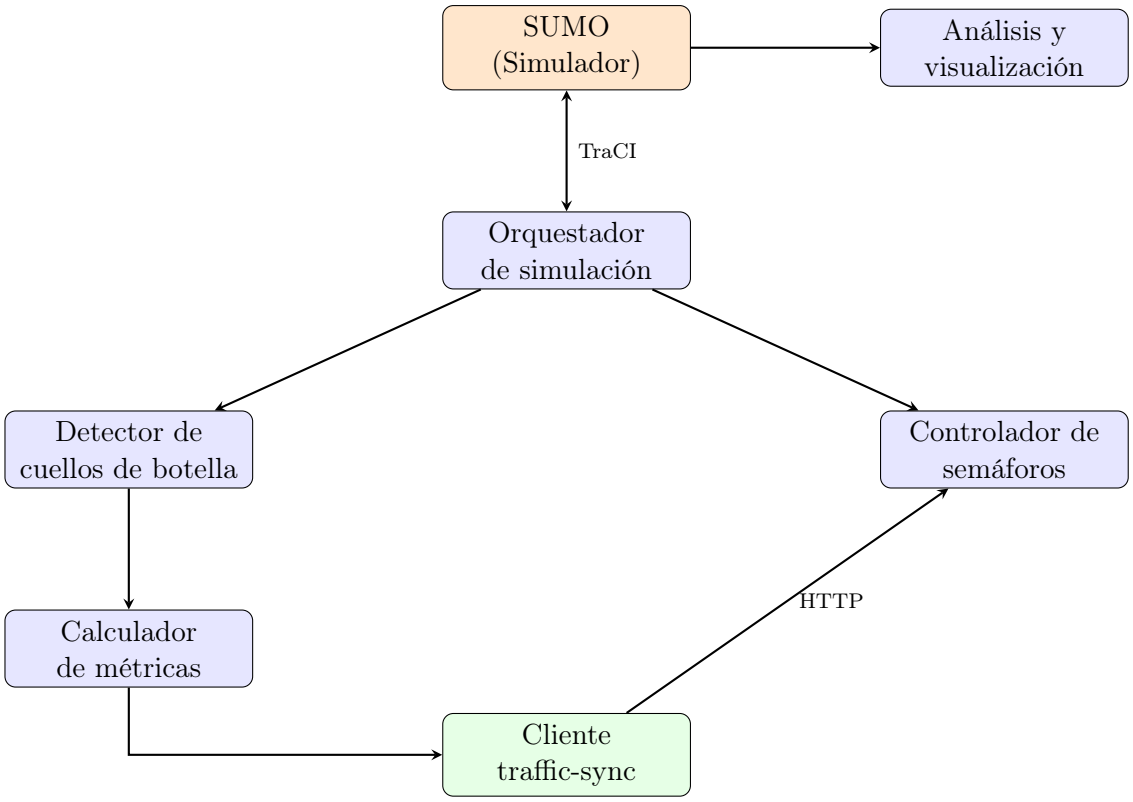



Figura 4.1: Arquitectura del módulo traffic-sim y sus componentes principales [44].

modelo de la demanda futura.

**Presión de fase.** Cada semáforo se modela como un conjunto de fases, y cada fase como un conjunto de pares (carril de entrada, carril de salida) habilitados simultáneamente. La presión de una fase $i$ se define como

$$P_i = \sum_{(\ell_{\text{in}},\ell_{\text{out}})\,\in\, i} \left[ \frac{q_{\ell_{\text{in}}}}{\kappa_{\ell_{\text{in}}}} - w \cdot r_{\ell_{\text{out}}} \right] + \beta \cdot (t - t_i^{\text{serv}}) \tag{4.1}$$

donde $q_\ell$ es el número de vehículos detenidos en el carril $\ell$ durante el último paso de simulación y $\kappa_\ell$ su capacidad nominal, aproximada a partir del largo del carril y del espacio promedio que ocupa un vehículo. El término $r_{\ell_{\text{out}}} = \max(q_{\ell_{\text{out}}}/\kappa_{\ell_{\text{out}}},\, o_{\ell_{\text{out}}})$ mide la congestión del carril de salida combinando su razón de cola con su ocupación física $o_{\ell_{\text{out}}} \in [0, 1]$, de modo que la presión reconozca tanto una cola detenida como un carril lleno que avanza con dificultad; la ocupación solo interviene cuando la velocidad media de la salida es baja, para no penalizar a una fase que alimenta un carril lleno pero circulando con normalidad.

El factor $w$ pondera el descuento por la congestión en los carriles de salida, y $\beta$ es un bono de inanición proporcional al tiempo transcurrido desde el último servicio de la fase, $t - t_i^{\text{serv}}$, que impide la exclusión indefinida de fases de baja demanda. Los valores adoptados para estos y los demás parámetros del controlador se resumen en la Tabla 4.1.

La razón de cola de entrada $q_{\ell_{\text{in}}}/\kappa_{\ell_{\text{in}}}$ se deja deliberadamente sin techo superior en 1. Bajo saturación de corredor completo, varias fases en competencia alcanzan de forma simultánea colas iguales o mayores a su capacidad nominal, y recortar el cociente aplana justamente la

señal que permite discriminar entre ellas; en pruebas preliminares, imponer ese techo derivó en bloqueos estables de los que la red no se recuperaba dentro del horizonte de simulación.

**Gate anti-*spillback*.** Un carril de salida que combina ocupación física alta con velocidad media casi nula se considera realmente atascado (no meramente lleno y drenando) y fuerza $r_{\ell_{\text{out}}}$ a un valor de penalización fijo, de modo que ninguna fase que lo alimente resulte elegida. La regla evita que un vehículo ingrese a la intersección sin posibilidad real de salir y termine bloqueando los movimientos transversales: con los teletransportes de SUMO deshabilitados, ese bloqueo constituye un interbloqueo (*gridlock*) irreversible dentro de la simulación.

La misma penalización opera además como mecanismo de coordinación entre cruces vecinos. Como todos los controladores locales comparten el proceso de simulación, cada uno consulta un mapa que asocia sus carriles de salida con el cruce siguiente que los controla y, si ese vecino ya fue marcado como estancado, penaliza el enlace correspondiente antes de calcular su propia presión. Así, la señal de "el tramo siguiente no drena" se propaga de intersección en intersección sin requerir mensajería ni un optimizador de *offsets* centralizado, aunque, como se discute en las limitaciones del Capítulo 8, esta coordinación implícita no sustituye a una optimización explícita de desfases entre semáforos coordinados.

La Figura 4.2 resume la cadena causal que convierte una decisión local aparentemente razonable (dar verde hacia el carril de salida con mayor presión instantánea) en un bloqueo estructural de la red, y señala sobre qué eslabón de esa cadena actúa cada una de las tres contramedidas descritas en esta sección.

**Ciclo de decisión.** El controlador reevalúa la fase activa a intervalos fijos y solo conmuta si se cumplen tres condiciones: que la fase en curso haya completado su verde mínimo, que exista al menos una alternativa no excluida, y que la presión de esa alternativa supere a la de la fase activa por un margen de histéresis. La histéresis es la propiedad de un sistema de decisión de no cambiar de estado ante una diferencia mínima, sino solo cuando la diferencia a favor del cambio es suficientemente grande: aquí, la fase alternativa debe superar a la activa en un factor mayor que uno (1,3 con los valores adoptados, Tabla 4.1) y no apenas en una fracción. Ese margen desalienta los cambios de fase innecesarios (*flapping*) ante presiones casi empatadas, que de otro modo harían alternar el verde de forma continua y desperdiciarían tiempo en las transiciones de amarillo y rojo total. Toda conmutación intercala una etapa de luz amarilla seguida de un despeje en rojo total antes de habilitar la fase entrante, replicando la seguridad de las transiciones del plan fijo de SUMO.

**Detección de estancamiento y modo drenaje.** Si tras varios servicios consecutivos la cola de entrada de una fase no disminuye, se infiere que el bloqueo se origina más adelante en la red y no responde a asignar más verde local: la fase se excluye temporalmente, con un período de exclusión que se duplica en cada reincidencia hasta un techo máximo (retroceso exponencial). Cuando todas las alternativas a la fase activa están excluidas, el controlador entra en *modo drenaje* y sirve de forma sostenida la fase con mayor cola de entrada en volumen absoluto.

El criterio de volumen absoluto, en lugar de la razón normalizada que gobierna la presión en régimen normal, responde al mismo argumento que justifica dejar la razón de entrada sin

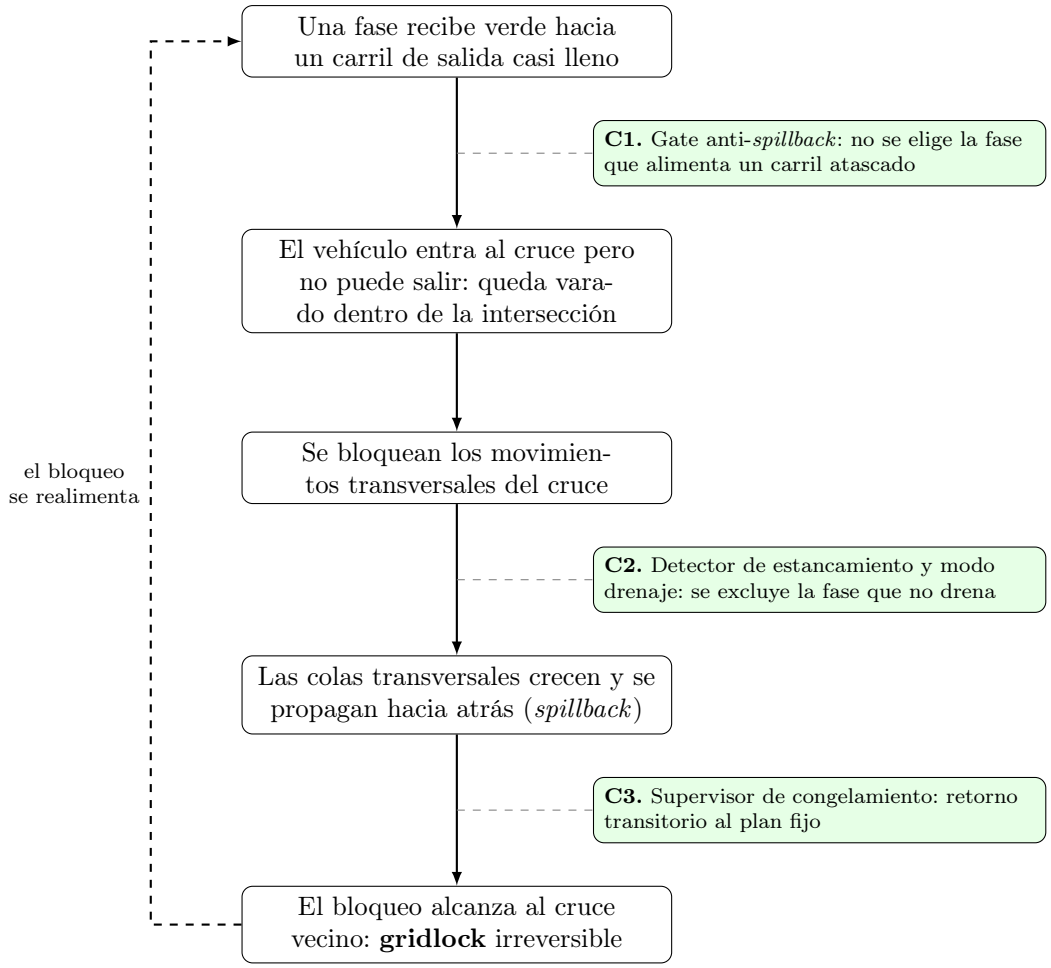



Figura 4.2: Formación del *gridlock* por *spillback* y contramedidas del controlador. La columna izquierda es la cadena causal del bloqueo; cada recuadro sombreado señala el eslabón sobre el que interviene una contramedida. La flecha discontinua de la izquierda indica que el bloqueo se realimenta: una vez cerrado el ciclo, y con los teletransportes de SUMO deshabilitados, la red no se recupera por sí sola.

techo: bajo saturación total varias fases alcanzan razones de cola equivalentes, de modo que la métrica normalizada deja de discriminar entre ellas justo cuando el modo drenaje necesita elegir un único corredor crítico.

**Supervisor de congelamiento.** Como resguardo de última instancia frente a un interbloqueo generalizado, un supervisor independiente monitorea la fracción de vehículos detenidos sobre el total de vehículos en red. Si esa fracción se mantiene alta durante una ventana sostenida, restaura en todos los semáforos el programa fijo original de SUMO, que por construcción no puede bloquearse, y suspende las decisiones de Max-Pressure durante un período de recuperación, tras el cual el control vuelve a Max-Pressure con el estado de estancamiento reiniciado.

**Relación con el optimizador estratégico.** El verde mínimo y el verde máximo duro de cada fase se definen como múltiplos del verde base equitativo del semáforo, esto es, del promedio de las duraciones de fase de su programa original. El presupuesto de ciclo que el optimizador PSO de traffic-sync fija dinámicamente actúa únicamente como un techo blando de referencia por fase, calculado como $C/n$ con $C$ el largo de ciclo optimizado y $n$ el número de fases del

semáforo: puede acotar el verde máximo efectivo por debajo del máximo duro en régimen fluido, pero no constituye un reparto exacto ni impide que Max-Pressure extienda una fase realmente saturada.

Queda así resuelto el conflicto de autoridad entre la capa estratégica, que optimiza en el agregado y con retraso, y la capa táctica, que reacciona en tiempo casi real a la condición local de cada carril.

Tabla 4.1: Parámetros del controlador táctico Max-Pressure.

| **Parámetro** | **Valor** | **Rol** |
|---|---|---|
| Período de decisión | 5 s | Cadencia de reevaluación de la fase activa |
| $w$ | 1,0 | Peso del descuento por congestión de salida |
| $\beta$ | 0,05 $s^{-1}$ | Bono de inanición por tiempo sin servicio |
| Espacio por vehículo | 7,5 m | Base del cálculo de la capacidad $\kappa_\ell$ |
| Umbral de detención | 0,1 m/s | Velocidad bajo la cual un vehículo cuenta como cola |
| Salida de avance lento | 1,0 m/s | Habilita el uso de la ocupación en $r_{\ell_{\text{out}}}$ |
| Ocupación de *spillback* | 0,85 | Umbral de carril de salida atascado |
| Velocidad de *spillback* | 0,5 m/s | Umbral de carril de salida atascado |
| Penalización por bloqueo | 2,0 | Valor forzado de $r_{\ell_{\text{out}}}$ ante atasco |
| Factor de histéresis | 1,3 | Margen de presión exigido para conmutar |
| Verde mínimo | 0,4× verde base | Duración mínima garantizada por fase |
| Verde máximo duro | 1,5× verde base | Cota superior no anulable por el PSO |
| Amarillo / todo-rojo | 3 s / 1 s | Transición segura entre fases |
| Servicios sin drenaje | 3 | Disparan la exclusión temporal de una fase |
| Exclusión inicial / máxima | 60 s / 240 s | Retroceso exponencial del estancamiento |
| Fracción de congelamiento | $\geq$ 0,7 durante 60 s | Dispara el supervisor |
| Período de recuperación | 240 s | Duración del retorno al plan fijo |

El Algoritmo 4.1 integra los mecanismos anteriores en el ciclo completo que el controlador ejecuta en cada intersección semaforizada, desde la resolución de una transición en curso hasta la actualización del detector de estancamiento tras un eventual cambio de fase. Las salvaguardas se evalúan antes que la presión, de modo que una red comprometida nunca queda a merced de la decisión local.

## 4.3. Flujo de Integración

El flujo de integración entre traffic-sim y traffic-sync sigue una secuencia cíclica que replica el comportamiento de un sistema de control de tráfico en línea:

1. **Ejecución de la simulación**: el orquestador inicia SUMO con el escenario suministrado y avanza la simulación paso a paso, según las rutas y patrones de demanda definidos en los archivos de configuración.

2. **Detección de cuellos de botella**: en intervalos configurables, el detector analiza para cada aproximación variables como densidad, velocidad y longitud de cola, y decide si existe un cuello de botella según los umbrales establecidos en la configuración del sistema.

**Algoritmo 4.1** Ciclo de decisión de Max-Pressure, ejecutado cada 5 s en cada intersección semaforizada.

**Entrada:** fase activa $a$; conjunto de fases $\mathcal{F}$; estado de estancamiento
**Salida:** fase a servir durante el próximo intervalo

**si** hay una transición en curso **entonces**
    avanzar la secuencia amarillo → todo-rojo → verde
    **retornar**
**si** la fracción de vehículos detenidos se mantuvo alta durante la ventana del supervisor **entonces**
    restaurar el plan fijo original y suspender el control táctico
    **retornar**
**si** $a$ no completó su verde mínimo **entonces**
    **retornar** ▷ mantener la fase activa
**para cada** fase $i \in \mathcal{F}$ **hacer**
    $P_i \leftarrow$ presión de $i$ según la Ecuación (4.1)
    **si** alguna salida de $i$ está atascada **entonces**
        forzar la penalización de bloqueo y recalcular $P_i$ ▷ gate anti-*spillback*
$\mathcal{C} \leftarrow \{\, i \in \mathcal{F} : i$ no está excluida por estancamiento $\}$
**si** $\mathcal{C} = \emptyset$ **entonces**
    $\mathcal{C} \leftarrow \{$ fase de mayor cola de entrada en volumen absoluto $\}$ ▷ modo drenaje
$i^* \leftarrow \arg\max_{i \in \mathcal{C}} P_i$
**si** $P_{i^*}$ supera a $P_a$ por el margen de histéresis **entonces**
    iniciar la transición hacia $i^*$
    actualizar el detector de estancamiento de $a$

3. **Construcción de métricas de tráfico**: cuando se confirma un cuello de botella en una intersección con semáforo, se calcula un conjunto de métricas agregadas para esa localización (vehículos por minuto, velocidad media en km/h, tiempo medio de circulación, densidad y estadísticos por tipo de vehículo), que se empaquetan en una carga útil JSON siguiendo el formato consumido por el módulo de optimización.

4. **Comunicación con el módulo de optimización**: el cliente HTTP envía la carga útil al servicio de optimización mediante una petición HTTP, y espera la respuesta con el largo de ciclo optimizado por clúster y el impacto estimado sobre la congestión.

5. **Aplicación de optimizaciones**: el controlador de semáforos incorpora, a través de la interfaz de control remoto, el largo de ciclo recibido desde el módulo de optimización. En modo estático lo traduce directamente en tiempos de verde y rojo por fase; en modo dinámico lo entrega como techo blando de referencia al controlador táctico Max-Pressure (Sección 4.2), que sigue decidiendo en tiempo real qué fase servir.

6. **Continuación de la simulación**: la simulación se reanuda con los nuevos parámetros de control y se repite el proceso de detección y optimización mientras existan vehículos en red o hasta alcanzar el tiempo límite definido.

De esta manera, traffic-sim se comporta como un laboratorio virtual donde se evalúa en bucle

el desempeño del controlador difuso Mamdani y del optimizador PSO, con escenarios construidos a partir de mapas reales. Los resultados, incluyendo métricas temporales y distribuciones de tiempos de viaje, se analizan mediante las utilidades de visualización.

Esta secuencia de seis pasos no ocurre a una única cadencia: cada mecanismo del sistema opera en una escala temporal propia, y la separación entre ellas es precisamente lo que evita que las decisiones estratégicas de traffic-sync y las decisiones tácticas locales de Max-Pressure compitan por autoridad sobre el mismo instante de control. La Tabla 4.2 resume esas cinco escalas, de la microsimulación a la recuperación supervisada, ordenadas de la más rápida a la más lenta. El rango completo abarca poco más de dos órdenes de magnitud, y ningún par de capas contiguas comparte cadencia: esa separación es la que permite que cada una decida sin bloquear a la siguiente.

Tabla 4.2: Escalas temporales del control jerárquico. Las cinco cadencias son concurrentes, no sucesivas: cada mecanismo decide sobre su propio período sin esperar a los demás.

| **Mecanismo** | **Capa** | **Cadencia** | **Decisión que toma** |
|---|---|---|---|
| Paso de microsimulación (SUMO) | Base | 1 s | Avance del estado de la red |
| Controlador Max-Pressure | Táctica | 5 s | Qué fase recibe verde |
| Detector de cuellos de botella | Enlace | 15 s | Si se solicita una optimización |
| Optimización difuso–PSO | Estratégica | 60 s[a] | Largo de ciclo del clúster |
| Supervisor de congelamiento | Salvaguarda | 240 s[b] | Si se devuelve el control al plan fijo |

[a] Período mínimo de enfriamiento entre optimizaciones de un mismo clúster, no una cadencia fija: la optimización se dispara por evento.
[b] Duración de la ventana de recuperación, no un período de muestreo.

## 4.4. Escenario de Simulación y Parámetros de Detección

El escenario descrito en esta sección corresponde a las pruebas de integración y escalabilidad del módulo traffic-sim, orientadas a validar el ciclo completo simulación–detección–optimización–aplicación bajo una carga vehicular sostenida. La campaña de evaluación del controlador Max-Pressure frente al baseline estático, con su metodología estadística A/B, se describe por separado en el capítulo de experimentos y utiliza una red distinta, la de San Lorenzo, con seis escenarios de demanda derivados de forma determinística (Capítulo 8, Sección 8.4).

**Red vial.** La red se construyó a partir de datos de OpenStreetMap para un sector del área metropolitana de Asunción, Paraguay (latitud $-25{,}34$–$-25{,}35$ S, longitud $-57{,}50$–$-57{,}51$ O), abarcando aproximadamente 0,8 km$^2$. La red comprende 22 nodos, 33 tramos y 7 intersecciones semaforizadas, con vehículos clasificados en automóvil, motocicleta, ómnibus y camión.

**Demanda vehicular.** Se inyectaron 5 000 vehículos con tiempos de salida distribuidos uniformemente en el horizonte de simulación (0–3 600 s), correspondiendo a una tasa media de $\approx 83$ vehículos/minuto. La simulación avanza en pasos de 1 s durante 3 600 pasos (1 hora de tráfico urbano), con un máximo de 1 000 vehículos simultáneos en red.

**Configuración semafórica base.** Cada semáforo opera inicialmente con un ciclo de 90 s,

tiempos de verde ajustables entre 10 s y 120 s por fase, 3 s de luz amarilla y 1 s de todo–rojo entre fases conflictivas.

**Umbrales del detector de cuellos de botella.** El detector analiza cada intersección cada 15 pasos de simulación (15 s) y confirma un evento solo si la condición persiste al menos 3 s. Se requiere un mínimo de 4 vehículos presentes y que se cumpla al menos una de las siguientes condiciones (dos de ellas simultáneas elevan la severidad, como se detalla más abajo):

- Densidad $> 50$ veh/km.
- Velocidad media $< 15$ km/h.
- Cola $\geq 3$ vehículos detenidos ($< 1$ m/s).

La severidad se clasifica como *alta* (densidad elevada y velocidad baja), *media* (densidad elevada y cola significativa, o velocidad baja y cola significativa) o *baja* (solo densidad elevada o velocidad muy reducida).

**Carga útil hacia el módulo de optimización.** Al detectar un cuello de botella, el cliente HTTP construye una carga útil JSON con el esquema de la Tabla 4.3.

Tabla 4.3: Esquema del payload enviado de traffic-sim a traffic-sync ante un cuello de botella. Los campos aparecen agrupados por su nivel de anidamiento en el documento JSON.

| **Campo** | **Tipo** | **Descripción / Ejemplo** |
|---|---|---|
| version | string | Versión del protocolo ("2.0") |
| type | string | Tipo de mensaje ("data") |
| timestamp | string | ISO 8601 UTC ("2025-09-12T14:23:01Z") |
| traffic_light_id | string | ID del semáforo SUMO ("20928101") |
| *En cada elemento de* sensors[] | | |
| controlled_edges | array | Tramos controlados (["-143977090#0", ...]) |
| vehicle_stats | object | Conteo por tipo: {car, motorcycle, bus, truck} |
| *En* sensors[].metrics | | |
| vehicles_per_minute | int | Volumen observado (18) |
| avg_speed_kmh | float | Velocidad media en km/h (12.4) |
| avg_circulation_time_sec | float | Tiempo medio de circulación en s (47.3) |
| density | float | Densidad en veh/km (67.2) |

# CAPÍTULO 5

# ALMACENAMIENTO DISTRIBUIDO Y TECNOLOGÍAS DESCENTRALIZADAS

El módulo traffic-storage actúa como capa de persistencia verificable del sistema, recibiendo métricas de tráfico y resultados de optimización, almacenando los datos en un repositorio distribuido y registrando huellas inmutables en infraestructuras descentralizadas. En este contexto, blockchain, BlockDAG e IPFS se utilizan de forma complementaria: blockchain y BlockDAG como ledgers inmutables para trazabilidad, e IPFS como almacén de archivos orientado a contenido [45, 8, 9].

## 5.1. Blockchain

Blockchain puede describirse como un libro mayor distribuido donde las transacciones se agrupan en bloques enlazados criptográficamente en una cadena lineal, de modo que cada bloque referencia al anterior mediante un resumen criptográfico (hash), proporcionando inmutabilidad y resistencia a manipulaciones retrospectivas [45, 46]. Los nodos de la red ejecutan un mecanismo de consenso (como Proof-of-Work o Proof-of-Stake) para acordar qué bloques se consideran válidos, garantizando que todos mantengan una vista consistente del historial de transacciones [47, 48].

La Figura 5.1 ilustra la estructura básica de una cadena de bloques, mostrando cómo cada bloque referencia criptográficamente al bloque anterior mediante su resumen criptográfico (hash), creando una cadena inmutable donde cualquier modificación en un bloque anterior invalida todos los bloques subsiguientes.

En este trabajo, blockchain se utiliza para registrar metadatos de almacenamiento de tráfico de forma inmutable, actuando como capa de "prueba de existencia" para datos que residen físicamente fuera de la cadena. El contrato inteligente de almacenamiento expone operaciones

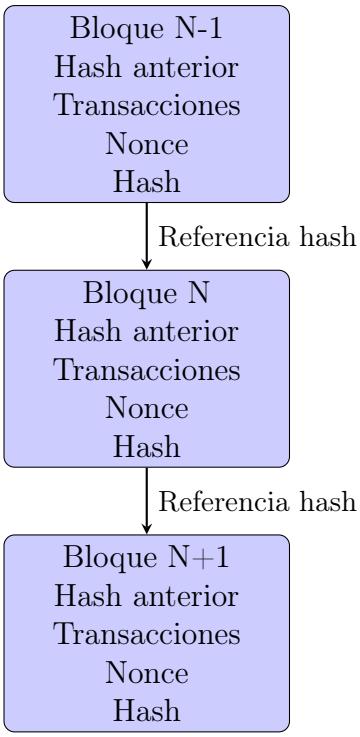



Figura 5.1: Estructura básica de una cadena de bloques, mostrando la dependencia criptográfica entre bloques consecutivos [45].

de alto nivel para registrar eventos de almacenamiento: cada llamada almacena, al menos, un identificador de semáforo, una marca de tiempo y un identificador de contenido (CID de IPFS), junto con el tipo de dato al que hace referencia. Este contrato se despliega y prueba utilizando un entorno de desarrollo estándar, con scripts de despliegue y pruebas automatizadas que verifican su funcionamiento básico.

El módulo traffic-storage ofrece una API HTTP que recibe cargas de datos en un formato unificado (versión 2.0), generadas por otros componentes del sistema (como los módulos de optimización y lógica difusa). Cuando llega una nueva medición o reporte, la API valida la carga útil, delega la persistencia de contenido a IPFS y, una vez obtenido el CID, invoca al contrato inteligente de almacenamiento para registrar ese CID junto con los metadatos relevantes en la blockchain. De este modo, los algoritmos de optimización y los módulos de monitoreo solo interactúan con una interfaz REST, mientras que los detalles de interacción on-chain quedan encapsulados en el módulo de almacenamiento.

## 5.2. BlockDAG

BlockDAG (Block Directed Acyclic Graph) generaliza la estructura lineal de blockchain permitiendo que cada bloque referencie a múltiples bloques padres anteriores, formando un grafo dirigido acíclico en lugar de una cadena única [49, 9]. Esta arquitectura permite que bloques creados en paralelo se integren en el consenso sin ser descartados como huérfanos, mejorando el aprovechamiento del ancho de banda de la red y posibilitando mayores tasas de

transacciones y menores latencias de confirmación que las típicamente alcanzables en cadenas lineales [49].

La Tabla 5.1 sintetiza las diferencias estructurales entre ambas arquitecturas. La comparación considera protocolos representativos de cada paradigma: Nakamoto consensus implementado en Bitcoin y Ethereum para blockchains lineales [45, 47], y PHANTOM/GHOSTDAG como generalización del consenso distribuido en estructuras DAG [9].

Tabla 5.1: Diferencias estructurales entre una blockchain lineal y un BlockDAG con consenso GHOSTDAG [9, 50]. La comparación es cualitativa: las cifras de rendimiento medidas en este trabajo se presentan en la Tabla 8.4.

| **Característica** | **Blockchain lineal** | **BlockDAG (GHOSTDAG)** |
|---|---|---|
| Topología | Cada bloque referencia un único padre | Cada bloque referencia varios padres |
| Bloques concurrentes | Se descartan como huérfanos | Se integran al grafo y cuentan para el consenso |
| Ordenamiento | Total, inducido por la propia cadena | Parcial en el grafo; el algoritmo de consenso lo completa a un orden total |
| Minería | Competitiva: el bloque que llega segundo se pierde | Cooperativa: todo bloque válido aporta trabajo al consenso |
| Límite de rendimiento | El protocolo frena la tasa de bloques para evitar huérfanos | El ancho de banda de la red es el límite práctico |
| Confirmación | Probabilística; exige esperar varios bloques sucesivos | Probabilística; la espera se acorta al integrar bloques paralelos |

En el contexto de este sistema, BlockDAG se considera como infraestructura de registro distribuido complementaria a la blockchain tradicional, pensada para escenarios de mayor escala donde múltiples intersecciones reportan datos con alta frecuencia. El módulo traffic-storage incorpora una capa de abstracción que permite, según la configuración, registrar eventos de almacenamiento tanto en un contrato EVM como en una red BlockDAG compatible, utilizando un cliente dedicado que construye y envía mensajes de registro a un nodo remoto. Estos mensajes contienen, de forma análoga a la variante blockchain, el identificador de semáforo, la marca de tiempo y el resumen criptográfico de contenido (CID de IPFS u otro identificador criptográfico).

La lógica de alto nivel en el módulo de almacenamiento se mantiene independiente de la tecnología subyacente: la API recibe métricas y resultados de optimización, delega la persistencia de archivos a IPFS y, posteriormente, invoca una interfaz de "registro de evidencia" que puede mapearse a blockchain, BlockDAG o a un esquema dual. Esta separación permite que, a medida que se exploren o adopten redes DAG de mayor throughput para despliegues reales en entornos de ciudad inteligente, el resto del sistema (clasificación de congestión, PSO, paneles de monitoreo) continúe interactuando con un mismo punto de entrada consistente.

## 5.3. IPFS

El InterPlanetary File System (IPFS) es un sistema de archivos distribuido peer-to-peer que utiliza direccionamiento por contenido: cada objeto se identifica por un Content Identifier (CID) que codifica un resumen criptográfico (hash) del contenido más metadatos mínimos sobre cómo interpretarlo [8]. El mismo archivo siempre produce el mismo CID, independientemente del nodo que lo procese, lo que permite verificar integridad de manera determinista y realizar deduplicación automática a escala de red [51].

La Figura 5.2 ilustra el flujo básico de almacenamiento y recuperación en IPFS, mostrando cómo un archivo se fragmenta, se le asigna un CID basado en su contenido, y puede ser recuperado desde cualquier nodo que posea los bloques correspondientes.

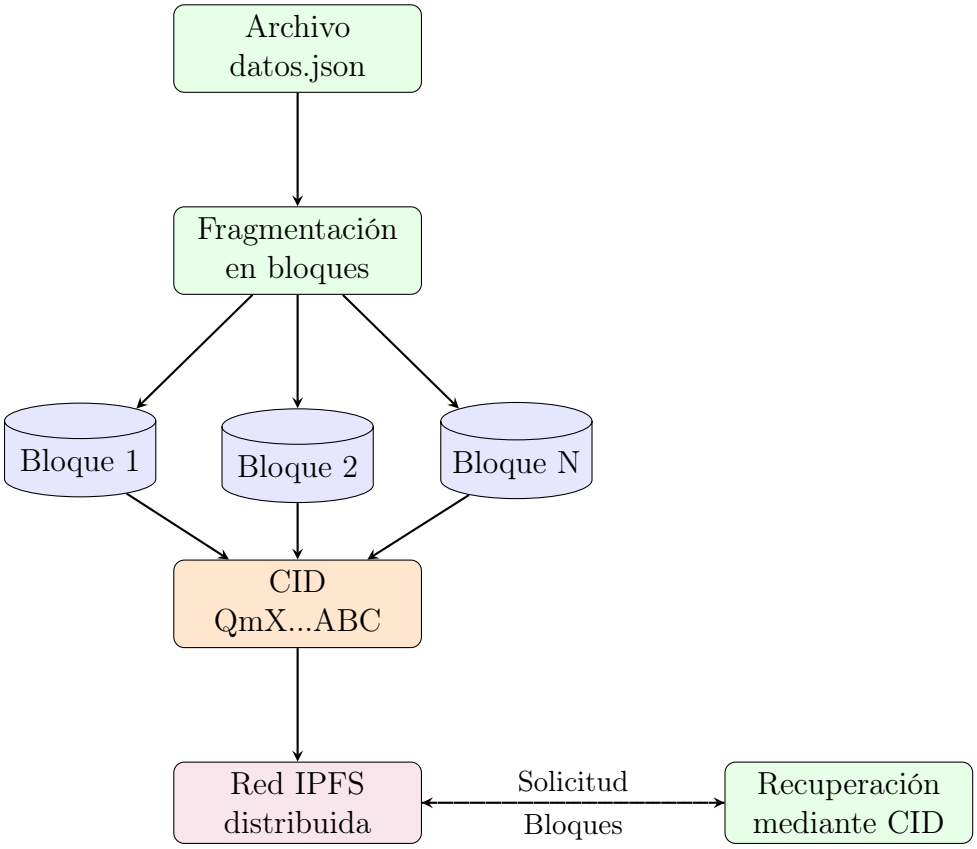



Figura 5.2: Arquitectura básica de IPFS: fragmentación de archivos, generación de CID y recuperación distribuida [8].

En este trabajo, IPFS se emplea como almacén de datos off-chain para los artefactos generados y consumidos por el sistema: mediciones agregadas de tráfico por grupo (clúster), indicadores de congestión difusa, configuraciones optimizadas de tiempos semafóricos y reportes experimentales. El módulo traffic-storage incluye un cliente específico para interactuar con un nodo IPFS local o con un servicio compatible, ofreciendo operaciones sencillas para subir y recuperar objetos. El flujo básico consiste en que la API recibe una carga útil de datos, lo serializa (por ejemplo, como JSON), lo envía al nodo IPFS y obtiene de vuelta un CID.

Ese CID se utiliza a continuación como enlace entre IPFS y las capas de consenso: se registra en blockchain o en BlockDAG mediante las interfaces descritas anteriormente, de modo que cualquier módulo que necesite auditar o reanalizar un conjunto de datos solo requiera conocer el CID y el identificador de semáforo asociado. Desde la perspectiva del resto del sistema, IPFS y las tecnologías de registro inmutable quedan encapsuladas detrás del módulo de almacenamiento: los módulos de simulación, clasificación difusa y optimización interactúan únicamente con la API REST, mientras que las garantías de verificabilidad, integridad y persistencia se logran combinando direccionamiento por contenido en IPFS con registros inmutables en blockchain y/o BlockDAG.

La Figura 5.3 presenta un diagrama de secuencia que ilustra el flujo completo de almacenamiento distribuido: desde la recepción de datos por la API hasta el registro del CID en la capa de consenso, mostrando la interacción entre los componentes del módulo de almacenamiento.

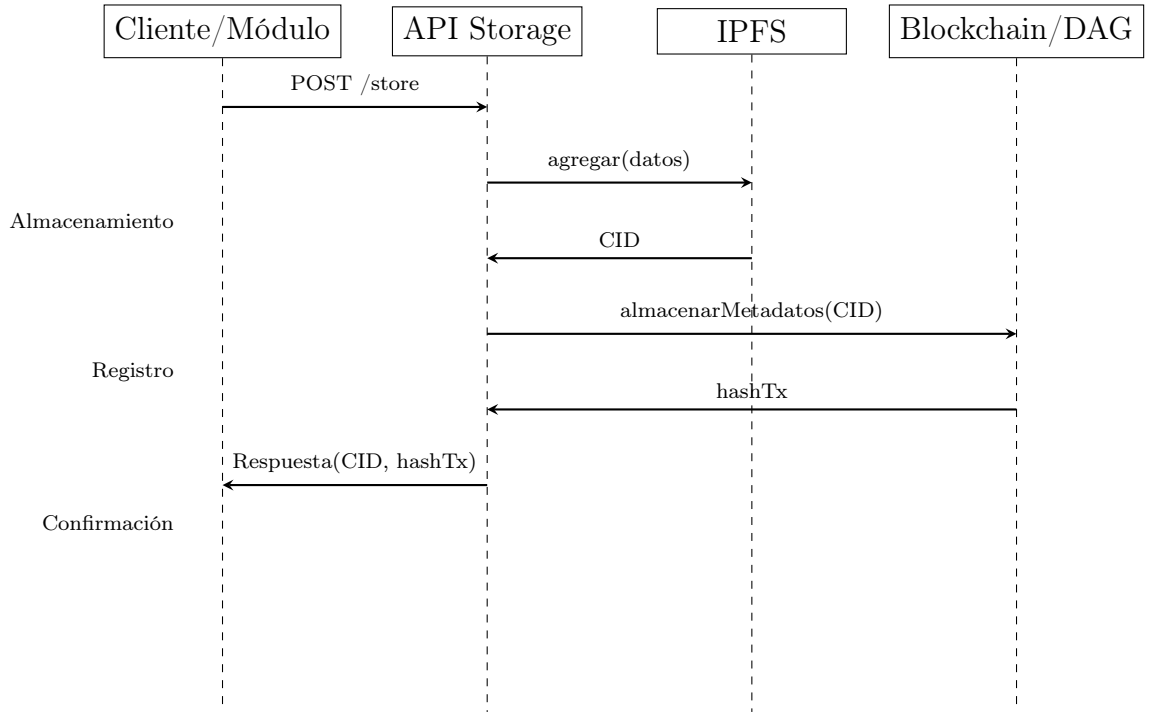



Figura 5.3: Diagrama de secuencia del flujo de almacenamiento distribuido en traffic-storage.

## 5.4. Interfaz del Contrato de Almacenamiento

El contrato inteligente de almacenamiento (Solidity ^0.8.19) es la única pieza de código desplegada on-chain en el sistema. Su diseño deliberadamente minimalista reduce la huella de gas al mínimo: no almacena datos en bruto sino únicamente los CID de IPFS junto con tres claves de búsqueda.

**Tipos.** El enum DataType distingue dos categorías de registros:

- Data (valor 0): lote de métricas de tráfico sin procesar.

- Optimization (valor 1): resultado del ciclo difuso-PSO (largo de ciclo óptimo por clúster, usado posteriormente como techo blando de referencia por el controlador táctico local).

El almacenamiento interno es un mapeo triple privado:

mapping(string =>mapping(uint256 =>mapping(DataType =>string))) private records;

indexado por (trafficLightId, timestamp, dataType), lo que permite acceso O(1) a cualquier registro conociendo las tres claves.

**Funciones y evento.** La Tabla 5.2 describe la interfaz pública completa del contrato.

Tabla 5.2: Interfaz pública del contrato de almacenamiento.

| **Nombre** | **Tipo** | **Descripción** |
|---|---|---|
| storeRecord( string trafficLightId, uint256 timestamp, DataType dataType, string cid) | external | Escribe cid bajo la clave compuesta (trafficLightId, timestamp, dataType) y emite RecordStored. Sin modificador de acceso: cualquier cuenta puede invocarla. |
| getRecord( string trafficLightId, uint256 timestamp, DataType dataType) | external view | Retorna el CID almacenado para la clave compuesta. Revierte con "Record not found" si no existe entrada para esa combinación. |
| RecordStored( string trafficLightId, uint256 timestamp, DataType dataType, string cid) | event | Emitido por storeRecord al completarse. Los indexadores de la red pueden filtrar eventos por semáforo o por tipo para reconstruir el historial de una intersección. |

**Estimaciones de gas.** Medidas sobre la red BlockDAG de prueba con las cargas reales del sistema (véase Capítulo 8), la llamada a storeRecord consume en promedio 36 232 unidades de gas para registros de tipo Data y 95 624 unidades para registros de tipo Optimization. La diferencia refleja el mayor tamaño del CID de IPFS en los paquetes de optimización, que contienen el largo de ciclo optimizado por clúster además de los metadatos. La función getRecord es de solo lectura (view) y no incurre en costo de gas cuando se invoca off-chain.

CAPÍTULO 6

# OPTIMIZACIÓN Y SINCRONIZACIÓN DE TRÁFICO

El módulo traffic-sync implementa el servicio de optimización de tráfico del sistema, exponiendo una API que recibe mediciones macroscópicas de tráfico por semáforo y devuelve un presupuesto de ciclo semafórico optimizado por clúster. Para ello combina tres componentes principales: un sistema de inferencia difusa tipo Mamdani, que evalúa el nivel de congestión a partir de variables de flujo, velocidad y densidad; un agrupamiento jerárquico por enlace de Ward, que reúne sensores con salidas difusas similares en clústeres de tráfico; y un algoritmo de optimización Particle Swarm Optimization (PSO) de una sola variable, que ajusta el largo del ciclo semafórico por clúster con el objetivo de minimizar una función de aptitud que combina la congestión difusa re-evaluada con un costo de espera y de capacidad [25, 6, 7, 43]. traffic-sync opera así como una capa de coordinación estratégica de ciclo: su salida es un presupuesto agregado de verde y rojo por clúster, que la capa de control táctico local reparte entre las fases de cada intersección según la longitud de cola observada en cada carril.

## 6.1. Notación

La Tabla 6.1 reúne los símbolos empleados a lo largo de esta sección, junto con su significado y unidades, dado que varios de ellos se reutilizan entre el sistema difuso, la función de aptitud del PSO y el algoritmo de optimización propiamente dicho.

Tabla 6.1: Notación empleada en la formulación matemática de traffic-sync

| **Símbolo** | **Significado** | **Unidades** |
|---|---|---|
| $VPM$ | Vehículos por minuto observados en el sensor | veh/min |
| $V$ | Velocidad promedio observada | km/h |
| $D$ | Densidad observada (vehicular por longitud de carril) | veh/km |
| congestion | Salida del sistema difuso de Mamdani | escala $[0, 10]$ |
| $C$ | Largo de ciclo, variable de decisión del PSO | s |
| $C^*$ | Largo de ciclo óptimo hallado por el PSO | s |
| $C_{\text{mín}}, C_{\text{máx}}$ | Cotas del rango admisible de $C$ (60 y 120) | s |
| $C_{\text{ref}}$ | Ciclo de referencia bajo el cual se midieron $D$ y $V$ (90) | s |
| $C_{\text{base}}$ | Semilla de ciclo con la que se inicializan las partículas del PSO | s |
| $Y$ | Tiempo total de amarillos y todo-rojo por ciclo (6) | s |
| $cf$, $cf_{\text{ref}}$ | Fracción de ciclo útil del candidato y del ciclo de referencia | adimensional |
| $D_{\text{adj}}, V_{\text{adj}}$ | Densidad y velocidad reescaladas al ciclo candidato $C$ | veh/km, km/h |
| load | Índice compuesto de carga de la red | $[0, 1]$ |
| $\alpha, \beta, \gamma$ | Pesos de densidad, velocidad y flujo en load (0,4/0,4/0,2) | adimensional |
| $D_{\text{máx}}, V_{\text{máx}}, VPM_{\text{máx}}$ | Extremos de los universos de $D$, $V$ y $VPM$ (150, 60, 50) | veh/km, km/h, veh/min |
| $wait_cost$ | Costo de espera; penaliza ciclos largos con carga baja | adimensional |
| $capacity_cost$ | Costo de capacidad; penaliza ciclos cortos con carga alta | adimensional |
| $W$ | Peso relativo de los costos de espera/capacidad (2,0) | adimensional |
| $f(C)$ | Función de aptitud que minimiza el PSO | adimensional |
| $f_{\text{gridlock}}(C)$ | Aptitud con la penalización adicional por atasco real | adimensional |
| $p_{\text{best}_i}$ | Mejor posición histórica de la partícula $i$ | s |
| $g_{\text{best}}$ | Mejor posición hallada por todo el enjambre | s |
| $w$ (PSO) | Coeficiente de inercia (0,8) | adimensional |
| $c_1, c_2$ | Coeficientes cognitivo y social (1,5 cada uno) | adimensional |

## 6.2. Lógica Difusa Mamdani

La lógica difusa proporciona un marco formal para representar razonamiento humano impreciso mediante variables lingüísticas y conjuntos difusos con grados de pertenencia entre 0 y 1 [6, 52]. En un sistema de inferencia Mamdani, las reglas tienen la forma "SI condición ENTONCES acción", donde tanto antecedentes como consecuentes se modelan como conjuntos difusos, y la salida final se obtiene tras un proceso de agregación y defuzzificación [25, 28]. Este enfoque resulta especialmente adecuado para describir estados de tráfico como "flujo libre", "denso" o "congestionado" a partir de variables macroscópicas continuas [29, 26].

La Figura 6.1 ilustra un ejemplo de variable lingüística para velocidad promedio con tres términos (baja, media, alta) definidos mediante funciones de membresía, mostrando cómo un mismo valor numérico puede tener diferentes grados de pertenencia a múltiples categorías lingüísticas.

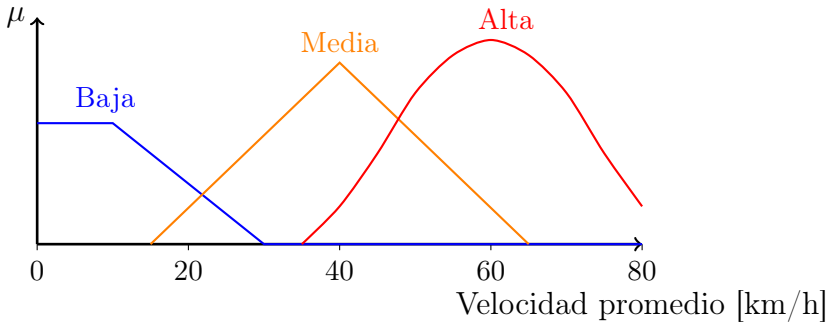



Figura 6.1: Variable lingüística *velocidad promedio* con tres conjuntos difusos modelados mediante funciones de membresía trapezoidal, triangular y aproximadamente gaussiana [28].

En traffic-sync, el sistema de inferencia difusa se implementa en el módulo de lógica difusa, que está compuesto principalmente por el módulo de definición del sistema y el módulo de evaluación. El módulo de definición del sistema define las variables lingüísticas de entrada (por ejemplo, vehículos por minuto, velocidad media, tiempo medio de circulación, densidad) y la variable de salida asociada al nivel de congestión, junto con sus funciones de membresía triangulares y trapezoidales calibradas en base a datos de tráfico y criterios de ingeniería [28, 29]. La base de reglas Mamdani captura relaciones cualitativas del tipo "SI flujo es alto Y velocidad es baja ENTONCES congestión es severa", que reflejan tanto conocimiento experto como patrones observados en los datos [26].

Los universos de discurso sobre los que se definen estas variables lingüísticas son $U_{VPM} = [0, 50]$ veh/min, $U_V = [0, 60]$ km/h, $U_D = [0, 150]$ veh/km y $U_{\text{congestion}} = [0, 10]$, todos discretizados con paso 1 en la implementación de scikit-fuzzy.

La Tabla 6.2 especifica los parámetros exactos de cada función de membresía implementada.

La Figura 6.2 traza la cadena de inferencia completa sobre un caso real, distinguiendo la evaluación de las 27 reglas mediante el conectivo AND como mínimo, la implicación que recorta cada consecuente a la altura de su antecedente, la agregación por máximo y la defuzzificación

Tabla 6.2: Parámetros de las funciones de membresía del sistema difuso Mamdani en traffic-sync

| **Variable** | **Término** | **Tipo** | **Parámetros** |
|---|---|---|---|
| Vehículos/min [0–50] | Bajo | Trapezoidal | [0, 0, 15, 25] |
| | Medio | Gaussiana | $\mu = 30,\ \sigma = 5$ |
| | Alto | Trapezoidal | [35, 40, 50, 50] |
| Velocidad km/h [0–60] | Baja | Trapezoidal | [0, 0, 20, 30] |
| | Media | Gaussiana | $\mu = 35,\ \sigma = 5$ |
| | Alta | Trapezoidal | [40, 50, 60, 60] |
| Densidad veh/km [0–150] | Baja | Trapezoidal | [0, 0, 50, 80] |
| | Media | Gaussiana | $\mu = 100,\ \sigma = 15$ |
| | Alta | Trapezoidal | [120, 130, 150, 150] |
| Congestión [0–10] | Ninguna | Trapezoidal | [0, 0, 2, 4] |
| | Leve | Triangular | [3, 5, 7] |
| | Severa | Trapezoidal | [6, 8, 10, 10] |

por centroide.

El módulo de evaluación expone funciones de alto nivel que reciben vectores de métricas de tráfico en formato estructurado y devuelven, para cada sensor o semáforo, un valor numérico de congestión y su categoría lingüística asociada. Estas funciones se utilizan en el pipeline principal del servicio web: el punto de entrada del servicio recibe un lote de sensores, construye las entradas para el sistema difuso, ejecuta la evaluación Mamdani sobre cada uno y genera un conjunto de salidas que alimentan tanto el agrupamiento jerárquico de sensores como la función de aptitud del PSO. A diferencia de una arquitectura donde la salida difusa fuese en sí misma la función objetivo del optimizador, en el diseño actual la congestión Mamdani es un *término* de una función de aptitud más amplia (Sección 6.4): el sistema difuso re-evaluado sobre las métricas ajustadas por el ciclo candidato aporta la componente de congestión, mientras que términos adicionales penalizan el costo de espera y de capacidad asociados a un ciclo largo o corto. De esta forma, la lógica difusa conserva su rol de módulo de evaluación de desempeño que transforma mediciones de tráfico en un índice de congestión interpretable, pero ese índice pasa a ser un insumo del criterio de optimización en lugar del criterio completo.

La Tabla 6.3 presenta la base de reglas completa del sistema de inferencia Mamdani implementado en traffic-sync. Las 27 reglas cubren todas las combinaciones posibles de los tres términos lingüísticos de cada variable de entrada.

## 6.3. Agrupamiento Jerárquico por Enlace de Ward

Antes de aplicar PSO, traffic-sync agrupa los sensores evaluados por el sistema difuso en clústeres de tráfico con comportamiento similar, de modo que el ciclo semafórico se optimice una vez por grupo y no una vez por sensor individual. El agrupamiento se realiza mediante clustering jerárquico aglomerativo con enlace de Ward, que en cada paso fusiona el par de clústeres que produce el menor incremento en la suma de cuadrados intra-clúster, favoreciendo

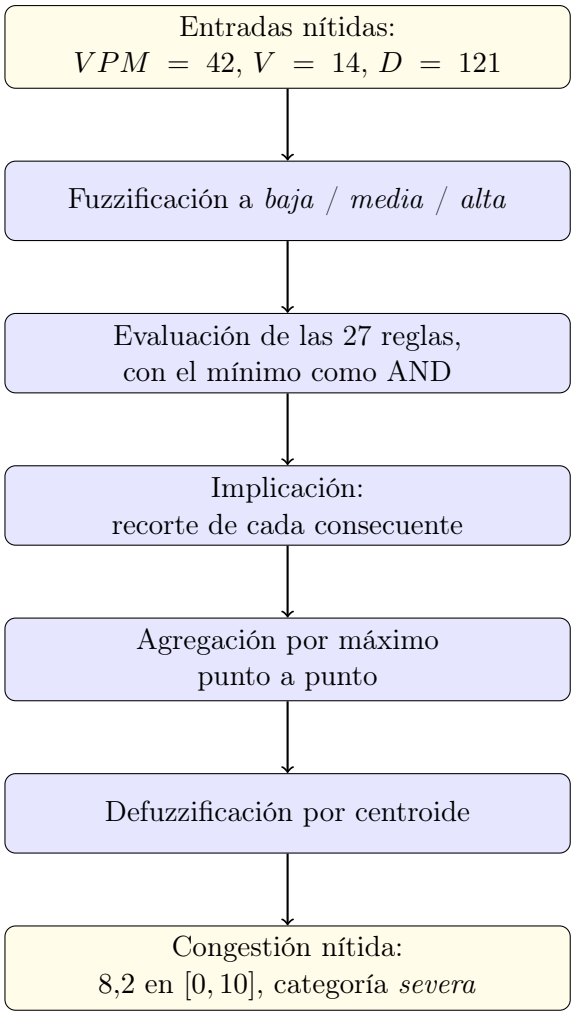



Figura 6.2: Cadena de inferencia difusa de Mamdani, ilustrada con el escenario de congestión severa de la Tabla 6.5. Los recuadros amarillos son datos y los azules, operaciones.

grupos compactos y de varianza mínima [43]. El vector sobre el que se agrupa es unidimensional: el valor *crisp* de congestión que produce la defuzzificación del sistema Mamdani para cada sensor, con distancia euclidiana entre observaciones. Sobre el dendrograma resultante se aplica un corte por distancia con umbral 2: toda fusión que ocurra por debajo de ese umbral de distancia se considera válida, y las ramas resultantes definen los clústeres planos finales.

Una vez asignados los clústeres, se calculan estadísticas descriptivas por grupo a posteriori: la moda de las categorías de congestión esperada y predicha, la media y desviación estándar del valor de congestión, y las medias de vehículos por minuto, velocidad y densidad. Estas estadísticas (en particular las medias de VPM, velocidad y densidad) son las que alimentan tanto la semilla del PSO (Ecuación 6.5 y el cálculo de $C_{\text{base}}$) como la función de aptitud, de modo que cada clúster se optimiza como una unidad representativa de los sensores que agrupa.

Tabla 6.3: Matriz de reglas del sistema de inferencia difusa Mamdani para nivel de congestión

| Vehículos/min | Velocidad | Densidad | | |
|---|---|---|---|---|
| | | Baja | Media | Alta |
| Bajo | Baja | Leve | Leve | Severa |
| | Media | Ninguna | Leve | Leve |
| | Alta | Ninguna | Ninguna | Leve |
| Medio | Baja | Leve | Severa | Severa |
| | Media | Leve | Leve | Severa |
| | Alta | Ninguna | Leve | Leve |
| Alto | Baja | Severa | Severa | Severa |
| | Media | Leve | Severa | Severa |
| | Alta | Leve | Leve | Severa |

## 6.4. Optimización de Enjambre de Partículas (PSO)

Optimización de Enjambre de Partículas (PSO) es un algoritmo de optimización metaheurística inspirado en el comportamiento colectivo de bandadas de aves y bancos de peces, en el que un conjunto de partículas explora el espacio de búsqueda guiado por su mejor experiencia individual y la mejor experiencia global del grupo [7, 53]. Cada partícula representa una solución candidata (en este caso, una posible longitud de ciclo semafórico para un grupo de semáforos) y actualiza su posición combinando tres componentes: inercia, atracción hacia su mejor posición personal y atracción hacia la mejor posición global conocida, ponderadas por parámetros que controlan el equilibrio entre exploración y explotación [54, 55].

La Figura 6.3 ilustra esquemáticamente el concepto del enjambre PSO en un espacio de búsqueda bidimensional, mostrando cómo las partículas se mueven influenciadas por su mejor experiencia personal ($p_i$) y la mejor experiencia global del grupo ($g$).

En el módulo traffic-sync, la lógica de PSO se implementa en el módulo de optimización por enjambre de partículas, que incluye el módulo de función de aptitud y el módulo de optimización. A diferencia de un diseño donde el PSO buscara directamente el tiempo de verde de cada fase, el espacio de búsqueda del optimizador se redujo a una sola variable por clúster: el largo de ciclo semafórico completo $C$. El reparto de ese ciclo entre las fases de cada intersección queda delegado a la capa de control táctico local Max-Pressure de traffic-sim, que decide en tiempo real cuánto verde recibe cada aproximación en función de la longitud de cola observada (Sección 4.2); esta sección no repite esa lógica local y se limita a describir el presupuesto de ciclo que traffic-sync le entrega. El módulo de optimización define la función principal de PSO, que recibe información agregada de tráfico por grupo (clúster) y aplica PSO para determinar el largo de ciclo óptimo por grupo. Cada partícula codifica un largo de ciclo propuesto, inicializado alrededor de un valor base calculado mediante una fórmula de carga normalizada de ingeniería de tráfico (función de cálculo de ciclo base en el módulo de aptitud), y restringido por límites

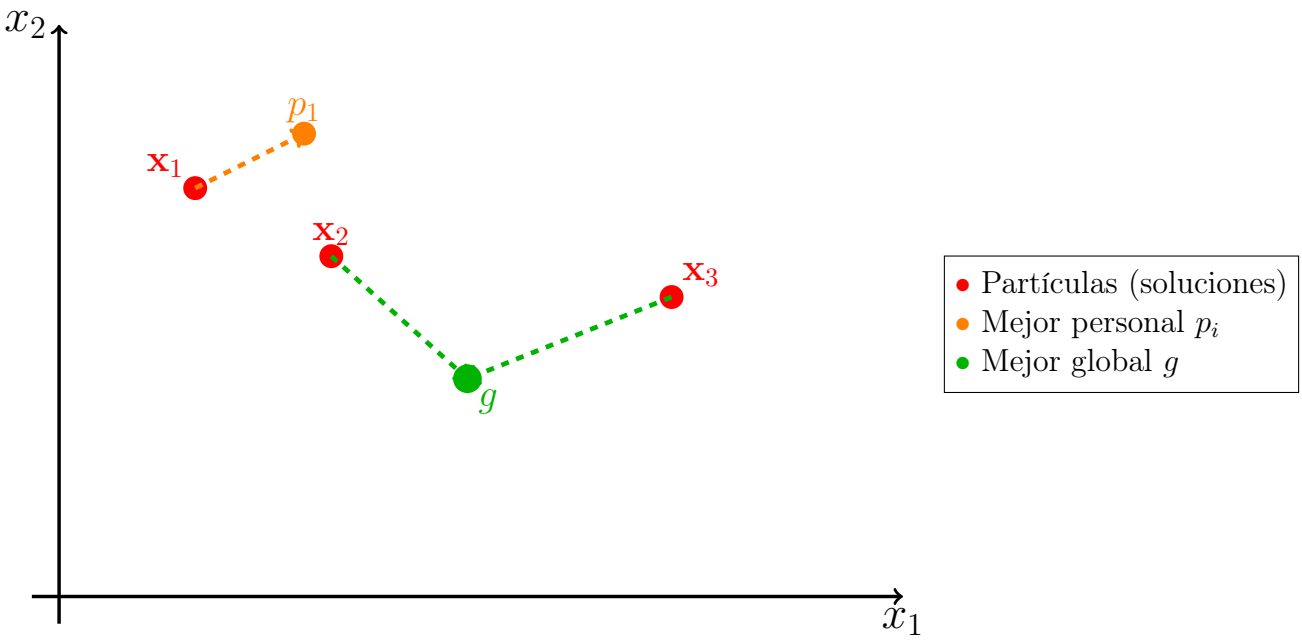



Figura 6.3: Esquema conceptual del enjambre PSO en un espacio de búsqueda bidimensional [7].

mínimos y máximos coherentes con la normativa [17, 2]. El algoritmo utiliza configuraciones típicas de PSO (número de partículas, peso de inercia, componentes cognitiva y social, límites de velocidad) y añade mecanismos prácticos como parada temprana y reinicios parciales para evitar estancamiento en óptimos locales pobres. Dado que el espacio de búsqueda es unidimensional y de rango acotado, el enjambre se redimensionó respecto de implementaciones típicas de PSO multidimensional: un problema en $\mathbb{R}^1$ no requiere ni el número de partículas ni el número de iteraciones que sí serían necesarios para explorar espacios de mayor dimensionalidad.

En cada iteración $k$, la partícula $i$ actualiza su velocidad y su posición según la regla canónica de PSO [7, 55], con la posición recortada al rango admisible de ciclo:

$$\begin{aligned} v_i^{k+1} &= w\, v_i^k + c_1 r_1 \left(p_{\text{best}_i} - x_i^k\right) + c_2 r_2 \left(g_{\text{best}} - x_i^k\right) \\ x_i^{k+1} &= \text{clip}\left(x_i^k + v_i^{k+1},\ C_{\text{mín}},\ C_{\text{máx}}\right) \end{aligned} \tag{6.1}$$

donde $r_1, r_2 \sim \mathcal{U}(0,1)$ se sortean en cada iteración, $w$ pondera la inercia y $c_1$, $c_2$ las atracciones hacia el mejor personal y el mejor global.

La Tabla 6.4 especifica la configuración completa del algoritmo PSO implementado en trafficsync.

La función de aptitud evalúa cada propuesta de largo de ciclo $C$ combinando las métricas del clúster con el sistema difuso Mamdani [1, 3], y agrega a esa congestión difusa un costo de espera y de capacidad que evita que el optimizador degenere hacia el extremo superior del rango de búsqueda. Dado un clúster con valores medios de vehículos por minuto ($VPM$), velocidad ($V$) y densidad ($D$), se define primero la fracción de ciclo útil $cf$ (la porción del ciclo no consumida por la pérdida de amarillo y todo-rojo), evaluada en el ciclo candidato $C$ y en el

Tabla 6.4: Parámetros de configuración del algoritmo PSO en traffic-sync

| **Parámetro** | **Símbolo** | **Valor** |
|---|---|---|
| Número de partículas | $N_p$ | 12 |
| Iteraciones máximas | $T_{max}$ | 40 |
| Factor de inercia | $w$ | 0,8 |
| Componente cognitiva | $c_1$ | 1,5 |
| Componente social | $c_2$ | 1,5 |
| Paciencia (parada temprana) | — | 5 iteraciones |
| Umbral de mejora | $\varepsilon$ | $1 \times 10^{-3}$ |
| Ciclo mínimo | $C_{\text{mín}}$ | 60 s |
| Ciclo máximo | $C_{\text{máx}}$ | 120 s |
| Ciclo de referencia | $C_{\text{ref}}$ | 90 s |
| Pérdida por amarillo/todo-rojo | $Y$ | 6 s |
| Reinicio parcial | — | 2 partículas $\sim \mathcal{N}(C_{\text{base}}, 10)$ si mejor aptitud $> 5$ |
| Espacio de búsqueda | $C$ | $[60,\ 120]$ s |

ciclo de referencia $C_{\text{ref}}$ bajo el cual fueron observadas las métricas:

$$cf = \frac{C - Y}{C}, \qquad cf_{\text{ref}} = \frac{C_{\text{ref}} - Y}{C_{\text{ref}}} \tag{6.2}$$

donde $Y = 6$ s es la pérdida total por amarillo y todo-rojo por ciclo, y $C_{\text{ref}} = 90$ s. A partir de esta razón se ajustan la densidad y la velocidad observadas al ciclo candidato:

$$D_{\text{adj}} = D \cdot \frac{cf_{\text{ref}}}{cf} \tag{6.3}$$

$$V_{\text{adj}} = V \cdot \frac{cf}{cf_{\text{ref}}} \tag{6.4}$$

Un ciclo más largo que $C_{\text{ref}}$ da $cf > cf_{\text{ref}}$, y reduce $D_{\text{adj}}$ y aumenta $V_{\text{adj}}$; un ciclo más corto invierte el efecto. La congestión difusa se re-evalúa con $VPM$ sin ajustar, $V_{\text{adj}}$ y $D_{\text{adj}}$ mediante el sistema Mamdani descrito en la Sección 6.2. A esa congestión se le suma un término de espera y de capacidad ponderado por la carga normalizada del cruce:

$$\text{load} = \alpha \cdot \text{mín}\left(\frac{D}{D_{\text{máx}}},\, 1\right) + \beta \cdot \left(1 - \text{mín}\left(\frac{V}{V_{\text{máx}}},\, 1\right)\right) + \gamma \cdot \text{mín}\left(\frac{VPM}{VPM_{\text{máx}}},\, 1\right) \tag{6.5}$$

con $\alpha = \beta = 0{,}4$ y $\gamma = 0{,}2$.

$$wait_cost = (1 - \text{load}) \cdot \frac{C - C_{\text{mín}}}{C_{\text{máx}} - C_{\text{mín}}}, \qquad capacity_cost = \text{load} \cdot \frac{Y/C}{Y/C_{\text{mín}}} \tag{6.6}$$

$$f(C) = \text{Mamdani}(VPM,\ V_{\text{adj}},\ D_{\text{adj}}) + W \cdot (wait_cost + capacity_cost) \tag{6.7}$$

donde $C_{\text{mín}} = 60$ s, $C_{\text{máx}} = 120$ s son los límites del espacio de búsqueda y $W = 2{,}0$ pondera

los costos de espera y capacidad. *wait_ cost* crece con el ciclo y pesa más cuando la carga es baja; *capacity_ cost* crece cuando el ciclo es corto y pesa más cuando la carga es alta, porque ahí la pérdida fija $Y$ recorta más capacidad relativa. Cuando el clúster está en situación de atasco real (densidad normalizada $D/D_{\text{máx}} \geq 0{,}6$ y velocidad normalizada $V/V_{\text{máx}} \leq 0{,}05$ simultáneamente, condición que se denota *gridlock*), se agrega además un término que penaliza explícitamente los ciclos largos:

$$f_{\text{gridlock}}(C) = f(C) + W \cdot \frac{C - C_{\text{mín}}}{C_{\text{máx}} - C_{\text{mín}}} \tag{6.8}$$

La distinción importa porque con carga alta pero tráfico aún en movimiento, un ciclo más largo sirve más vehículos por ciclo; pero en un atasco real, con velocidad prácticamente nula, un ciclo largo solo empuja más vehículos hacia un corredor que ya no puede drenarlos. El PSO minimiza $f(C)$ (o $f_{\text{gridlock}}(C)$ si corresponde) sobre $C \in [C_{\text{mín}}, C_{\text{máx}}]$, y la semilla inicial de las partículas se calcula con la misma lógica de carga: $C_{\text{base}} = C_{\text{mín}}$ si el clúster está en gridlock, y $C_{\text{base}} = \text{clip}\big(C_{\text{ref}} + (\text{load} - 0{,}5) \cdot 60,\ C_{\text{mín}},\ C_{\text{máx}}\big)$ en caso contrario, de modo que clústeres más cargados que el promedio arrancan la búsqueda desde ciclos más largos y viceversa.

### 6.4.1. Memoización de Resultados

Dado que la función de aptitud (Ecuación 6.7) depende únicamente de las estadísticas agregadas del clúster (vehículos por minuto, velocidad y densidad medias) y no de ningún estado externo, el PSO es determinista respecto de esa terna de valores: para las mismas estadísticas de entrada, el algoritmo converge siempre al mismo largo de ciclo óptimo. traffic-sync explota esta propiedad mediante una caché de resultados en memoria del proceso, indexada por una clave compuesta por las tres medias del clúster redondeadas a un decimal (VPM, velocidad y densidad). Cuando una nueva evaluación de clúster produce una clave ya presente en la caché, se reutiliza directamente el resultado almacenado sin ejecutar el enjambre; en caso contrario, se ejecuta la optimización completa y el resultado se guarda bajo esa clave para evaluaciones futuras.

El redondeo a un decimal es deliberado: convierte diferencias de ruido de medición insignificantes en la misma clave de caché, mientras que sigue distinguiendo condiciones de tráfico genuinamente distintas. Este mecanismo es particularmente relevante porque el orquestador del sistema (traffic-control) aplica un período de enfriamiento (*cooldown*) de 60 s entre solicitudes sucesivas de optimización para un mismo semáforo: bajo tráfico real, las estadísticas agregadas de un clúster cambian poco entre ventanas de cooldown consecutivas, por lo que el acierto de caché es el caso dominante en operación. Empíricamente, una evaluación completa del PSO (enjambre de 12 partículas por hasta 40 iteraciones, cada una re-evaluando el sistema difuso Mamdani) toma del orden de 12 s, mientras que un acierto de caché se resuelve en menos de 1 ms, dos a tres órdenes de magnitud más rápido.

La Figura 6.4 muestra el flujo completo del optimizador, incluida la vía rápida de memoización que domina en operación bajo el cooldown de 60 s descrito arriba.

El pipeline se expone a través de un único punto de entrada de la API, que recibe un lote de sensores y devuelve un lote de optimizaciones. El ciclo óptimo de cada clúster se traduce en un presupuesto de verde y rojo mediante green = red = $\lfloor (C^* - Y)/2 \rfloor$, repartiendo en partes iguales el ciclo útil una vez descontada la pérdida por amarillo y todo-rojo. Cada entrada de la respuesta reúne ese presupuesto, el ciclo óptimo, los niveles de congestión original y optimizado y sus categorías lingüísticas, y la consumen el módulo de control de semáforos, que la reenvía a la capa Max-Pressure local de traffic-sim como techo blando del ciclo, y el módulo de almacenamiento.

En conjunto, la integración entre lógica difusa Mamdani y PSO en traffic-sync permite que la optimización de tiempos semafóricos se base en una medida de congestión que captura adecuadamente la percepción y el comportamiento del tráfico urbano, mientras que el metaheurístico proporciona un mecanismo flexible para ajustar los parámetros de control en escenarios donde la función objetivo no es derivable y debe evaluarse a través de simulación o reglas de inferencia [28, 56].

La Figura 6.5 traza ese recorrido completo, desde la recepción de métricas hasta el presupuesto de ciclo, e incorpora una relación que la descripción secuencial no alcanza a expresar: la realimentación por la que la función de aptitud vuelve a invocar al sistema difuso en cada iteración del PSO, de modo que las etapas difusa y de optimización no se ejecutan una sola vez y en orden, sino anidadas.

## 6.5. Ejemplos de Ciclos de Optimización

La Tabla 6.5 presenta cinco ejecuciones reales del pipeline de optimización de traffic-sync (módulos de clustering, aptitud y PSO invocados directamente sobre datos de un único clúster), con condiciones de tráfico que cubren el rango de operación del sistema: desde flujo libre hasta un atasco real (*gridlock*). En cada caso se muestran las métricas de entrada observadas, el nivel de congestión evaluado por el sistema Mamdani antes de la optimización, el largo de ciclo $C^*$ elegido por PSO junto con el presupuesto de verde y rojo derivado ($\lfloor (C^* - Y)/2 \rfloor$), y los términos de costo de espera y de capacidad (Ecuación 6.6) que explican esa elección de ciclo.

Los valores de la Tabla 6.5 se obtuvieron ejecutando directamente el módulo de optimización PSO de traffic-sync sobre un solo clúster construido con las medias indicadas para cada escenario.

**Carga baja a moderada (flujo libre y flujo moderado).** El ciclo óptimo se mantiene cerca del extremo inferior del rango [60, 120] s, reflejando que la carga normalizada del cruce (Ecuación 6.5) es baja y el término *wait_ cost* domina la aptitud, empujando al optimizador hacia ciclos cortos que minimizan la espera promedio.

**Congestión severa, sin gridlock.** Con velocidad todavía apreciable (14 km/h) y por lo tanto sin cumplir la condición de *gridlock*, el optimizador asigna un ciclo largo (111 s) porque la carga alta hace dominar a *capacity_ cost*, que penaliza los ciclos cortos por la pérdida proporcional de capacidad.

Tabla 6.5: Resultados reales del módulo traffic-sync ante cinco escenarios de tráfico.

| **Entrada (observada)** | | | | **Salida PSO** | | | | |
|---|---|---|---|---|---|---|---|---|
| **Escenario** | **VPM** | **Vel. (km/h)** | **Dens. (veh/km)** | **Congestión** | **Ciclo C* (s)** | **Verde= Rojo (s)** | **Costo espera** | **Costo capac.** |
| Flujo libre | 8 | 48,0 | 12,0 | 1,6 | 60 | 27 | 0,00 | 0,14 |
| Flujo moderado | 18 | 22,0 | 58,0 | 4,9 | 68 | 30 | 0,07 | 0,42 |
| Flujo alto | 28 | 32,0 | 78,0 | 5,0 | 61 | 27 | 0,01 | 0,50 |
| Congestión severa | 42 | 14,0 | 121,0 | 8,2 | 111 | 52 | 0,17 | 0,43 |
| Gridlock | 45 | 2,0 | 135,0 | 8,4 | 60 | 27 | 0,00 | 0,93 |

**Gridlock.** El contraste relevante es con el escenario anterior: pese a tener una densidad y una congestión difusa originales muy similares a las de congestión severa (8,4 frente a 8,2), la velocidad casi nula (2 km/h) activa la condición de atasco real (Ecuación 6.8), y el ciclo óptimo cae al mínimo del rango (60 s) en lugar de crecer, confirmando el comportamiento anti-*gridlock* descrito en la Sección 6.4.

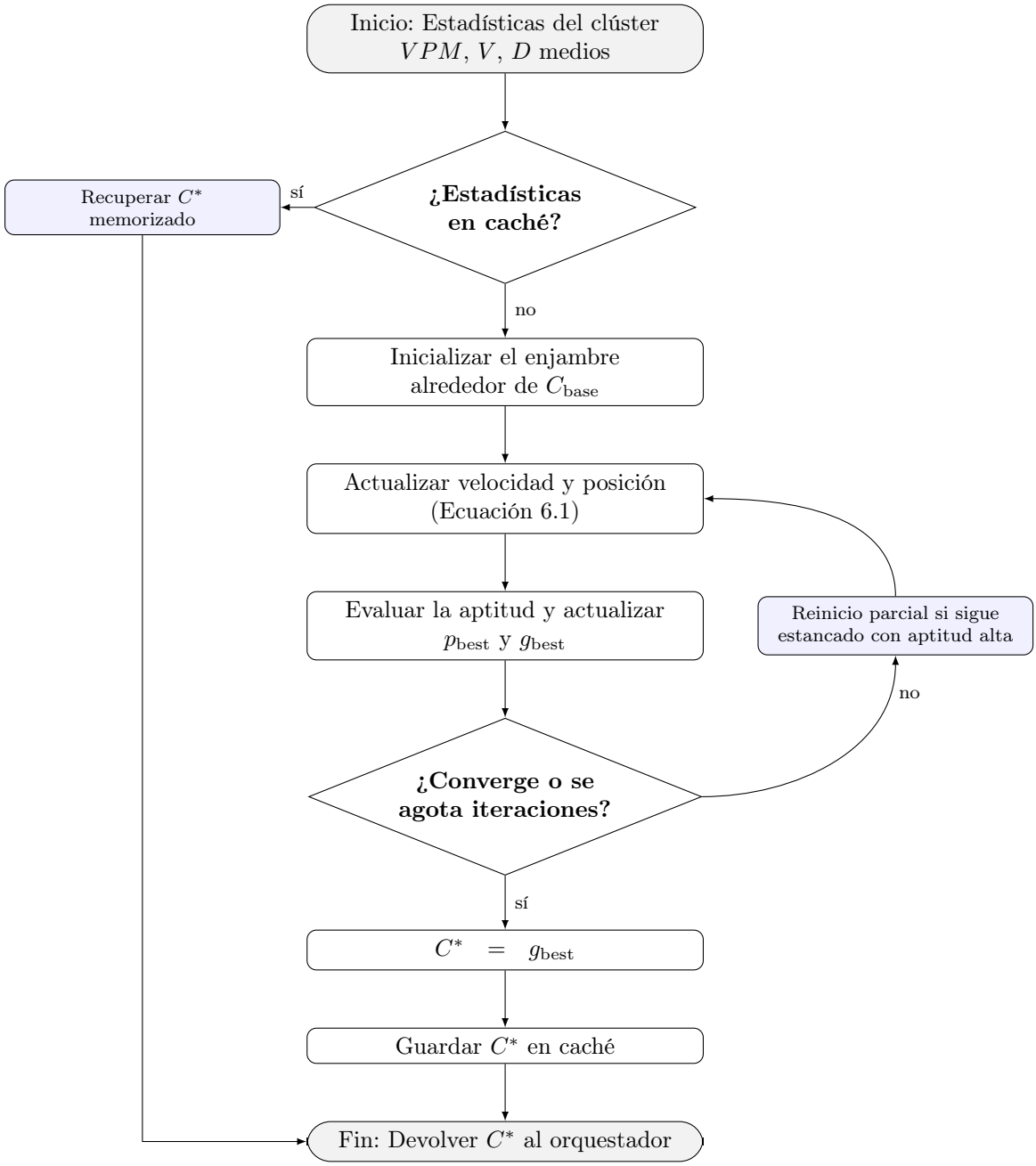



Figura 6.4: Bucle del PSO unidimensional con memoización. Un acierto de caché entrega el ciclo memorizado en menos de un milisegundo; en caso contrario se ejecuta el enjambre hasta converger o agotar su presupuesto de iteraciones. Ambas ramas confluyen en la entrega de $C^*$ al orquestador. El tamaño del enjambre, los umbrales y la regla de reinicio figuran en la Tabla 6.4.

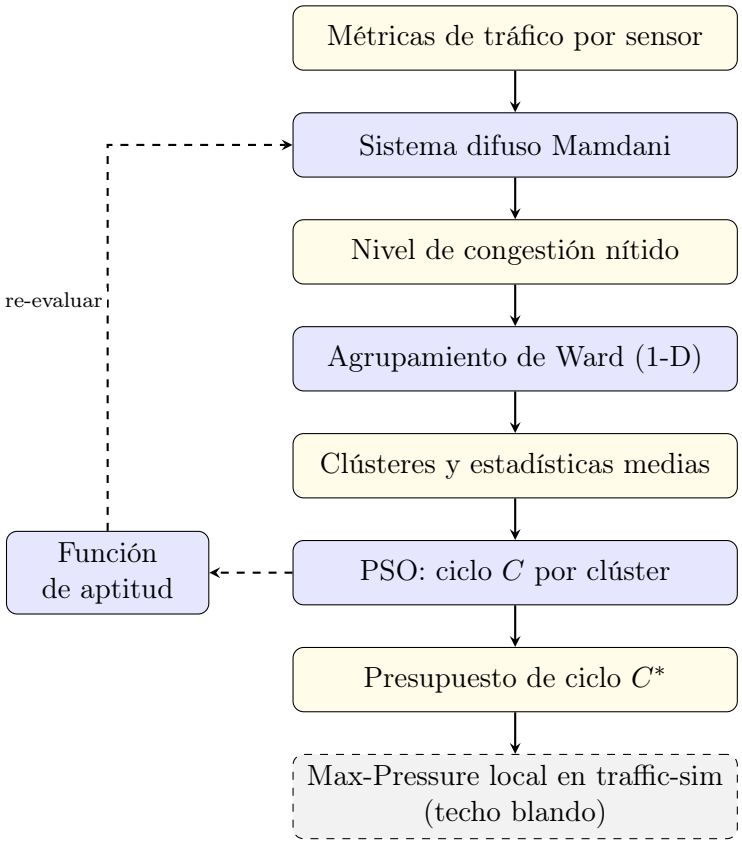



Figura 6.5: Arquitectura del módulo traffic-sync, que integra la lógica difusa Mamdani, el agrupamiento jerárquico de Ward y la optimización PSO del ciclo. La flecha discontinua marca la realimentación: la función de aptitud vuelve a invocar al sistema difuso en cada iteración del enjambre. La capa Max-Pressure local, externa a traffic-sync, recibe el presupuesto de ciclo como techo blando.

# CAPÍTULO 7

# CONTROL Y ORQUESTACIÓN DEL SISTEMA

El módulo traffic-control actúa como orquestador central del sistema distribuido de gestión de tráfico: recibe observaciones vehiculares desde simulaciones o despliegues reales, valida y registra los datos, coordina el envío de información hacia los servicios de optimización (traffic-sync) y almacenamiento distribuido (traffic-storage), y expone una API REST unificada para clientes externos. Su objetivo es desacoplar los productores de datos (por ejemplo, traffic-sim) de los módulos especializados de optimización y persistencia, garantizando un flujo coherente y trazable de información a través del sistema.

## 7.1. Arquitectura del Módulo

La estructura de traffic-control se organiza en capas bien definidas: configuración, API, modelos de datos, servicios de dominio y utilidades de soporte. El servicio web principal (basado en FastAPI) implementa el punto de entrada /process, que procesa observaciones vehiculares completas, y puntos de entrada auxiliares como /healthcheck y operaciones sobre metadatos. El módulo de configuración centraliza parámetros del sistema, como las URLs de los servicios de almacenamiento y optimización, la cadena de conexión a la base de datos y los niveles de logging. El módulo de configuración de logs define la estructura de registro para toda la aplicación.

La capa de modelos incluye esquemas de validación para estructurar peticiones y respuestas (esquemas de datos, optimización y descarga) y modelos de respuesta estandarizados. El módulo de validación encapsula la validación semántica de los datos: verifica versión de la carga útil, formato de marcas de tiempo, tipo de mensaje (datos u optimización), límites de tamaño de lote (1–10 sensores) e integridad de campos como identificador de semáforo, métricas y estadísticas vehiculares. Cualquier inconsistencia o violación de estos contratos se transforma

en una respuesta de error coherente mediante el manejador centralizado de errores.

La persistencia principal del sistema se realiza mediante IPFS y BlockDAG a través del módulo de almacenamiento, que garantiza almacenamiento distribuido, inmutabilidad y trazabilidad verificable. Complementariamente, el módulo de base de datos gestiona un índice auxiliar de metadatos en una base de datos SQL local, con el punto de entrada a la conexión de base de datos y el modelo de metadatos definiendo el esquema de almacenamiento: cada registro guarda, como mínimo, el identificador del semáforo, la marca de tiempo, el tipo de dato (por ejemplo, observación de tráfico u optimización) y referencias a los identificadores de contenido (CIDs y resúmenes criptográficos de transacción) almacenados en el módulo de almacenamiento. El servicio de base de datos proporciona operaciones de alto nivel para crear y consultar este índice local, que se exponen a través de puntos de entrada de metadatos (por ejemplo, /metadata/traffic-light/{id}, /metadata/recent, /metadata/stats) para facilitar consultas rápidas, mientras que los datos completos permanecen en el almacenamiento distribuido.

La Figura 7.1 ilustra la arquitectura del módulo traffic-control, mostrando las capas de API, validación, servicios de dominio y persistencia, así como la integración con los módulos especializados del sistema.

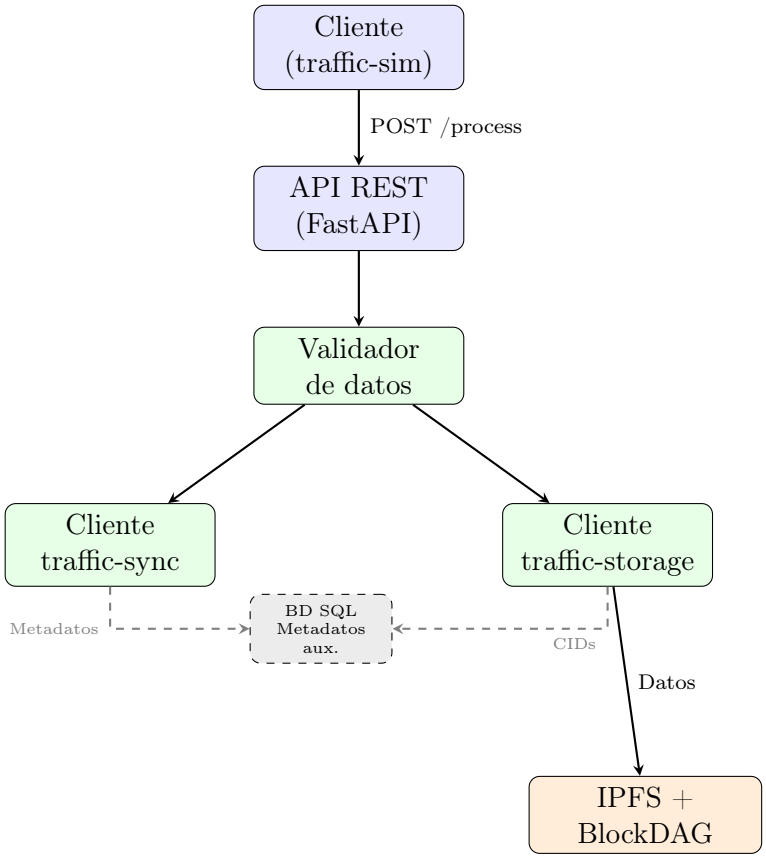



Figura 7.1: Arquitectura del módulo traffic-control mostrando el flujo de orquestación. El almacenamiento principal se realiza mediante IPFS y BlockDAG a través de traffic-storage, mientras que la base de datos SQL se utiliza únicamente como índice auxiliar de metadatos.

## 7.2. Flujo de Procesamiento

El punto de entrada POST /process es el principal para observaciones de tráfico, tanto provenientes de traffic-sim como de otros clientes. El flujo de procesamiento sigue una secuencia de pasos bien definida:

1. **Recepción y validación**: la API recibe una carga útil en formato unificado (versión 2.0), que puede contener uno o varios sensores asociados a un mismo identificador de semáforo. El módulo de validación verifica que el mensaje cumpla con la estructura esperada (campos obligatorios en métricas y estadísticas vehiculares, rango de tamaño de lote, formatos de fecha, tipo de mensaje), generando errores explícitos en caso de inconsistencias.

2. **Preprocesamiento de datos**: el servicio de procesamiento de datos puede aplicar transformaciones ligeras sobre los datos (normalización de campos, ajuste de formatos, enriquecimiento con información temporal) antes de enviarlos a los módulos externos. Esta capa asegura que, aunque distintos productores tengan pequeñas variaciones, el sistema trabaje con un formato interno coherente.

3. **Almacenamiento distribuido principal**: el proxy de almacenamiento construye una petición hacia la API del módulo de almacenamiento, que implementa el almacenamiento principal del sistema mediante IPFS y BlockDAG. Los datos completos se almacenan de forma distribuida e inmutable, recibiendo de vuelta identificadores de contenido (CIDs de IPFS) y resúmenes criptográficos de transacción en BlockDAG. Este es el mecanismo de persistencia central del sistema, garantizando trazabilidad, integridad e inmutabilidad de los registros históricos.

4. **Índice auxiliar de metadatos**: mediante el servicio de base de datos, se registra en una base de datos SQL local un resumen del evento (identificador de semáforo, marca de tiempo, tipo, estado del procesamiento, CIDs y resúmenes criptográficos de transacción), lo que permite consultas rápidas sobre qué datos fueron recibidos, cuándo y con qué resultado. Esta base de datos actúa únicamente como índice auxiliar para facilitar consultas locales, complementando los registros inmutables almacenados en IPFS y BlockDAG gestionados por el módulo de almacenamiento.

5. **Interacción con el módulo de optimización**: el proxy de sincronización envía los datos de tráfico al módulo de optimización mediante el punto de entrada /evaluate, en formato compatible con el sistema difuso y el optimizador PSO. Para lotes de sensores, la función de envío por lotes reempaqueta el lote en el formato esperado por el módulo de optimización (incluyendo el campo de identificador de semáforo de referencia) y recibe un conjunto de optimizaciones, una por grupo detectado.

6. **Respuesta al cliente**: finalmente, el servicio de procesamiento compone una respuesta estandarizada utilizando los modelos de respuesta definidos, indicando el estado del pro-

cesamiento (por ejemplo, éxito o error), un mensaje descriptivo y, cuando corresponda, datos derivados como tiempos de verde optimizados e impacto estimado sobre la congestión. Esta respuesta se devuelve al cliente original (por ejemplo, el módulo de simulación), que puede aplicar las decisiones de control correspondientes.

Durante todo el flujo, el manejador centralizado de errores proporciona mecanismos para capturar y clasificar errores en distintas categorías (validación, almacenamiento, sincronización, base de datos, errores genéricos), generando respuestas HTTP adecuadas y registros de log con contexto detallado para facilitar la depuración.

## 7.3. Interacción con los Módulos

Desde el punto de vista del sistema global, traffic-control se posiciona como el módulo de orquestación que articula la interacción entre simulación, optimización y almacenamiento distribuido:

- **Relación con traffic-sim**: el módulo de simulación basado en SUMO detecta cuellos de botella y, cuando identifica una situación crítica en una intersección, construye una carga útil con métricas agregadas de tráfico y lo envía a traffic-control mediante el punto de entrada /process. De este modo, traffic-sim no necesita conocer detalles sobre traffic-sync ni traffic-storage; sólo interactúa con un servicio central que gestiona validación, almacenamiento y consulta de optimizaciones.

La Figura 7.2 muestra un diagrama de secuencia detallado del flujo completo de procesamiento en traffic-control, ilustrando las interacciones entre todos los componentes del sistema desde la recepción de datos hasta la persistencia distribuida.

- **Relación con el módulo de optimización**: traffic-control actúa como cliente del módulo de optimización a través del proxy de sincronización. Para cada observación (o lote) recibida, reenvía los datos relevantes al módulo de optimización, que ejecuta el pipeline de lógica difusa, agrupamiento de Ward y PSO para producir, por clúster, el largo de ciclo optimizado (presupuesto de verde y rojo que la capa de control local reparte entre fases) y las categorías de congestión. La respuesta del módulo de optimización se integra en la respuesta de traffic-control, permitiendo que los consumidores (por ejemplo, el módulo de simulación) apliquen las configuraciones recomendadas.
- **Relación con el módulo de almacenamiento**: mediante el proxy de almacenamiento, traffic-control sube y recupera datos hacia/desde el módulo de almacenamiento, que a su vez utiliza IPFS, blockchain y BlockDAG como backend de almacenamiento y registro inmutable. traffic-control registra en su base de datos los identificadores de contenido (por ejemplo, CIDs) devueltos por el módulo de almacenamiento, de modo que las consultas de metadatos pueden resolverse localmente y, cuando es necesario, seguir la cadena de referencias hasta los datos almacenados en el sistema distribuido.

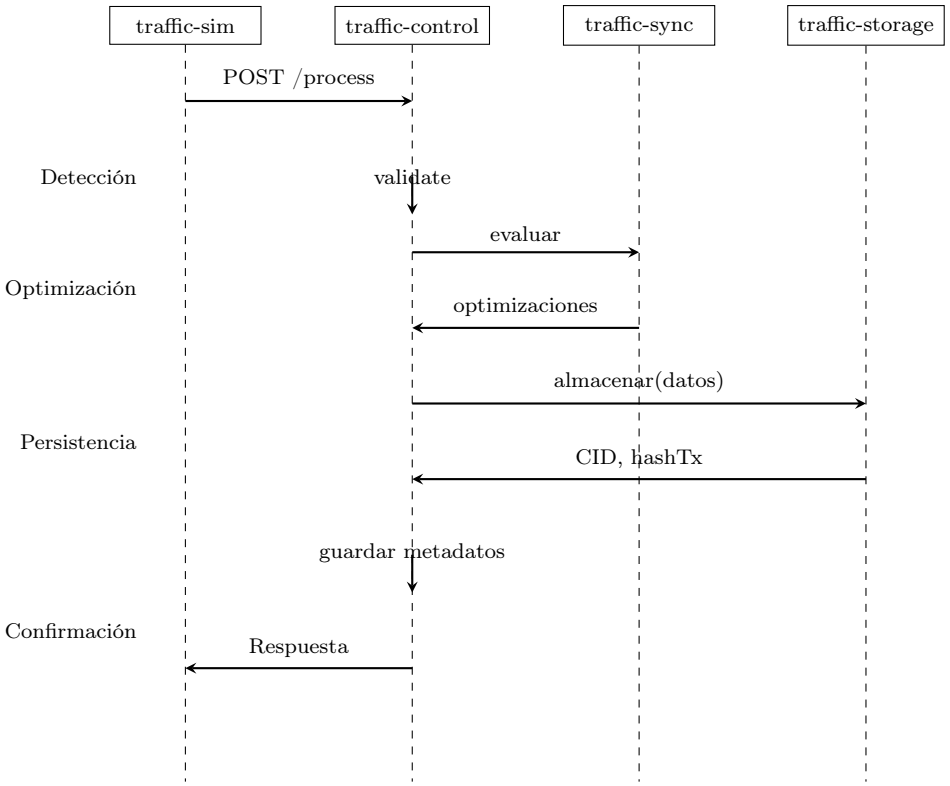



Figura 7.2: Diagrama de secuencia del flujo completo de procesamiento en el sistema distribuido.

Esta arquitectura modular permite que cada componente se especialice en su responsabilidad principal: traffic-sim en la generación de escenarios y detección de cuellos de botella; traffic-sync en la evaluación de congestión y optimización de tiempos semafóricos mediante lógica difusa y PSO; traffic-storage en la persistencia verificable de datos; y traffic-control en la validación, coordinación y exposición de un punto de entrada unificado al sistema. En conjunto, estos módulos forman una plataforma escalable y descentralizada para la gestión inteligente de tráfico urbano.

## 7.4. Contrato de la API REST

La Tabla 7.1 describe el contrato público de traffic-control: los endpoints disponibles, el método HTTP, los parámetros o cuerpo esperado y el código de respuesta en el caso exitoso.

El endpoint POST /process es el punto de entrada principal. Acepta un JSON con los campos version (cadena semver), type ("data"), timestamp (ISO 8601 UTC), traffic_light_id (solo dígitos) y sensors (lista de 1 a 10 elementos con metrics y vehicle_stats). En caso de éxito, la respuesta contiene el estado del procesamiento, los tiempos de verde optimizados devueltos por traffic-sync y los identificadores de contenido (CID) y hash de transacción devueltos por traffic-storage. Los errores de validación retornan 422 (Unprocessable Entity) con detalle de campo; los errores de servicio externo retornan 503.

Los módulos internos exponen sus propios endpoints de uso exclusivo del orquestador: traffic-sync ofrece POST /evaluate (lista de observaciones de sensores, retorna tiempos de verde y nivel

Tabla 7.1: Endpoints de la API REST de traffic-control.

| **Método** | **Ruta** | **Entrada / Parámetros** | **HTTP** |
|---|---|---|---|
| GET | /healthcheck | — | 200 |
| POST | /process | JSON TrafficData (versión 2.0; 1–10 sensores) | 200 |
| GET | /metadata/traffic-light/{id} | ?limit=100 (int) | 200 |
| GET | /metadata/type/{type} | type: data \| optimization; ?limit=100 | 200 |
| GET | /metadata/recent | ?limit=50 (int) | 200 |
| GET | /metadata/stats | — | 200 |
| DELETE | /metadata/traffic-light/{id} | — | 200 |

de congestión por grupo), y traffic-storage ofrece POST /upload/ (sube datos a IPFS y registra el CID en BlockDAG, retorna CID y hash de transacción) y POST /download/ (recupera datos de IPFS a partir de un CID). Estos endpoints no forman parte de la interfaz pública del sistema y no son accedidos directamente por traffic-sim.

# CAPÍTULO 8

# EVALUACIÓN EXPERIMENTAL DEL SISTEMA

Los capítulos anteriores han descrito la arquitectura, los fundamentos teóricos y la implementación de cada módulo del sistema propuesto. El presente capítulo tiene como objetivo demostrar experimentalmente que el sistema cumple con los requisitos de rendimiento, integridad y eficiencia planteados para su aplicación en entornos de gestión de tráfico urbano. Para ello, se presentan los resultados de experimentos controlados ejecutados sobre los dos módulos que constituyen el núcleo computacional del sistema: traffic-sync, responsable del análisis de congestión y la optimización semafórica, y traffic-storage, encargado de la persistencia verificable y descentralizada de datos de tráfico.

Las evaluaciones persiguen cinco objetivos específicos. En primer lugar, se estudia la escalabilidad del módulo de análisis y optimización, analizando cómo evolucionan el tiempo de procesamiento y el consumo de recursos del sistema conforme aumenta el número de sensores simulados que operan simultáneamente. En segundo lugar, se mide la eficiencia del subsistema de almacenamiento distribuido, cuantificando las latencias extremo a extremo de las operaciones de escritura y lectura sobre la capa IPFS–BlockDAG bajo distintas condiciones de carga vehicular. En tercer lugar, se verifica la integridad de los datos almacenados mediante verificación criptográfica, comprobando que los registros recuperados desde infraestructuras descentralizadas conservan fielmente el contenido original. En cuarto lugar, se contextualiza el desempeño obtenido mediante una comparación con tecnologías alternativas de almacenamiento distribuido (particularmente con redes blockchain basadas en Ethereum) con el fin de evidenciar las ventajas operativas de la arquitectura propuesta para aplicaciones de monitoreo urbano de alta frecuencia. En quinto lugar, se valida el desempeño del control semafórico jerárquico (optimización de ciclo mediante PSO combinada con control local Max-Pressure) mediante tres campañas de comparaciones A/B, cada una frente a un plan de tiempo fijo de referencia distinto (MOPC 60 s, netconvert 90 s y ciclo largo 120 s), multi-escenario y multi-semilla, que

permiten establecer bajo qué condiciones de demanda el sistema propuesto supera a la práctica de temporización fija vigente.

Todos los experimentos se ejecutaron en hardware de consumo estándar, sin requerir infraestructura especializada, lo que pone de manifiesto la viabilidad del sistema para despliegues en entornos municipales con recursos computacionales limitados.

Los resultados de escalabilidad de traffic-sync y de eficiencia de traffic-storage presentados en las Secciones 8.1 y 8.2 fueron publicados originalmente en [10] y se reproducen aquí como parte de la evaluación integral del sistema completo, incluyendo el control semafórico jerárquico descrito en la Sección 8.4.

## 8.1. Evaluación del módulo traffic-sync

El módulo traffic-sync constituye el motor analítico del sistema: recibe métricas de tráfico, clasifica el estado de congestión mediante un sistema de inferencia difusa tipo Mamdani y calcula las configuraciones óptimas de temporización semafórica mediante el algoritmo PSO. Si bien en la operación actual el módulo procesa un sensor a la vez, la arquitectura fue diseñada desde el inicio para soportar la evaluación conjunta de múltiples sensores agrupados por zonas urbanas. Con el fin de anticipar este escenario de uso futuro, en el que los clústeres de semáforos deberán analizarse en paralelo para lograr una dispersión efectiva del tráfico, se realizaron pruebas de escalabilidad simulando entre 250 y 2000 sensores emitiendo datos de forma simultánea.

### 8.1.1. Entorno y Protocolo de Prueba

Los experimentos se realizaron en dos máquinas de cómputo de propósito general, sin hardware acelerador ni red especializada, reflejando las condiciones típicas de un despliegue municipal de bajo presupuesto; ambas configuraciones corresponden a hardware de consumo estándar disponible comercialmente. La Tabla 8.1 resume las especificaciones de ambos entornos.

Las transacciones de almacenamiento se enviaron a la red *Primordial TestNet*, una red BlockDAG de pruebas con identificador de cadena (Chain ID) 1043, accesible a través del nodo RPC público https://rpc.primordial.bdagscan.com [49], sobre el contrato inteligente de almacenamiento (Solidity 0.8.19, dirección 0xC3d520EBE9A9F52FC5E1519f 17F5a9A01d8ac68f) cuya interfaz se describe en el Capítulo 5. La capa de almacenamiento en IPFS se implementó mediante Pinata[1] como servicio de nodo gestionado.

Se definieron siete niveles de carga: 250, 500, 1000, 1250, 1500, 1750 y 2000 sensores simulados. Para cada nivel, se midieron tres métricas: el tiempo total de ejecución del ciclo de análisis y optimización (en segundos), el uso promedio de CPU durante el procesamiento (en porcentaje) y el uso de memoria RAM en el punto de mayor carga (en megabytes). Las pruebas se ejecutaron en el Entorno A (laptop AMD Ryzen 5 5500U, Ubuntu 24.04 LTS), sin soporte de GPU y bajo condiciones de red doméstica, reflejando un escenario de despliegue realista en

[1] https://pinata.cloud

Tabla 8.1: Especificaciones del entorno experimental

| **Parámetro** | **Entorno A (traffic-sync)** | **Entorno B (traffic-storage)** |
|---|---|---|
| Procesador | AMD Ryzen 5 5500U | AMD Ryzen 7 3700U |
| Núcleos / Hilos | 6 / 12 | 4 / 8 |
| Frec. máxima | 4,05 GHz | 2,3 GHz (base) |
| RAM disponible | 7,1 GB | 5,9 GB |
| Almacenamiento | N/A | 233 GB SSD NVMe |
| Sistema operativo | Ubuntu 24.04 LTS | Ubuntu 24.04.2 LTS |
| GPU | No utilizada | No utilizada |
| Red | Wi-Fi doméstica | Wi-Fi doméstica |

hardware de bajo costo. Los datos de entrada correspondieron a observaciones sintéticas en el formato unificado versión 2.0, incluyendo métricas de velocidad promedio, densidad vehicular y número de vehículos por minuto.

### 8.1.2. Tiempo de Ejecución

La Figura 8.1 muestra la evolución del tiempo total de ejecución en función de la cantidad de sensores simulados. Los valores obtenidos, que van desde 15,87 s para 250 sensores hasta 67,59 s para 2000 sensores, evidencian un crecimiento aproximadamente lineal con una pendiente de $\approx 0{,}034$ s por sensor adicional procesado. Este comportamiento lineal es altamente favorable: indica que el módulo no incurre en sobrecarga superlineal a medida que se incrementa la carga de trabajo, y que los tiempos de respuesta son predecibles y controlables mediante particionamiento horizontal de los sensores. Para la escala más exigente evaluada (2000 sensores), el ciclo completo de análisis y optimización se completa en 67,6 s, lo que representa una capacidad de $\approx 0{,}034$ s/sensor y confirma la viabilidad del módulo para escenarios de optimización por zonas a escala urbana real.

### 8.1.3. Uso de CPU y Memoria

La Figura 8.2 presenta de forma simultánea el uso promedio de CPU y el consumo de memoria RAM bajo cada nivel de carga. El comportamiento de ambas métricas exhibe tendencias en apariencia contrapuestas pero explicables por la naturaleza de la implementación.

El uso de CPU disminuye progresivamente desde el 72,3 % observado con 250 sensores hasta el 50,3 % con 2000 sensores. Esta reducción relativa se explica por la naturaleza de la medición: el porcentaje de CPU se calcula como la razón entre el tiempo de procesador activo y el tiempo total transcurrido (tiempo de pared). A medida que el volumen de sensores crece, el tiempo de pared aumenta de forma aproximadamente lineal, mientras que el tiempo de CPU activo por unidad de trabajo permanece prácticamente constante; el denominador crece más rápido que el numerador. Entre lotes sucesivos existen períodos de relativa inactividad (serialización de respuestas, despacho de peticiones HTTP, sobrecarga del intérprete Python) que incrementan

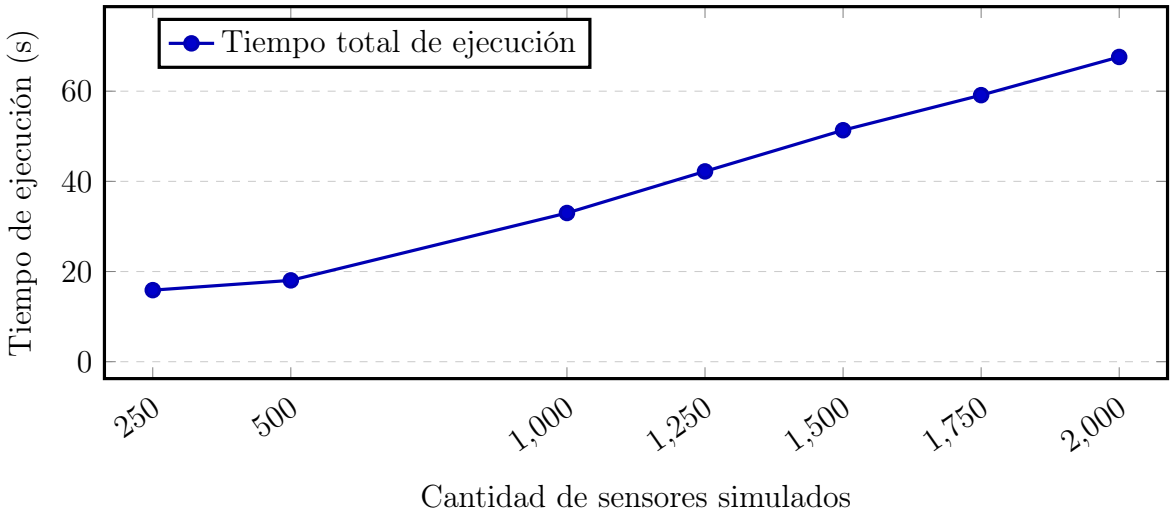



Figura 8.1: Tiempo de ejecución del módulo traffic-sync en función de la cantidad de sensores simulados. El crecimiento aproximadamente lineal (≈0,034 s/sensor) confirma la escalabilidad del módulo.

el tiempo de pared sin contribuir proporcionalmente a los ciclos de procesador. Cabe señalar que tanto el sistema de inferencia difusa Mamdani como el optimizador PSO son operaciones mayoritariamente CPU-*bound* por unidad; la caída del porcentaje refleja un efecto de dilución temporal, no una reducción del trabajo computacional efectivo por sensor.

El consumo de memoria RAM, por su parte, se mantiene estable en el rango de 198–219 MB en todos los escenarios evaluados, creciendo apenas 20,8 MB al pasar de 250 a 2000 sensores (+10,5 %). Esta estabilidad es un indicador clave de la viabilidad del módulo en dispositivos de recursos acotados, como unidades de cómputo embebido tipo Raspberry Pi, donde la memoria disponible suele ser el factor limitante más crítico.

## 8.2. Evaluación del módulo traffic-storage

El módulo traffic-storage es la capa de persistencia verificable del sistema. Ante cada reporte de tráfico o resultado de optimización, el módulo sube el contenido en formato JSON a IPFS y registra el identificador de contenido (CID) resultante en el contrato inteligente de la red BlockDAG, vinculándolo de forma inmutable al semáforo de origen y la marca de tiempo correspondiente. La evaluación de este módulo se realizó mediante un protocolo de prueba por escenarios de tráfico vehicular simulado, que caracteriza la latencia, el costo y el consumo de gas del pipeline completo, y valida su comportamiento bajo distintas densidades de datos.

### 8.2.1. Métricas de Transacción Base

Para caracterizar el desempeño del pipeline de almacenamiento en condiciones controladas, se ejecutaron transacciones reales de subida y descarga contra la API de traffic-storage, con almacenamiento en IPFS mediante el servicio Pinata y registro en la red BlockDAG Primordial

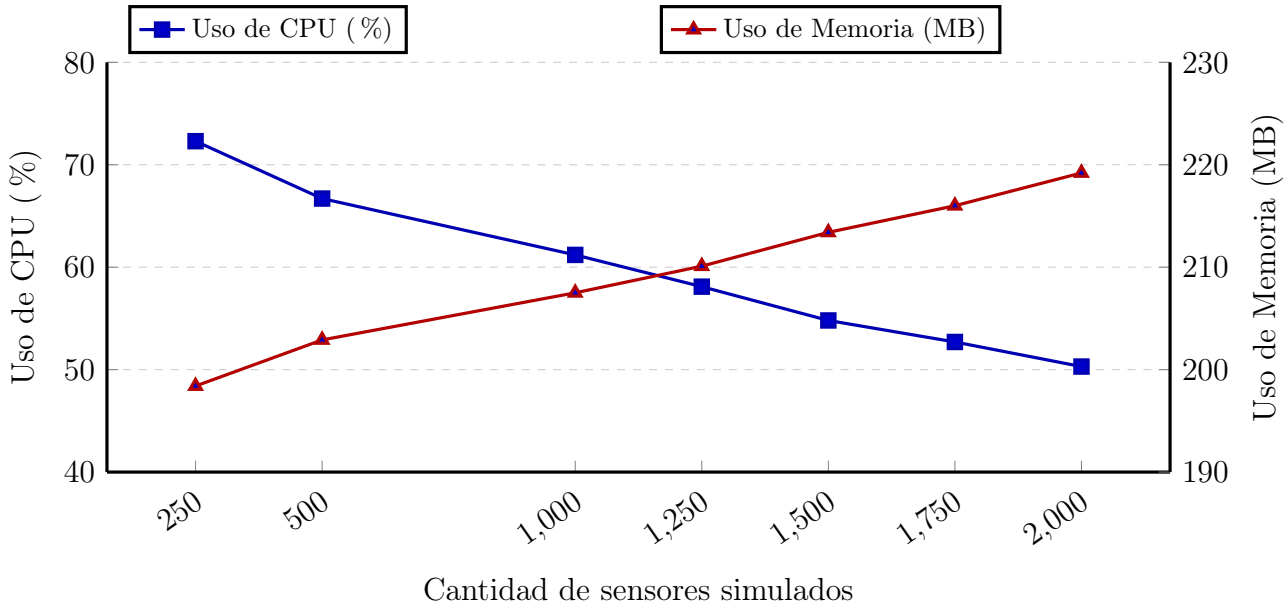



Figura 8.2: Uso promedio de CPU y consumo de memoria RAM del módulo traffic-sync bajo diferentes cargas de sensores simulados. El CPU disminuye conforme aumenta la concurrencia; la memoria permanece prácticamente estable en el rango 200–220 MB.

TestNet. Se definieron tres escenarios de densidad vehicular (bajo, medio y alto) con cinco corridas por escenario y operación, lo que totaliza 15 subidas y 15 descargas exitosas (código HTTP 200). Se midieron el tiempo total de la operación extremo a extremo, medido desde el inicio de la petición HTTP, y el consumo de gas y el costo por transacción. El tiempo de confirmación en la red se obtuvo de los registros de ejecución detallados (Sección 8.2.2). La Tabla 8.2 resume los resultados obtenidos.

Tabla 8.2: Métricas de rendimiento del módulo traffic-storage (IPFS + BlockDAG)

| **Métrica** | **Subida (upload)** | **Bajada (download)** |
|---|---|---|
| Tiempo total (s) | $6{,}97 \pm 0{,}90$ | $1{,}83 \pm 0{,}49$ |
| Confirmación (s) | $\approx 3{,}5^{\ddagger}$ | — |
| Gas utilizado (upload) | $36{,}232$–$95{,}624^{\dagger}$ | — |
| Costo estimado (USD) | $< 0{,}00002$–$< 0{,}00005^{\dagger}$ | — |

Media $\pm$ desviación estándar muestral sobre $n = 15$ corridas por operación. $^{\ddagger}$ Obtenido de los registros de ejecución detallados (Sección 8.2.2); la descarga no requiere confirmación.
$^{\dagger}$ Rango entre datos de tráfico (36.232 unidades, <$0,00002) y resultados de optimización (95.624 unidades, <$0,00005); la descarga usa una función de solo lectura (*view*) sin costo on-chain. Detalle por tipo en Tabla 8.3.

El tiempo de subida extremo a extremo se mantuvo entre 6,7 y 7,3 s en promedio según el escenario (bajo: 6,7 s; medio: 6,9 s; alto: 7,3 s) y la descarga en torno a 1,8 s. La confirmación en la red BlockDAG, del orden de 3,5 s, representa cerca de la mitad de la latencia de subida, y el costo por transacción se mantuvo inferior a $\$5 \times 10^{-5}$ USD según el tipo de dato almacenado. Estos valores son adecuados para el registro verificable de reportes y resultados de optimización, que no forman parte del lazo de control en tiempo estricto. Estas mediciones, obtenidas de los registros de ejecución completos del módulo, actualizan las cifras preliminares del artículo de

conferencia. La comparación con tecnologías alternativas se detalla en la Sección 8.3.

### 8.2.2. Descomposición del Pipeline de Almacenamiento

El tiempo total de una operación de subida no es atómico: comprende varias etapas secuenciales con latencias propias. El análisis de los registros de ejecución detallados permite identificar las tres fases que componen el pipeline de almacenamiento. La primera fase es la subida a IPFS: la serialización del payload JSON, la autenticación con la API de Pinata y la transferencia del contenido hasta obtener el CID insume aproximadamente 1,2 s. La segunda fase es la preparación de la transacción BlockDAG: incluye la verificación del estado del nodo y el balance de la cuenta, el cálculo dinámico del precio de gas, la obtención del nonce actual y la firma criptográfica de la transacción; en conjunto, esta fase demanda aproximadamente 2,3 s. La tercera fase es la confirmación en la red: el tiempo que transcurre desde el envío de la transacción hasta que es incluida en un bloque minado y confirmada es de aproximadamente 3,5 s (3,48 s en la corrida trazada en detalle). La suma de las tres fases ($\approx$ 7,0 s) es coherente con el tiempo total de subida medido (Tabla 8.2).

La Figura 8.3 muestra la composición proporcional de estas etapas para las operaciones de subida y descarga. En el caso de la descarga, el pipeline se reduce a dos fases: la consulta al contrato inteligente para recuperar el CID asociado a un semáforo y timestamp determinados ( 0,9 s), seguida de la descarga del contenido desde el gateway de IPFS usando dicho CID ( 0,7 s), con un tiempo total de  1,6 s en la corrida trazada (1,83 s en promedio sobre las 15 descargas).

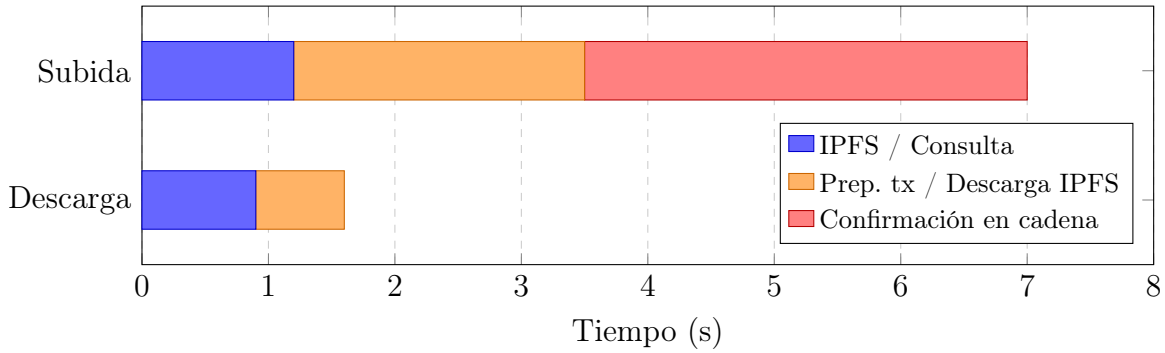



Figura 8.3: Desglose del tiempo de ejecución en las operaciones de subida y descarga del módulo traffic-storage. La confirmación en la red BlockDAG domina la latencia de subida ($\approx$50 %), mientras que la descarga se completa en $\approx$1,6 s sin necesidad de esperar confirmación.

### 8.2.3. Consumo de Gas y Costos Operativos

El consumo de gas de las transacciones en la red BlockDAG varía según el tipo de dato almacenado. Las operaciones de almacenamiento de datos de tráfico sin procesar consumen aproximadamente 36.232 unidades de gas, mientras que las operaciones de almacenamiento de resultados de optimización, que incluyen un mayor volumen de campos calculados, consumen

aproximadamente 95.624 unidades. La Tabla 8.3 resume el consumo y el costo estimado en dólares para cada tipo de operación.

Tabla 8.3: Consumo de gas y costo estimado por tipo de operación de almacenamiento

| **Tipo de dato** | **Gas consumido (unidades)** | **Costo estimado (USD)** |
|---|---|---|
| Datos de tráfico (Data) | 36.232 | $< 0{,}00002$ |
| Optimizaciones (Optimization) | 95.624 | $< 0{,}00005$ |

Incluso para el tipo de operación más costosa, el almacenamiento de resultados de optimización con 95.624 unidades de gas, el costo por transacción se mantiene por debajo de \$0,00005 USD. Considerando una frecuencia de reporte típica de un ciclo cada 60 segundos por semáforo, el costo anual de almacenamiento on-chain para un semáforo individual sería inferior a \$0,88 USD según las condiciones observadas durante los experimentos. Sin embargo, esta estimación debe interpretarse con cautela: los experimentos se realizaron sobre una red BlockDAG pública de pruebas (testnet), cuyo token no posee valor de mercado real. Los costos en una red de producción (mainnet) dependerán del precio del token en el momento del despliegue y de la demanda de la red, por lo que este valor constituye únicamente un orden de magnitud comparativo, no una proyección de costo operativo real.

## 8.3. Comparación con Tecnologías Alternativas

Los resultados experimentales adquieren mayor significado cuando se los sitúa en relación con las alternativas tecnológicas disponibles para el almacenamiento distribuido con trazabilidad. Esta sección compara la solución adoptada, IPFS combinado con una red BlockDAG, frente a la plataforma blockchain de propósito general más utilizada, Ethereum, tanto desde la perspectiva teórica apoyada en literatura existente como desde los valores medidos experimentalmente.

### 8.3.1. BlockDAG frente a Ethereum

Las características de ambas plataformas se describieron en el Capítulo 5; los resultados medidos en este trabajo, con confirmación en $\approx$3,5 s y costos inferiores a $\$2\times10^{-5}$ USD por transacción de datos de tráfico, ratifican las ventajas teóricas de BlockDAG en condiciones de uso real sobre la red Primordial TestNet. La Tabla 8.4 sintetiza la comparación desde la perspectiva del monitoreo urbano de tráfico.

## 8.4. Evaluación del control semafórico: campaña A/B

Mientras que las secciones anteriores caracterizan el desempeño computacional de los módulos traffic-sync y traffic-storage de forma aislada, esta sección evalúa el efecto del control

Tabla 8.4: Comparación técnica entre Ethereum y BlockDAG en el contexto de monitoreo urbano de tráfico. Valores de BlockDAG medidos en Primordial TestNet.

| **Criterio** | **Ethereum** | **BlockDAG** |
|---|---|---|
| Latencia de confirmación | 15–60 s [57, 48] | ≈3,5 s (medido) |
| Costo por transacción | $0,50–5,00 USD [58] | <$0,00002 USD (medido) |
| Throughput (TPS) | 15–30 [47] | >1.000 [9] |
| Modelo de consenso | Proof-of-Stake | PoW asíncrono |
| Estructura de datos | Cadena lineal | Grafo acíclico |
| Contratos inteligentes | Maduro (EVM, Solidity) | Disponible (EVM-compatible) |
| Escalabilidad | Limitada | Alta |
| Compatibilidad edge/IoT | Baja (nodos pesados) | Alta (nodos ligeros) |

semafórico jerárquico propuesto, PSO de ciclo combinado con Max-Pressure local, sobre el tránsito vehicular en simulación, en comparación directa con planes de tiempo fijo de referencia representativos de la práctica vigente. Esta campaña constituye la validación empírica sobre un mapa real de la ciudad de San Lorenzo, Paraguay, que [10] planteó como trabajo en curso, ejecutada ahora sobre la versión jerárquica del control (PSO de ciclo por clúster más Max-Pressure local) en lugar del ajuste directo de tiempos de señal evaluado en aquel trabajo.

### 8.4.1. Diseño experimental

El protocolo de evaluación se basa en comparaciones pareadas A/B: para cada combinación de un plan fijo de referencia, un escenario de demanda y una semilla aleatoria, se ejecutan dos corridas de simulación sobre la misma red vial, la misma matriz de demanda y la misma semilla del motor SUMO, difiriendo únicamente en el mecanismo de control semafórico. La corrida A opera con el plan de tiempo fijo de referencia y la corrida B con el sistema completo propuesto (PSO de ciclo más Max-Pressure local). La red vial utilizada corresponde al microcentro de la ciudad de San Lorenzo, con seis intersecciones semaforizadas cuya lógica de fases fue generada automáticamente mediante netconvert, la herramienta estándar de generación de redes de SUMO. Se consideraron tres planes fijos de referencia, seis escenarios de demanda y diez semillas aleatorias independientes por celda, lo que produce $3 \times 6 \times 10 = 180$ comparaciones A/B. Las semillas se generaron mediante un generador criptográficamente seguro (CSPRNG) y quedaron registradas para garantizar la reproducibilidad exacta de cada corrida.

La campaña se ejecutó en una máquina virtual de propósito general, sin hardware acelerador, cuyas especificaciones se resumen en la Tabla 8.5.

### 8.4.2. Escenarios de demanda

Los seis escenarios evaluados se derivaron de forma determinista, sin componentes aleatorios, a partir de un mismo archivo base de rutas, de modo que las diferencias entre escenarios provienen exclusivamente de la manipulación programática de los volúmenes y patrones de

Tabla 8.5: Especificaciones de la máquina que ejecutó la campaña A/B de control semafórico

| **Parámetro** | **Valor** |
|---|---|
| Sistema operativo | Ubuntu 24.04.4 LTS (Noble Numbat), aarch64 |
| Virtualización | Máquina virtual KVM (virt-7.2) |
| Kernel | Linux 6.17.0-1020-oracle |
| Procesador | ARM Neoverse-N1 (2 núcleos) |
| RAM | 11,65 GiB (13 % en uso durante la ejecución) |
| Almacenamiento | 192,69 GiB, ext4 (22 % en uso) |
| GPU | RedHat Virtio 1.0 (no utilizada para cómputo) |
| Red | Tailscale, IP local 10.0.0.19/24 |

inserción, y no de variabilidad estocástica adicional. La Tabla 8.6 resume su definición.

Tabla 8.6: Escenarios de demanda evaluados en la campaña A/B de control semafórico

| **Escenario** | **Definición** |
|---|---|
| demanda_baja | 35 % de la flota vehicular base. |
| demanda_media | 65 % de la flota vehicular base. |
| demanda_alta | 100 % de la flota vehicular base. |
| demanda_saturada | 140 % de la flota vehicular base, mediante duplicación intercalada de vehículos. |
| hora_pico | Perfil de inserción triangular, con dos tercios de las salidas concentradas en el tercio central de la ventana de simulación. |
| corredor | Calle dominante con demanda duplicada respecto de la base y calles transversales reducidas al 20 %, dentro de una ventana de 1800 s. |

### 8.4.3. Planes fijos de referencia

Los tres planes fijos de referencia representan puntos distintos del espacio de diseño de temporización semafórica convencional, desde el reparto normativo igualitario hasta el residuo de un algoritmo de generación automática de planes. La Tabla 8.7 los resume.

Tabla 8.7: Planes fijos de referencia utilizados en la campaña A/B

| **Plan** | **Definición y origen** |
|---|---|
| MOPC 60 s | Ciclo de 60 s con 25 s de verde por fase, siguiendo el reparto igualitario recomendado por el Manual de Carreteras del Paraguay en ausencia de datos de volumen vehicular por aproximación (sección 113.14.3) [13]. El tiempo perdido por ciclo asociado a este reparto es del 16,7 %. |
| netconvert 90 s | Ciclo de 90 s con 42 s de verde, calculado como el residuo del algoritmo de generación automática de planes de netconvert: el ciclo por defecto de 90 s y el verde semilla de 31 s se ajustan mediante estiramiento igualitario, de modo que $31 + (90 - 68)/2 = 42$. El tiempo perdido por ciclo es del 6,7 %. |
| Ciclo largo 120 s | Ciclo de 120 s con 57 s de verde. El tiempo perdido por ciclo es del 5 %. |

### 8.4.4. Metodología de evaluación insesgada

La comparación directa de tiempos de viaje promedio a partir del registro de viajes completados que produce SUMO está expuesta a un sesgo de supervivencia: ese registro documenta únicamente los viajes que terminan dentro de la ventana de simulación, de modo que, cuando uno de los dos lados de la comparación colapsa parcialmente (los vehículos quedan atrapados en congestión y no llegan a destino), su promedio de tiempo de viaje se calcula solo sobre los vehículos rápidos que lograron escapar, produciendo una estimación artificialmente favorable para el lado colapsado. Para neutralizar este efecto se adoptó un veredicto compuesto por comparación. En primer lugar, se aplica una guardia de throughput con autoridad sobre las métricas por viaje: si el número de vehículos que completan su viaje difiere en más de un 10 % entre las corridas A y B, el veredicto se determina por throughput, independientemente de lo que indiquen los tiempos de viaje promedio. En segundo lugar, se incorpora el tiempo total de sistema a partir del resumen agregado que produce SUMO, que contabiliza a los vehículos aún circulando o atrapados al cierre de la ventana de simulación y por lo tanto no hereda el sesgo de supervivencia del registro por viaje. En tercer lugar, se construye evidencia pareada por vehículo: cada vehículo simulado tiene la misma hora de salida y la misma ruta en la corrida A y en la corrida B, de modo que su tiempo de viaje en ambas corridas puede compararse directamente como si fuera el mismo vehículo bajo dos políticas de control distintas, en lugar de comparar dos poblaciones de vehículos potencialmente distintas; esta evidencia pareada es la que alimenta el test de Wilcoxon de rangos con signo de la Sección 8.4.6, que ordena por magnitud las diferencias de tiempo de viaje de cada par de vehículos y contrasta si su mediana es distinta de cero, sin asumir que esas diferencias siguen una distribución normal. Se define además una banda de empate del 2 %, dentro de la cual una comparación se considera neutra en lugar de favorable o desfavorable, y se registra el número de vehículos nunca insertados en cada corrida (diferencia entre vehículos cargados y vehículos efectivamente insertados en la red), indicador adicional de saturación de la demanda de entrada. Este conteo loaded−inserted captura una población de vehículos estructuralmente invisible para el registro de viajes completados: un vehículo que nunca logra incorporarse a la red, por bloqueo sostenido en su punto de inserción, no genera ningún registro de viaje y por lo tanto no puede empeorar ni mejorar ningún promedio calculado sobre viajes completados; solo el tiempo total de sistema y este conteo explícito dejan constancia de su existencia.

Dos corridas observadas durante la campaña confirman la magnitud real del sesgo: en una, el tiempo de viaje promedio mejoró +18 % mientras el throughput caía −55 % por un colapso parcial autoinducido del sistema propuesto; en otra, en sentido inverso, el tiempo de viaje promedio empeoró −52 % mientras el throughput subía +116 %, porque en ese caso el colapso parcial ocurrió del lado del plan fijo de referencia. Sin la guardia de throughput, ambas corridas se habrían clasificado en el sentido opuesto al correcto.

### 8.4.5. Resultados globales

Sobre las 180 comparaciones A/B ejecutadas, el sistema propuesto resultó favorable en 140 (77,8 %), con un desglose por plan de referencia de 48/60 comparaciones favorables frente al plan MOPC 60 s, 46/60 frente a netconvert 90 s y 46/60 frente al ciclo largo 120 s. La Tabla 8.8 desagrega estos resultados por escenario y campaña, indicando en cada celda el número de corridas favorables sobre 10 y las medianas de variación porcentual de throughput ($\Delta$thr) y de tiempo total de sistema ($\Delta$sys). Es importante notar que estas medianas se calculan sobre el total de las 10 corridas de cada celda, incluyendo tanto las favorables como las desfavorables, de modo de no reintroducir el sesgo de selección que la metodología de la Sección 8.4.4 busca evitar.

Tabla 8.8: Corridas favorables (de 10) y medianas de variación porcentual de throughput y tiempo de sistema, por escenario y campaña de comparación

| | **MOPC 60 s** | | | **netconvert 90 s** | | | **Ciclo largo 120 s** | | |
|---|---|---|---|---|---|---|---|---|---|
| **Escenario** | fav. | $\Delta$thr (%) | $\Delta$sys (%) | fav. | $\Delta$thr (%) | $\Delta$sys (%) | fav. | $\Delta$thr (%) | $\Delta$sys (%) |
| corredor | 10/10 | +207,5 % | +57,3 % | 10/10 | +159,6 % | +57,7 % | 10/10 | +22,6 % | +28,6 % |
| demanda_baja | 9/10 | +5,5 % | +42,8 % | 8/10 | +10,9 % | +35,9 % | 8/10 | +14,8 % | +39,2 % |
| demanda_media | 9/10 | +51,8 % | +20,8 % | 9/10 | +25,5 % | +9,1 % | 9/10 | +48,6 % | +9,0 % |
| demanda_alta | 5/10 | +9,0 % | +1,9 % | 5/10 | +7,6 % | -7,2 % | 5/10 | +15,4 % | -8,8 % |
| demanda_saturada | 8/10 | +46,3 % | +6,0 % | 8/10 | +19,6 % | -5,2 % | 7/10 | +25,2 % | -7,3 % |
| hora_pico | 7/10 | +22,7 % | +3,0 % | 6/10 | +15,3 % | -1,3 % | 7/10 | +23,8 % | -4,7 % |

Como complemento de la Tabla 8.8, que reporta la fracción de corridas favorables por celda, la Tabla 8.9 reporta la mediana, por celda, del porcentaje de vehículos pareados que registraron una mejora individual de tiempo de viaje respecto del total de vehículos pareados de esa corrida (evidencia pareada por vehículo, Sección 8.4.4). Esta cifra es independiente del veredicto agregado de la celda: una celda puede tener una mayoría de corridas favorables y, sin embargo, un porcentaje de vehículos pareados mejorados moderado, si la mejora se concentra en una fracción de los vehículos y no se reparte de forma pareja.

Tabla 8.9: Mediana, por escenario y campaña, del porcentaje de vehículos pareados con mejora individual de tiempo de viaje

| **Escenario** | **MOPC 60 s** | **netconvert 90 s** | **Ciclo largo 120 s** |
|---|---|---|---|
| corredor | 51,7 % | 59,8 % | 69,7 % |
| demanda_baja | 68,3 % | 68,9 % | 70,3 % |
| demanda_media | 47,0 % | 47,9 % | 48,5 % |
| demanda_alta | 34,2 % | 32,8 % | 38,5 % |
| demanda_saturada | 23,7 % | 34,6 % | 32,5 % |
| hora_pico | 40,8 % | 39,9 % | 40,7 % |

Se observa que demanda_saturada, pese a tener una fracción alta de corridas favorables (7 a 8 de 10, Tabla 8.8), presenta la mediana de vehículos pareados mejorados más baja de todos los escenarios (23,7 % a 34,6 %): la ventaja agregada de esta celda proviene en buena medida de

un throughput global mayor y no de una mejora homogénea sobre la mayoría de los vehículos individuales, señal consistente con un régimen de saturación en el que el sistema propuesto evita el colapso pero no elimina la congestión residual.

La Figura 8.4 presenta gráficamente el conteo de corridas favorables por escenario y campaña, mientras que las Figuras 8.5 y 8.6 muestran las medianas de variación porcentual de throughput y de tiempo de sistema, respectivamente.

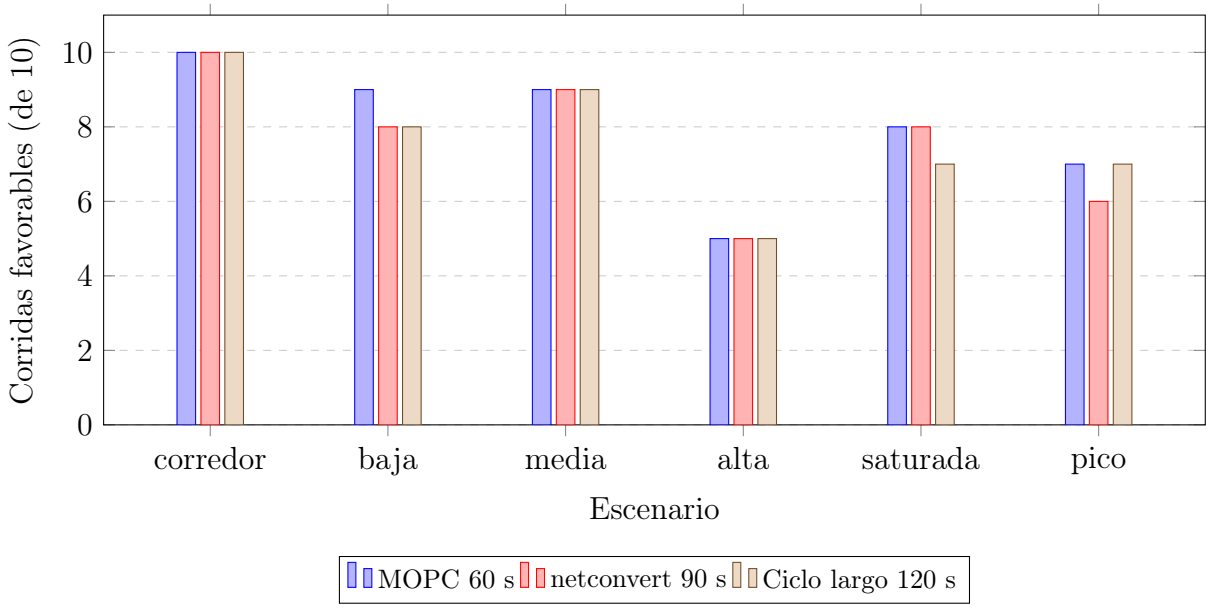



Figura 8.4: Corridas favorables al sistema propuesto, de 10 semillas, por escenario y campaña.

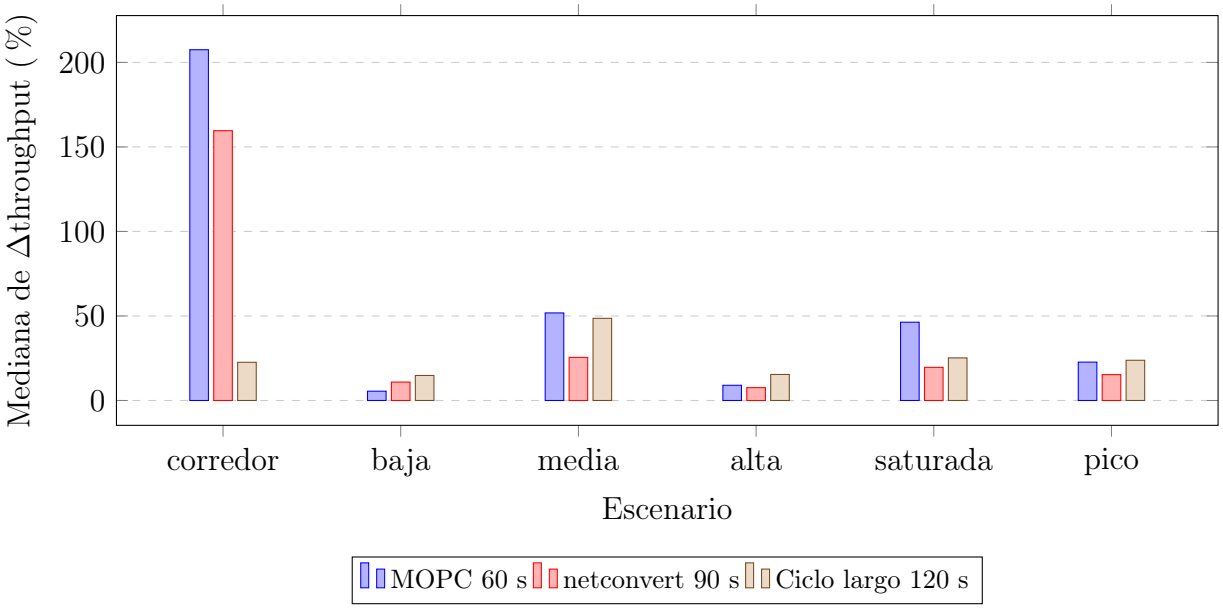



Figura 8.5: Mediana de la variación porcentual de throughput del sistema propuesto respecto de cada plan fijo, por escenario.

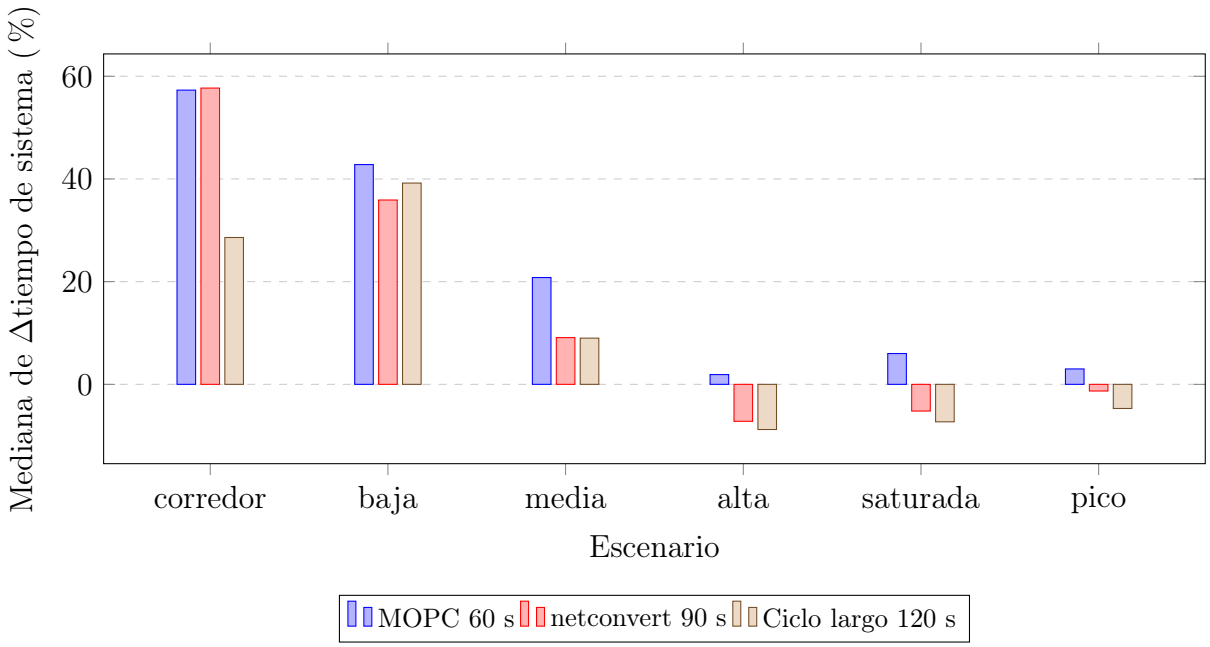



Figura 8.6: Mediana de la variación porcentual del tiempo total de sistema del sistema propuesto respecto de cada plan fijo, por escenario.

### 8.4.6. Rigor estadístico

Las medianas de la Tabla 8.8 resumen la magnitud del efecto, pero no informan por sí solas si ese efecto es estadísticamente distinguible del ruido introducido por la semilla aleatoria. Para cada una de las 180 corridas se aplicó por lo tanto un conjunto de pruebas de significancia sobre los tiempos de viaje pareados por vehículo. El test de Wilcoxon de rangos con signo, aplicado sobre las diferencias pareadas vehículo a vehículo (mismo vehículo, misma ruta, mismo instante de salida en ambas corridas), constituye la evidencia más sólida de ese conjunto, porque el tamaño de muestra $n$ resulta idéntico por construcción entre las condiciones A y B: al no depender de comparar dos muestras de tamaño distinto, no hereda el riesgo de que una población deprimida por colapso parcial sesgue el estadístico. El test de permutación, que reconstruye la distribución nula del estadístico de interés reordenando al azar las etiquetas de condición, aporta una verificación no paramétrica independiente del supuesto de normalidad. El test U de Mann-Whitney ordena en conjunto todos los tiempos de viaje de ambas corridas (sin usar el emparejamiento) y contrasta si los valores de una tienden a ser sistemáticamente mayores que los de la otra, evaluando así la separación estocástica entre las dos distribuciones de tiempos de viaje; sirve de contraste frente al resultado pareado. El estadístico $d$ de Cohen cuantifica la magnitud del efecto en unidades de desviación estándar agrupada, independientemente de si alcanza significancia estadística, lo que permite distinguir un efecto pequeño pero consistente de un efecto grande pero ruidoso. Finalmente, el intervalo de confianza (IC) bootstrap al 95 %, obtenido remuestreando con reemplazo la distribución de diferencias pareadas un número elevado de veces, acota la incertidumbre del efecto medido sin depender de supuestos distribucionales.

La Tabla 8.10 resume, por campaña, cuántas de las 60 corridas alcanzaron significancia al

5 % y al 1 % en el test de Wilcoxon pareado, el valor medio del tamaño de efecto $|d|$ de Cohen y la mediana del porcentaje de vehículos pareados con mejora individual.

Tabla 8.10: Rigor estadístico por campaña: corridas significativas (de 60), tamaño de efecto medio y mediana del porcentaje de vehículos pareados mejorados

| **Campaña** | $p < 0,05$ **(de 60)** | $p < 0,01$ **(de 60)** | $\|d\|$ **de Cohen medio** | **% pareados mejorados (mediana)** |
|---|---|---|---|---|
| MOPC 60 s | 53 | 51 | 0,31 | 43,7 % |
| netconvert 90 s | 58 | 56 | 0,34 | 42,3 % |
| Ciclo largo 120 s | 59 | 58 | 0,36 | 42,9 % |

Entre 53 y 59 de las 60 comparaciones de cada campaña, es decir entre el 88 % y el 98 %, alcanzan significancia al 5 % en el test pareado, y la gran mayoría de ellas la mantienen al 1 %, lo que descarta que los resultados de la Sección anterior sean producto del azar de la selección de semillas. El tamaño de efecto de Cohen medio, entre 0,31 y 0,36, corresponde en la convención habitual a un efecto de magnitud pequeña a moderada: es consistente con que la mediana de vehículos pareados mejorados se ubique alrededor del 42 % al 44 % y no cerca del 100 %, es decir, el sistema propuesto no mejora a todos los vehículos por igual, sino que desplaza la distribución completa de tiempos de viaje lo suficiente como para que la mayoría de las corridas resulten significativas pese a un efecto por vehículo moderado.

La Figura 8.7 contrasta, para las 180 comparaciones, la variación porcentual del throughput frente a la variación porcentual del tiempo de sistema, coloreando cada punto según si la corrida fue clasificada como favorable.

### 8.4.7. Distribución completa por semilla

Las medianas reportadas hasta aquí resumen diez corridas por celda en un único número, lo que oculta la dispersión introducida por la semilla aleatoria. Para exponerla completamente, las Figuras 8.8 y 8.9 presentan diagramas de caja de la variación porcentual de throughput y de tiempo de sistema, respectivamente, para las diez semillas de cada uno de los seis escenarios de la campaña MOPC 60 s. En cada caja, la línea central marca la mediana, los bordes de la caja el primer y tercer cuartil (rango intercuartílico, RIC) de las diez observaciones, y los bigotes se extienden hasta el valor más extremo dentro de 1,5 RIC desde cada borde de la caja; los cuartiles se calcularon en Python mediante interpolación lineal sobre las diez observaciones ordenadas de cada escenario.

### 8.4.8. Grillas de veredictos

La Figura 8.10 representa, para cada una de las tres campañas, la matriz completa de veredictos de las 60 comparaciones A/B correspondientes: cada celda de cada matriz corresponde a una combinación única de escenario (fila) y semilla (columna), es decir, a una comparación A/B completa, coloreada en verde si resultó favorable al sistema propuesto y en rojo en caso

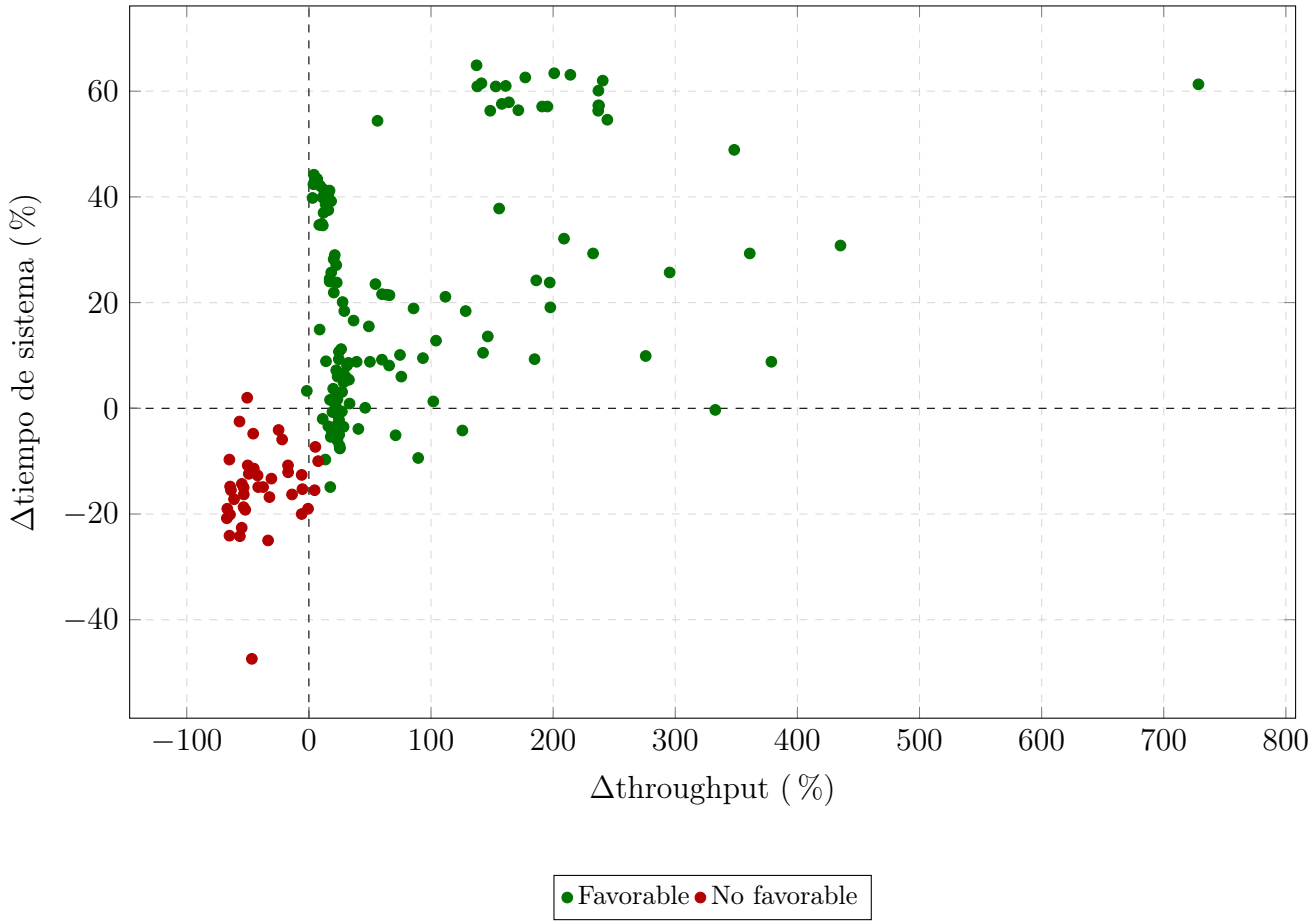



Figura 8.7: Dispersión de las 180 comparaciones A/B: variación porcentual de throughput (eje horizontal) contra variación porcentual de tiempo de sistema (eje vertical), coloreada según el veredicto de la corrida. Las líneas discontinuas marcan el cero de cada eje.

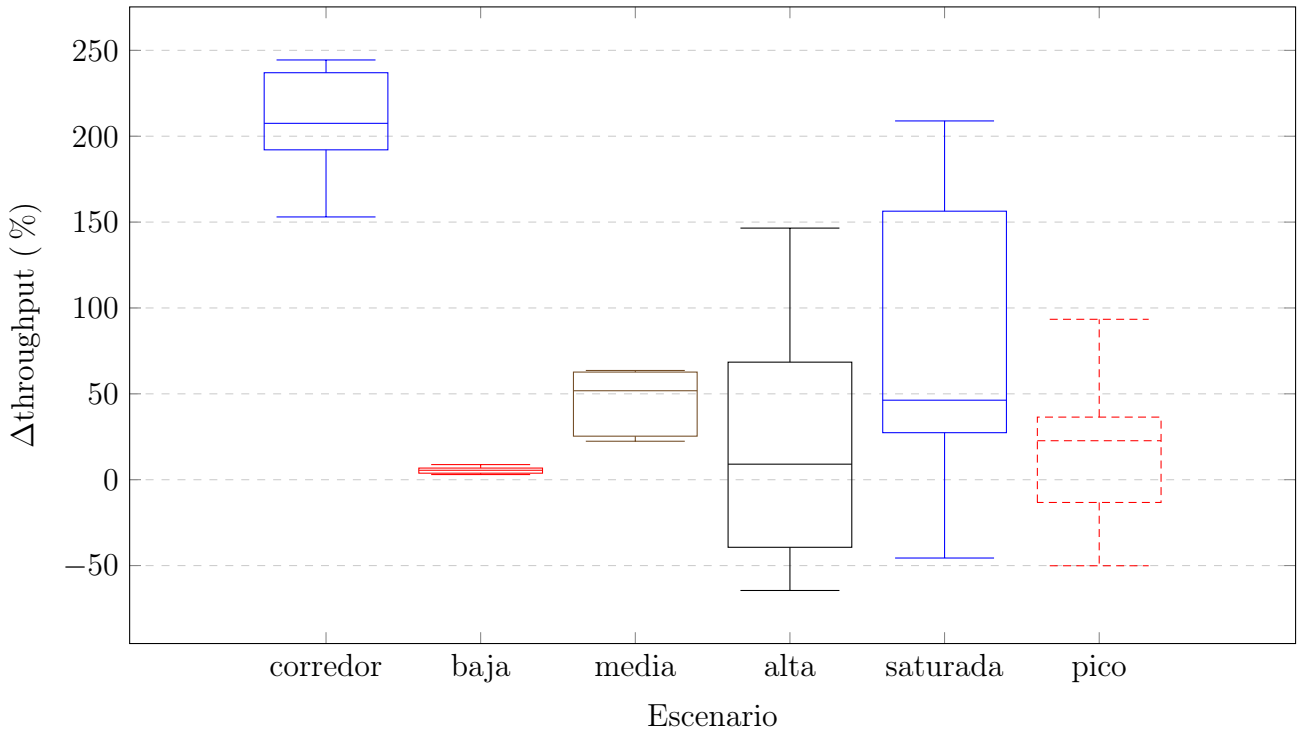



Figura 8.8: Distribución completa, por escenario, de la variación porcentual de throughput sobre las diez semillas de la campaña MOPC 60 s.

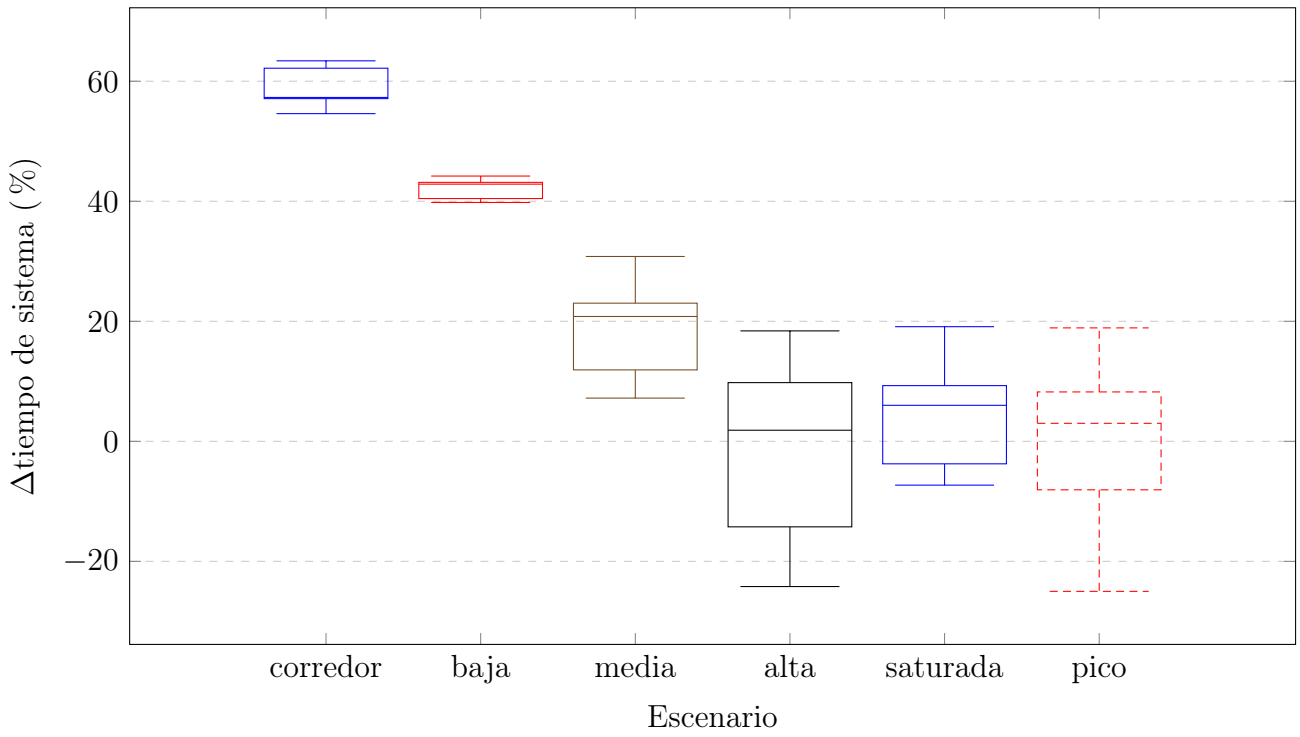



Figura 8.9: Distribución completa, por escenario, de la variación porcentual del tiempo total de sistema sobre las diez semillas de la campaña MOPC 60 s, con la misma convención de caja y bigotes que la Figura 8.8.

contrario. Las diez semillas, generadas mediante un generador criptográficamente seguro y registradas para reproducibilidad exacta, son 157300, 214181, 281772, 372292, 400290, 423459, 449463, 843099, 950893 y 974616.

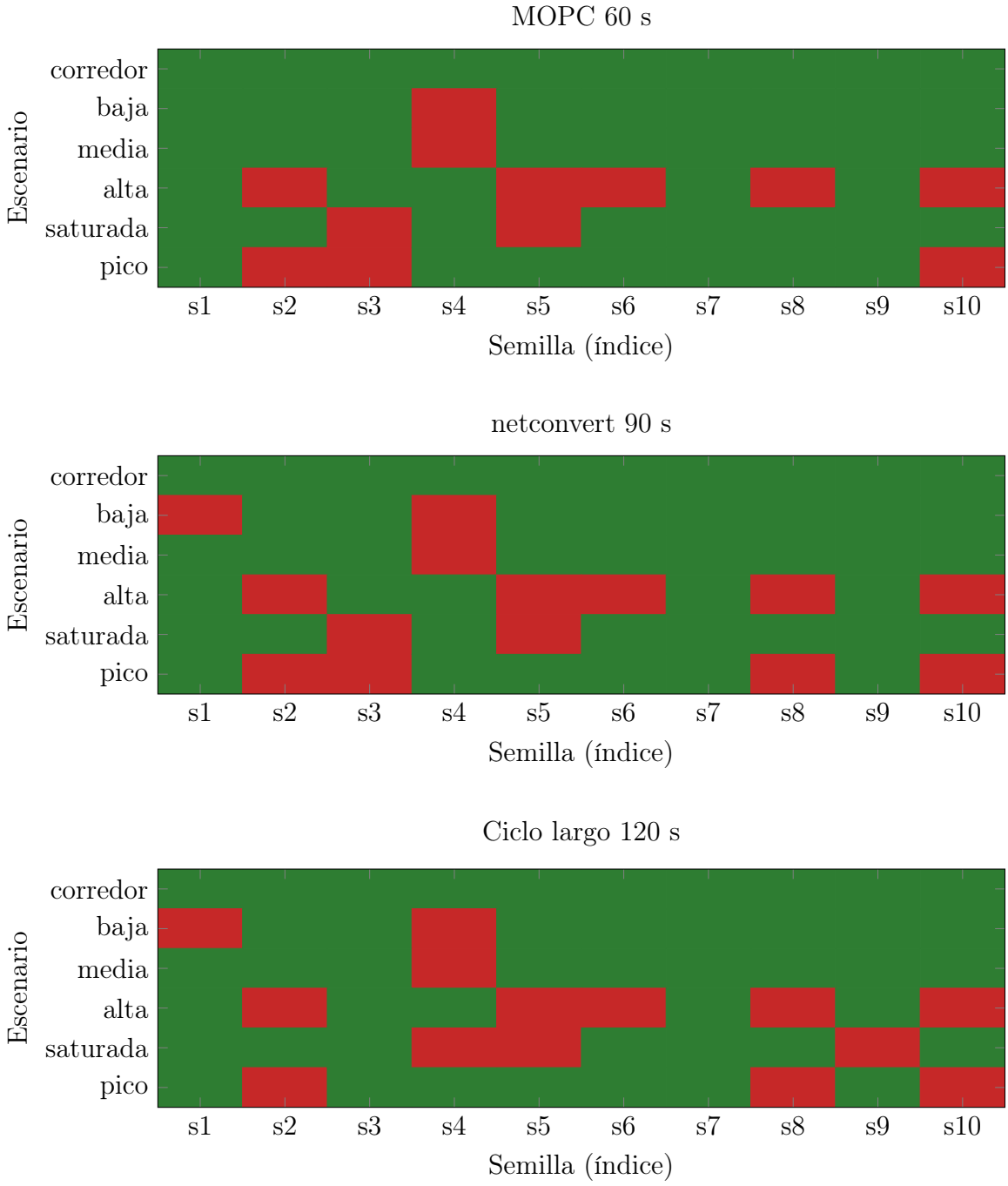



Figura 8.10: Matrices de veredicto de las 180 comparaciones A/B, una por campaña de plan fijo de referencia.

### 8.4.9. Análisis de patrones

Del conjunto de resultados se desprenden cuatro patrones consistentes.

**Ventaja creciente con la asimetría de demanda.** La ventaja del sistema propuesto cre-

ce con la asimetría de la demanda entre aproximaciones: el escenario corredor, en el que la calle dominante concentra el doble de demanda que las transversales, es el único en resultar favorable en las 10 corridas de las tres campañas simultáneamente, con una mediana de $\Delta$throughput de +207,5 % frente al plan MOPC 60 s y de +22,6 % frente al ciclo largo 120 s. Esta magnitud refleja que un reparto de verde igualitario resulta especialmente ineficiente cuando la demanda real está lejos de ser simétrica, condición que el control adaptativo del sistema propuesto explota de forma directa.

**Tipo de ventaja según el plan de referencia.** El tipo de ventaja que obtiene el sistema propuesto depende del ciclo del plan fijo contra el que se compara. Frente al plan MOPC de 60 s, cuyo ciclo corto y alto tiempo perdido relativo lo hacen más vulnerable a saturarse, la ventaja proviene principalmente de evitar el colapso: el sistema propuesto sostiene mayor throughput y menor tiempo de sistema incluso en escenarios de demanda alta y saturada. En cambio, frente a los planes de 90 s y 120 s, cuyo tiempo perdido por ciclo es menor y cuya capacidad de servicio está mejor dimensionada, la ventaja del sistema propuesto se concentra en los regímenes de demanda baja y media y en el escenario de asimetría marcada (corredor), mientras que en demanda alta y saturada las medianas de $\Delta$sys se vuelven negativas (-7,2 % y -5,2 % frente a netconvert 90 s; -8,8 % y -7,3 % frente al ciclo largo 120 s), señal de que el margen de mejora disponible se reduce cuando el plan de referencia ya opera cerca de su capacidad.

**Límite del sistema: colapsos en demanda alta.** El escenario demanda_alta es el peor evaluado en las tres campañas, con solo 5 de 10 corridas favorables frente a cada uno de los tres planes fijos. La inspección de las corridas desfavorables de esta celda revela colapsos parciales autoinducidos, en los que el propio control adaptativo del sistema, al reaccionar localmente a la congestión mediante Max-Pressure, favorece temporalmente ciertos movimientos y produce bloqueos en cascada (spillback) en intersecciones vecinas. Estos colapsos se concentran de forma recurrente en un subconjunto reducido de semillas, lo que sugiere una sensibilidad estructural del sistema a configuraciones iniciales de demanda cercanas al punto de saturación de la red, más que un fallo aleatorio y disperso.

**Ninguna semilla es uniformemente desfavorable.** En las matrices de veredicto de la Figura 8.10, el patrón dominante es horizontal: filas completas o casi completas de un mismo color (corredor en verde en las tres matrices; demanda_alta con una mezcla persistente de rojo en las tres matrices) muestran que el escenario de demanda determina el resultado con mucha mayor fuerza que la semilla concreta, mientras que dentro de una misma fila el color rara vez cambia de forma sistemática por columna: no existe una semilla que sea uniformemente desfavorable a través de todos los escenarios.

## 8.5. Discusión

Los resultados obtenidos permiten evaluar el sistema de forma integral, identificando sus fortalezas, sus limitaciones actuales y las líneas de trabajo que podrían potenciar su desempeño en escenarios de despliegue real.

### 8.5.1. Viabilidad del Sistema

Los tres frentes evaluados confirman la viabilidad del sistema para entornos con presupuesto limitado, como ciudades intermedias de la región: traffic-sync escala linealmente dentro de la memoria disponible en hardware embebido de bajo costo, y traffic-storage ofrece latencias de subida y descarga compatibles con su modelo de reporte periódico sobre datos agregados, no de control en tiempo estricto. El cuello de botella identificado es el tiempo de confirmación en la red BlockDAG, mitigable mediante la gestión de nonces y la precarga del estado de gas sin modificar la infraestructura de red. El Capítulo 9 retoma estas cifras junto con las de la campaña A/B en la síntesis final del trabajo.

### 8.5.2. Desafíos Identificados y Limitaciones

Durante la ejecución de las pruebas se identificaron dos desafíos técnicos relevantes para un despliegue de producción: la dependencia de Pinata como nodo IPFS gestionado por terceros, y la variabilidad en los tiempos de minado de la red BlockDAG pública de pruebas. El Capítulo 9 desarrolla estas limitaciones junto con las del control semafórico y las líneas de trabajo futuro que se desprenden de ellas.

### 8.5.3. Hallazgos del control semafórico

La campaña A/B de la Sección 8.4 mide el efecto del sistema sobre la circulación vehicular bajo una metodología diseñada para no premiar al sistema propuesto por artefactos de medición: sin la guardia de throughput, el sesgo de supervivencia habría inflado la tasa de éxito reportada, en particular en los escenarios de demanda alta y saturada, donde ambos lados de la comparación operan cerca del límite de capacidad de la red. El límite observado en demanda alta simétrica (5 de 10 corridas favorables) no es una falla del control adaptativo sino el reflejo de que, cuando no hay asimetría de demanda que explotar, un plan fijo bien dimensionado se acerca a una asignación de verde casi óptima y el costo de conmutación de Max-Pressure erosiona parte del margen disponible. El Capítulo 9 retoma este límite, junto con los colapsos parciales autoinducidos identificados en esa misma celda (Sección 8.4.9), en la discusión de limitaciones y trabajo futuro.

Las evaluaciones cubren los tres frentes del sistema: traffic-sync demostró escalabilidad lineal ($\approx$0,034 s/sensor) y operación estable en menos de 220 MB de RAM; traffic-storage alcanzó confirmación BlockDAG en $\approx$3,5 s a un costo inferior a $\$2\times10^{-5}$ USD por transacción de datos, superando a Ethereum en latencia de confirmación (77–94 %) y costo operativo ($>$4 órdenes de magnitud); y la campaña A/B de control semafórico resultó favorable al sistema propuesto en el 77,8 % de las 180 comparaciones, con una ventaja que crece con la asimetría de demanda entre aproximaciones y un límite claro en demanda simétrica cercana a la capacidad de la red. Estos resultados establecen la base cuantitativa para las conclusiones del capítulo siguiente.

# CAPÍTULO 9
# CONCLUSIÓN

El presente trabajo de tesis abordó el diseño, la implementación y la evaluación experimental de un sistema modular y descentralizado para la optimización del tráfico urbano y la persistencia verificable de datos asociados. La arquitectura propuesta integra simulación de tráfico mediante SUMO, clasificación de congestión con inferencia difusa tipo Mamdani, ajuste adaptativo de temporización semafórica mediante optimización por enjambre de partículas (PSO) y almacenamiento distribuido basado en IPFS y contratos inteligentes sobre una red BlockDAG. A diferencia de plataformas centralizadas convencionales, el sistema expone sus capacidades mediante APIs REST y garantiza trazabilidad e integridad de los registros sin depender de un operador único de confianza.

Los capítulos anteriores desarrollaron los fundamentos teóricos, el diseño de cada módulo y su interacción dentro del flujo completo: desde la generación de escenarios controlados y la detección de cuellos de botella en traffic-sim, pasando por el análisis y la optimización en traffic-sync, hasta la publicación auditable de resultados en traffic-storage. El capítulo de evaluación experimental consolidó estas piezas con mediciones reproducibles en hardware de consumo, demostrando que el sistema es viable para entornos municipales con recursos limitados.

## 9.1. Resultados principales

La evaluación del módulo traffic-sync confirmó una escalabilidad aproximadamente lineal del tiempo de procesamiento conforme aumenta el número de sensores simulados: para 2000 sensores, el ciclo completo de análisis y optimización se completó en 67,6 s ($\approx$ 0,034 s por sensor). El consumo de memoria RAM se mantuvo estable entre 198 y 219 MB en todos los escenarios, y el uso de CPU disminuyó del 72,3 % al 50,3 % a medida que creció la concurrencia, comportamiento coherente con el procesamiento asíncrono de FastAPI y con el bajo costo

computacional unitario de los componentes difuso y PSO. Estos resultados respaldan la viabilidad de despliegues sobre hardware embebido de bajo costo, como unidades Raspberry Pi en campo.

En traffic-storage, las transacciones de almacenamiento alcanzaron confirmación en la red BlockDAG en ≈3,5 s, con costos inferiores a $\$2\times10^{-5}$ USD por transacción. Los tiempos de subida extremo a extremo se situaron entre 6,7 y 7,3 s según la densidad vehicular del escenario, mientras que la descarga se completó en ≈1,8 s, adecuado para flujos de auditoría y consulta sobre datos agregados de ciclos completados, no sobre control en tiempo estricto. La verificación de integridad mediante el CID de IPFS demostró que los contenidos recuperados conservan fielmente el original. Frente a Ethereum, el sistema redujo entre 77 y 94 % la latencia de confirmación y en más de cuatro órdenes de magnitud el costo operativo. Frente a arquitecturas IoT-cloud centralizadas, la ventaja no es de latencia sino de garantías: inmutabilidad asegurada por consenso distribuido y no por confianza en un operador central; la comparación experimental directa de latencia con esas plataformas queda como trabajo futuro.

En conjunto, los experimentos establecen que la combinación IPFS+BlockDAG constituye una alternativa técnicamente competitiva para monitoreo urbano de alta frecuencia con requisitos de trazabilidad auditables, mientras que el núcleo analítico escala de forma predecible hacia escenarios de optimización por zonas urbanas.

El hallazgo más destacado de esta tesis es la campaña de evaluación A/B del control semafórico jerárquico propuesto, PSO de ciclo por clúster combinado con Max-Pressure local, frente a tres planes fijos de referencia (MOPC 60 s, netconvert 90 s y ciclo largo 120 s), sobre seis escenarios de demanda y diez semillas aleatorias en la red vial de San Lorenzo. De las 180 comparaciones pareadas ejecutadas bajo una metodología insesgada frente al sesgo de supervivencia, el sistema propuesto resultó favorable en 140 (77,8 %), con un desglose estable entre los tres planes de referencia (48/60, 46/60 y 46/60 respectivamente), lo que indica que la ventaja no depende de la elección particular del plan fijo con el que se compare. La dominancia del sistema fue máxima en condiciones de demanda asimétrica: en el escenario corredor, donde una calle concentra el doble de la demanda base y las transversales se reducen al 20 %, el control propuesto resultó favorable en las 10 de 10 corridas frente a las tres campañas de comparación. Este resultado se complementa con un límite honesto del enfoque: en demanda alta simétrica, donde ambas aproximaciones a la intersección compiten en condiciones similares y cercanas a la capacidad de saturación, el sistema propuesto solo resultó favorable en 5 de 10 corridas frente a cada plan de referencia, reflejo de que un plan fijo bien dimensionado se aproxima a una asignación de verde casi óptima cuando no existe asimetría de demanda que explotar, y de que el propio costo de conmutación de Max-Pressure erosiona parte del margen disponible en ese régimen.

## 9.2. Divulgación científica

Los hallazgos centrales de esta investigación fueron sintetizados y publicados en el artículo de conferencia internacional titulado *Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments*, presentado en la *2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies* (CHILECON), celebrada en Valparaíso, Chile, en octubre de 2025.[1] El artículo documenta, en formato condensado, la arquitectura modular del sistema, las métricas de escalabilidad y almacenamiento aquí ampliadas, y la comparación con plataformas basadas en Ethereum y soluciones centralizadas. La publicación valida externamente la arquitectura y la metodología de la tesis y posiciona la propuesta en el ámbito de sistemas inteligentes para ciudades con almacenamiento verificable y despliegue en dispositivos de bajo costo.

## 9.3. Limitaciones

Pese a los resultados favorables, deben reconocerse limitaciones relevantes para un despliegue productivo. En primer lugar, las evaluaciones se realizaron principalmente en entorno simulado y sobre hardware de laboratorio; la transferencia a escenarios urbanos reales implica heterogeneidad de sensores, variabilidad en las comunicaciones y resistencia institucional a soluciones descentralizadas, aspectos aún no validados en pilotos de campo.

En segundo lugar, de las 180 comparaciones A/B de la campaña de evaluación del control semafórico, 40 (22,2 %) resultaron desfavorables al sistema propuesto, y esos casos se concentran de forma marcada en el escenario de demanda alta simétrica, donde la ventaja del control adaptativo se diluye frente a planes fijos bien dimensionados. Adicionalmente, se identificaron colapsos parciales autoinducidos en un subconjunto recurrente de semillas de ese mismo escenario: la naturaleza reactiva y local de Max-Pressure, sin coordinación explícita entre intersecciones vecinas, puede producir bloqueos en cascada (spillback) que el supervisor de congelamiento mitiga pero no elimina por completo dentro de la ventana de simulación, una fragilidad residual frente al gridlock que constituye una limitación conocida del enfoque.

En tercer lugar, traffic-storage depende de un nodo IPFS gestionado por terceros (Pinata), introduce variabilidad en los tiempos de minado de la red pública de pruebas BlockDAG y concentra el cuello de botella en la confirmación on-chain (≈3,5 s dentro del pipeline de subida). No se implementaron aún mecanismos activos de tolerancia a fallos ni replicación automática de contenido.

Finalmente, la evaluación del control semafórico se realizó exclusivamente en simulación (SUMO) sobre una única red vial urbana, la del microcentro de San Lorenzo, por lo que la generalización a otras topologías de red y a condiciones de tráfico de campo permanece por validar.

[1] Disponible en IEEE Xplore: https://ieeexplore.ieee.org/document/11476672.

## 9.4. Trabajo futuro

Como continuación natural de este trabajo, se identifican las siguientes líneas de investigación y desarrollo:

- Optimizar los desplazamientos (offsets) entre intersecciones coordinadas para generar ondas verdes a lo largo de corredores viales, complementando el ciclo por clúster con coordinación temporal explícita entre semáforos vecinos.
- Diseñar predictores más tempranos de formación de gridlock que refuercen al supervisor de congelamiento, anticipando los colapsos autoinducidos observados en demanda alta simétrica antes de que la red alcance el punto de bloqueo en cascada.
- Integrar fuentes de datos reales provenientes de dispositivos de conteo, sensores IoT con conectividad LoRaWAN y sistemas de visión por computadora.
- Desplegar nodos IPFS propios y evaluar redes BlockDAG privadas o permisionadas para despliegues municipales, reduciendo la variabilidad de confirmación y la dependencia de proveedores externos.
- Optimizar la gestión de nonces y la precarga del estado de gas para acortar la fase de confirmación en el pipeline de almacenamiento.
- Diseñar esquemas de comunicación asíncrona mediante instancias intermedias que alimenten en tiempo real los módulos del sistema desde el campo.
- Ejecutar pilotos en ciudades intermedias de la región para medir resiliencia, calidad de servicio e impacto en la fluidez del tráfico frente a plataformas centralizadas, y validar con datos de campo los resultados obtenidos en simulación.
- Comparar el control jerárquico propuesto con enfoques de aprendizaje por refuerzo para la asignación de fases y ciclo, como referencia adicional de desempeño.
- Explorar la integración con marcos de *digital twins* urbanos para planificación y simulación avanzada.

## 9.5. Síntesis final

El sistema desarrollado en esta tesis ofrece una base tecnológica sólida que responde a criterios de trazabilidad, seguridad, integridad y eficiencia operativa en la gestión del tráfico urbano. Dos pilares sostienen esta afirmación: por un lado, la trazabilidad verificable del almacenamiento distribuido, con confirmación en segundos y costo insignificante frente a Ethereum; por otro, la evidencia experimental de que el control semafórico jerárquico, PSO de ciclo por clúster combinado con Max-Pressure local, mejora la circulación vehicular frente a planes fijos

de referencia en la mayoría de las condiciones de demanda evaluadas (140 de 180 comparaciones, 77,8 %), con una ventaja particularmente marcada ante demanda asimétrica y un límite reconocido en demanda alta simétrica. Este enfoque modular y descentralizado, respaldado por evidencia cuantitativa y por la publicación derivada en CHILECON 2025, lo posiciona como una propuesta viable para municipios con presupuesto limitado. Las mejoras futuras deberán cerrar la brecha entre los resultados de laboratorio y la evidencia práctica en entornos urbanos reales, fortaleciendo la coordinación entre intersecciones y la robustez del control local frente al gridlock.

# REFERENCIAS


[1] Markos Papageorgiou, Christina Diakaki, Vaya Dinopoulou, Apostolos Kotsialos y Yibing Wang. «Review of Road Traffic Control Strategies». En: *Proceedings of the IEEE* 91.12 (2003), págs. 2043-2067. DOI: 10.1109/JPROC.2003.819610.

[2] Nicholas J. Garber y Lester A. Hoel. *Traffic and Highway Engineering*. 6th. Boston: Cengage Learning, 2019.

[3] Aleksandar Stevanovic, Jelena Stevanovic, Kai Zhang y Stuart Batterman. «Optimizing Traffic Control to Reduce Fuel Consumption and Vehicular Emissions: Integrated Approach with VISSIM, CMEM, and VISGAOST». En: *Transportation Research Record* 2128 (2010). Incluye comparación de desempeño entre planes fijos y controladores adaptativos, págs. 105-113. DOI: 10.3141/2128-12.

[4] Li Li, Yisheng Lv y Fei-Yue Wang. «Traffic Signal Timing via Deep Reinforcement Learning». En: *IEEE/CAA Journal of Automatica Sinica* 3.3 (2016). Deep Q-learning para control semafórico adaptativo, págs. 247-254. DOI: 10.1109/JAS.2016.7508798.

[5] Michael Behrisch, Laura Bieker, Jakob Erdmann y Daniel Krajzewicz. «SUMO – Simulation of Urban MObility: An Overview». En: *Proceedings of the Third International Conference on Advances in System Simulation (SIMUL 2011)*. Barcelona, Spain: IARIA, 2011, págs. 63-68.

[6] Lotfi A. Zadeh. «Fuzzy Sets». En: *Information and Control* 8.3 (1965), págs. 338-353. DOI: 10.1016/S0019-9958(65)90241-X.

[7] James Kennedy y Russell Eberhart. «Particle Swarm Optimization». En: *Proceedings of the IEEE International Conference on Neural Networks*. Vol. 4. IEEE, 1995, págs. 1942-1948. DOI: 10.1109/ICNN.1995.488968.

[8] Juan Benet. *IPFS - Content Addressed, Versioned, P2P File System*. Whitepaper fundacional de IPFS. 2014. arXiv: 1407.3561 [cs.DC]. URL: https://arxiv.org/abs/1407.3561.

[9] Yonatan Sompolinsky y Aviv Zohar. «PHANTOM and GHOSTDAG: A Scalable Generalization of Nakamoto Consensus». En: *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '21)*. New York, NY, USA: ACM, 2021, págs. 57-70. DOI: 10.1145/3479722.3480990.

[10] Kevin Mathias Galeano Saldivar, María José Duarte Kowalewski, Marcos D. Villagra y Derlis O. Gregor. «Scalable and Decentralized Urban Traffic Optimization with Verifiable Storage for Smart City Deployments». En: *2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON)*. Valparaíso, Chile: IEEE, 2025. DOI: 10.1109/CHILECON56137.2025.11476672.

[11] Roger P. Roess, Elena S. Prassas y William R. McShane. *Traffic Engineering*. 5th. Upper Saddle River: Pearson, 2019.

[12] Federal Highway Administration. *Traffic Signal Timing Manual*. https://ops.fhwa.dot.gov/publications/fhwahop08024/. FHWA-HOP-08-024, consultado: 19 de julio de 2026. 2008.

[13] Ministerio de Obras Públicas y Comunicaciones. *Manual de Carreteras del Paraguay. Unidad 3: Diseño de Carreteras. Volumen 3.3: Señalización y Obras Complementarias*. Sección 3.3.2.7: Semáforos. Asunción, Paraguay: MOPC, 2019.

[14] Nathan H. Gartner, John D. C. Little y Henry Gabbay. «Optimization of Traffic Signal Settings by Mixed-Integer Linear Programming: Part I: The Network Coordination Problem». En: *Transportation Science* 9.4 (1975). Formulación MILP para optimización de redes semafóricas, págs. 321-343. DOI: 10.1287/trsc.9.4.321.

[15] S. C. Wong, H. Yang y H. K. Lo. «Traffic Signal Optimisation: Modeling and Computation». En: *Transportation Research Part B: Methodological* 32.3 (1998). Formulación matemática del problema de optimización de señales, págs. 157-172. DOI: 10.1016/S0191-2615(97)00028-X.

[16] J. A. Hillier y R. Rothery. «The Synchronization of Traffic Signals for Minimum Delay». En: *Transportation Science* 1.2 (1967). Clásico sobre formulación de optimización de señales, págs. 81-94. DOI: 10.1287/trsc.1.2.81.

[17] F. V. Webster. «Traffic Signal Settings». En: *Road Research Technical Paper* 39 (1958). Clásico método de diseño de tiempos semafóricos.

[18] Byungkyu Park, Carroll J. Messer y Thomas Urbanik. «Traffic Signal Optimization Program for Oversaturated Conditions: Genetic Algorithm Approach». En: *Transportation Research Record* 1683 (1999). Uso de algoritmos genéticos para condiciones sobresaturadas, págs. 133-142. DOI: 10.3141/1683-17.

[19] Halim Ceylan y Michael G. H. Bell. «Traffic Signal Timing Optimization Based on Genetic Algorithm Approach, Including Drivers' Routing». En: *Transportation Research Part B: Methodological* 38.4 (2004). Algoritmos genéticos para optimización semafórica, págs. 329-342. DOI: 10.1016/S0191-2615(03)00015-8.

[20] Nathan H. Gartner, Farhad J. Pooran y Carleton M. Andrews. *Optimized Policies for Adaptive Control: A Case Study for Arterial Traffic*. Capítulos sobre control actuado y coordinado. Boston: Springer, 2001.

[21] P. B. Hunt, D. I. Robertson, R. D. Bretherton y R. I. Winton. «SCOOT—A Traffic Responsive Method of Coordinating Signals». En: *Transport and Road Research Laboratory Report LR 1014* (1982). Sistema de control adaptativo SCOOT.

[22] A. G. Sims y K. W. Dobinson. «The Sydney Coordinated Adaptive Traffic (SCATS) System: Philosophy and Benefits». En: *IEEE Transactions on Vehicular Technology* 29.2 (1980). Sistema SCATS de control adaptativo, págs. 130-137. DOI: 10.1109/T-VT.1980.23833.

[23] X. Li et al. «A PSO Based Signal Timing Optimization Approach of Phase Combination». En: *Proceedings of the International Conference on Computer Technology and Engineering*. Aplicación de PSO a optimización de tiempos semafóricos. Atlantis Press, 2016, págs. 116-120.

[24] R. Gonçalo et al. «On Tuning the Particle Swarm Optimization for Solving the Traffic Light Time Problem». En: *ICCSA 2022 – Lecture Notes in Computer Science*. PSO integrado con SUMO para optimización de semáforos. Springer, 2022.

[25] Ebrahim H. Mamdani y Sedrak Assilian. «An Experiment in Linguistic Synthesis with a Fuzzy Logic Controller». En: *International Journal of Man-Machine Studies* 7.1 (1975), págs. 1-13. DOI: 10.1016/S0020-7373(75)80002-2.

[26] Rizka Zuliani Musriroh. «Application of Mamdani Method on Fuzzy Logic to Decision of Traffic Light System». En: *Proceedings of International Conference on Informatics and Computational Sciences*. SciTePress, 2020, pág. 85200. DOI: 10.5220/0008520000000000.

[27] Costas P. Pappis y Ebrahim H. Mamdani. «A Fuzzy Logic Controller for a Traffic Junction». En: *IEEE Transactions on Systems, Man, and Cybernetics* 7.10 (1977), págs. 707-717. DOI: 10.1109/TSMC.1977.4309605.

[28] Timothy J. Ross. *Fuzzy Logic with Engineering Applications*. 3rd. Chichester, UK: John Wiley & Sons, 2010.

[29] Gökhan Erdinç. «Application of a Mamdani-Based Fuzzy Traffic State Identifier». Tesis sobre identificación de estado de tráfico usando lógica difusa Mamdani. Thesis. Università "La Sapienza"di Roma, 2023. URL: https://iris.uniroma1.it/retrieve/d51daa72-738b-40a8-b0f8-2f25473ebf1b/Erdin%C3%A7_Application-Mamdani-based_2023.pdf.

[30] Baher Abdulhai, Rob Pringle y Grigoris J. Karakoulas. «Reinforcement Learning for True Adaptive Traffic Signal Control». En: *Journal of Transportation Engineering* 129.3 (2003). Q-learning aplicado a control semafórico, págs. 278-285. DOI: 10.1061/(ASCE)0733-947X(2003)129:3(278).

[31] Seung-Bae Cools, Carlos Gershenson y Bart D'Hooghe. «Self-Organizing Traffic Lights: A Realistic Simulation». En: *Advances in Applied Self-Organizing Systems*. Sistemas autoorganizados para control semafórico. Springer, 2013, págs. 45-55. DOI: 10.1007/978-1-4471-5113-5_3.

[32] Leandros Tassiulas y Anthony Ephremides. «Stability Properties of Constrained Queueing Systems and Scheduling Policies for Maximum Throughput in Multihop Radio Networks». En: *IEEE Transactions on Automatic Control* 37.12 (1992), págs. 1936-1948. DOI: 10.1109/9.182479.

[33] Pravin Varaiya. «Max Pressure Control of a Network of Signalized Intersections». En: *Transportation Research Part C: Emerging Technologies* 36 (2013), págs. 177-195. DOI: 10.1016/j.trc.2013.08.014.

[34] Anastasios Kouvelas, Jennie Lioris, S. Alireza Fayazi y Pravin Varaiya. «Maximum Pressure Controller for Stabilizing Queues in Signalized Arterial Networks». En: *Transportation Research Record* 2421.1 (2014), págs. 133-141. DOI: 10.3141/2421-15.

[35] S. Erdogan, Bashar Al-Omari y Hesham Rakha. «Signal Timing Optimization for Coordinated Arterials: A Review of Progression-Based Methods». En: *Journal of Transportation Engineering* 139.4 (2013), págs. 358-370. DOI: 10.1061/(ASCE)TE.1943-5436.0000506.

[36] Pablo Alvarez López, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner y Evamarie Wießner. «Microscopic Traffic Simulation using SUMO». En: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, págs. 2575-2582. DOI: 10.1109/ITSC.2018.8569938.

[37] Pradeep Kumar Singh, Roshan Singh, Sukumar Nandi y Sukumar Nandi. «Managing Smart Traffic Lights Using Fog Computing and Blockchain». En: *IEEE Internet of Things Journal* 7.7 (2020). Blockchain aplicado a gestión de semáforos inteligentes, págs. 6243-6252. DOI: 10.1109/JIOT.2020.2968410.

[38] Ye Yuan y Fei-Yue Wang. «Blockchain and Cryptocurrencies: Model, Techniques, and Applications». En: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 48.9 (2018). Aplicaciones de blockchain en sistemas ciber-físicos, págs. 1421-1428. DOI: 10.1109/TSMC.2018.2854904.

[39] Mauro Dell'Orco, Ozgur Baskan y Mario Marinelli. «A Harmony Search Algorithm Approach for Optimizing Traffic Signal Timings». En: *PROMET - Traffic & Transportation* 25.4 (2013). Metaheurísticas aplicadas a optimización semafórica, págs. 349-358. DOI: 10.7307/ptt.v25i4.1245.

[40] Vahid Azimirad, Naser Pariz y Saeed Balochian. «A Novel Fuzzy Model and Control of Single Intersection at Urban Traffic Network». En: *IEEE Systems Journal* 4.1 (2010). Modelo difuso para intersecciones urbanas, págs. 107-111. DOI: 10.1109/JSYST.2010.2043159.

[41] Sunil Kumar Yadav y Richa Sharma. «Fuzzy Logic Based Traffic Light Controller for Heterogeneous Traffic». En: *International Journal of Advanced Research in Computer Science* 12.3 (2021). Control difuso para tráfico heterogéneo, págs. 1-5.

[42] Nazri Mohd Nawi, Azlan Khan y M. Z. Rehman. «A New Levenberg-Marquardt based Back Propagation Algorithm Trained with Cuckoo Search». En: *Procedia Technology* 11 (2013). Optimización de sistemas neuro-fuzzy, págs. 18-23.

[43] Joe H. Ward. «Hierarchical Grouping to Optimize an Objective Function». En: *Journal of the American Statistical Association* 58.301 (1963), págs. 236-244. DOI: 10.1080/01621459.1963.10500845.

[44] DLR Institute of Transportation Systems. *SUMO User Documentation*. Consultado: 19 de julio de 2026. Eclipse SUMO. 2025. URL: https://sumo.dlr.de/docs/.

[45] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Consultado: 19 de julio de 2026. 2008. URL: https://bitcoin.org/bitcoin.pdf.

[46] Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller y Steven Goldfeder. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton University Press, 2016. ISBN: 978-0-691-17169-2.

[47] Vitalik Buterin. «Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform». En: (2014). Whitepaper, Consultado: 19 de julio de 2026. URL: https://ethereum.org/en/whitepaper/.

[48] Juan Garay, Aggelos Kiayias y Nikos Leonardos. «The Bitcoin Backbone Protocol: Analysis and Applications». En: *Proceedings of the 19th International Conference on Principles of Distributed Systems (OPODIS 2015)*. Consultado: 19 de julio de 2026. 2015, págs. 1-15. URL: https://opodis2015.irisa.fr/files/2016/01/Garay_OPODIS_2015.pdf.

[49] Qiang Wang, Jiajing Yu, Shuo Chen y Yong Xiang. «SoK: DAG-based Blockchain Systems». En: *ACM Computing Surveys* 55.12 (2023), Article 256. DOI: 10.1145/3576899.

[50] X. Zhang et al. «SoK: DAG-based Consensus Protocols». En: *arXiv preprint* (2025). Systematization of Knowledge sobre protocolos de consenso basados en DAG. arXiv: 2411.10026 [cs.CR]. URL: https://arxiv.org/abs/2411.10026.

[51] IPFS Project. *IPFS Architecture*. Especificación oficial de arquitectura IPFS, define formalmente nodeID := multihash(publicKey) y esquema de identidad basado en PKI. 2024. URL: https://github.com/ipfs/specs/blob/main/ARCHITECTURE.md.

[52] Lotfi A. Zadeh. «Similarity Relations and Fuzzy Orderings». En: *Information Sciences* 3.2 (1971), págs. 177-200. DOI: 10.1016/S0020-0255(71)80005-1.

[53] Russell C. Eberhart y James Kennedy. «A New Optimizer Using Particle Swarm Theory». En: *Proceedings of the Sixth International Symposium on Micro Machine and Human Science (MHS '95)*. IEEE, 1995, págs. 39-43. DOI: 10.1109/MHS.1995.494215.

[54] Maurice Clerc y James Kennedy. «The Particle Swarm – Explosion, Stability, and Convergence in a Multidimensional Complex Space». En: *IEEE Transactions on Evolutionary Computation* 6.1 (2002), págs. 58-73. DOI: 10.1109/4235.985692.

[55] Yuhui Shi y Russell Eberhart. «Parameter Selection in Particle Swarm Optimization». En: (1998). Proposed inertia weight adaptation, págs. 591-600.

[56] Riccardo Poli, James Kennedy y Tim Blackwell. «Particle Swarm Optimization: An Overview». En: *Swarm Intelligence* 1.1 (2007), págs. 33-57. DOI: 10.1007/s11721-007-0002-0.

[57] Marco Conoscenti, Antonio Vetro y Juan Carlos De Martin. «Blockchain for the Internet of Things: A Survey». En: *2016 IEEE 19th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 2016, págs. 1-6. DOI: 10.1109/SNPD.2016.7515925.

[58] Florian Eckermann, Hendrik Beckmann, Benjamin Hagedorn y Marcus Kaulartz. «Blockchain Technology in the Energy Sector: A Systematic Review of Challenges and Opportunities». En: *Renewable and Sustainable Energy Reviews* 100 (2019). Análisis de costos y limitaciones de blockchain, págs. 143-174. DOI: 10.1016/j.rser.2018.10.014.